Chapter 5: System Implementation, Testing and Evaluation

5.1 Introduction

Chapter 5 at a glance. What CAMR is. A language model writes text by predicting the next word, and it only knows what it absorbed in training. Models small enough for a laptop know little. CAMR is a memory placed beside the model, like a notebook in an open-book exam: documents are filed as short notes, and for each question CAMR finds the few notes that matter and gives the model only those. The model itself is never changed.

What was tested. 22 models from 0.5 to 14.8 billion parameters, alone and with CAMR; CAMR against eight designs that industry uses to give models documents (RAG); three business automations (a support desk, an agent that does office tasks, and a daily news digest); and what it all costs in time and money.

What was found.

- With memory, a small laptop model knows as much as a large cloud model about less-known facts, from about 1.5 billion parameters upwards (20 of 22 models matched or beat the cloud model).
- Careful answering works. Alone, the smallest model stated a wrong answer for 88.5% of questions; with CAMR for 3.0%, while getting 60.5% right. The cloud model alone got 67.5% right but stated a wrong answer 31.5% of the time.
- Against today's standard (RAG): with a small model CAMR was better on both counts (62.5% right against 45.5%; 3.5% wrong against 47.0%). With the cloud model RAG answered more (71.5% against 63.0%) but stated about 5 times as many wrong answers.
- In business automations, memory made the cloud model useful: alone it handed 96.7% of support tickets to a person; with CAMR it resolved 100.0% of the answerable ones and sent no wrong reply.
- Memory does not fix reasoning: for maths, a stronger model or a calculator is needed.
- Bigger models are slow on a laptop: every model of 6 billion parameters or more answered more slowly than the cloud model.

How to read the numbers. Most results are percentages of a set of questions (usually 200). *Correct* means a right answer. *Wrong* means a wrong answer stated as fact, the harmful kind. *Declined* or *not confirmed* means the system said it did not know or passed the question on: safe, but not useful. A good system has many correct answers and very few wrong ones. Each section starts with a box like this one that says what it found in plain words; the glossary in this section explains the terms used throughout.

This chapter reports what was built, how it was tested, and what the measurements showed, including where the system failed. One idea runs through it: the language models are used **as downloaded and are never retrained**. Everything the system learns is kept outside the model, in a memory engine called CAMR (Capability-Adaptive Memory Retrieval). Only a

few small settings were tuned, always on practice questions, and two retrained models (LoRA, a cheap way of retraining) were built purely for comparison, to test the proposal's claim that memory beats retraining.

Four things were built. (1) The **memory engine**, which files documents as short notes in a single database file and decides, for each question, which notes a small model should read. (2) **Verified answering**, which states an answer as fact only when a stored note supports it, and otherwise says so or passes the question on. (3) **Learning without retraining**, by looking up what memory lacks and by keeping corrections. (4) **Applications**: CAMR Personal, a desktop assistant, and a gateway that lets business automations (built in n8n) and agents (models that carry out tasks with tools) use the engine.

Each was compared with alternatives: the same model without memory, a large cloud model (Kimi K3), eight popular ways of giving a model documents (retrieval-augmented generation, RAG), pasting every document into every question (context stuffing), and retraining. Every number in this chapter comes from a results file in the repository, and results are reported the way they came out. Section 5.9.1 opens the results with a scorecard that marks each claim as met, partly met, or not met, and section 5.9.17 lists every flaw found, with its cause.

Table 5.1: Where each part of the system is described and evaluated

Component	What it does	Implemented in	Evaluated in
Memory engine	Files documents as short notes; for each question, finds the notes that matter, keeps only those about the right subject, and fits them within a size limit	5.4	5.9.2, 5.9.6
Verified answering	States an answer only when a stored sentence supports it	5.4.4	5.9.3, 5.9.5
Learning without training	Looks up what memory lacks; learns formulas from corrections	5.4.5	5.9.8
Large data and device limits	Loads large document collections so they are searchable at once, and stops safely before the computer runs out of memory or disk	5.4.6	5.9.11
Learned routing	Decides for each question how much memory to read, or whether to ask the cloud model	5.4.7	5.9.12
CAMR Personal	Desktop assistant with memory	5.4.8	5.6.4, 5.9.13
Automation gateway	Lets business automations (n8n) and agents use CAMR or any other answering system in the same way	5.4.10	5.9.9, 5.9.10

Table 5.2 explains in plain words the terms used throughout the chapter. Each main section also explains a term briefly, in brackets, the first time it appears there, so a section can be read on its own.

Table 5.2: Key terms in plain words

Term	**What it means**
Language model	A program that writes text one word at a time by predicting what comes next. It only "knows" what was in the text it was trained on.
Parameters; 0.5B, 3B, 7B	The internal numbers a model learned in training. "3B" means about 3 billion of them. More parameters usually means a more capable model that is also slower and needs more memory.
Small (on-device) model	A model small enough to run on an ordinary laptop without the internet; here from 0.5B to 14.8B.
Cloud model; Kimi K3	A very large model run by a company and used over the internet, paid per use. Kimi K3, by Moonshot AI, is the strong reference model in this study.
Frozen model	A model whose parameters are never changed. Everything new is kept outside it, in CAMR's memory.
CAMR	The memory engine built in this project. It files documents as short notes and, for each question, decides which few notes the model should read.
Note; fact line	One short piece of stored text, usually one sentence. A fact line has the form "subject: relation: value" (*Mississippi Burning: Screenplay: Chris Gerolmo*) and can be read without the model.
Token	A piece of a word; models read and write tokens. 100 tokens are about 75 English words. Speed and cloud cost are counted in tokens.
Prompt	Everything handed to the model at once: instructions, any notes, and the question.
Reader	The model that reads the notes and writes the answer. It can be a small model or the cloud model.
Retrieval (search); embedding	Finding the notes most relevant to a question. An embedding is a list of numbers that captures what a text means, so a search can match meaning, not only words.
Keyword search (BM25)	Classic search that matches words, like a library catalogue.
RAG (retrieval-augmented generation)	Today's standard way to give a model documents: search for the passages closest to the question and paste them into the prompt. Eight versions were compared with CAMR.
Reranker	A second, more careful model that re-orders search results by how well each one answers the question.
Context stuffing	Pasting every document into every question. Simple, and accurate while the documents are few, but slow and expensive as they grow.
Fine-tuning; LoRA	Retraining a model on new examples so that knowledge goes into its parameters. LoRA is a cheap way of doing it. It is the alternative to memory that this study tests against.
Verified answer	CAMR states an answer as fact only when a stored note supports it; otherwise it says it cannot confirm the answer, or passes the question on.
Hallucination	A confident answer that is made up. The harmful kind of error.
Declined; not confirmed	The system said it did not know, or would not state an answer. Safe, but not useful.
Escalate	Hand a question to a person, or to a stronger model, instead of answering it.
Other meaning (named)	An answer about a different person or thing with the same name, labelled as such (*Happy (2011 film): ...*), so the reader can see the mix-up.
Single-hop; multi-hop	A single-hop question needs one fact. A multi-hop question needs two facts joined, for example *Where was the director of film X born?*
Long-tail facts	Facts about less famous people and things, which models rarely saw in training.
Benchmark	A public set of questions with known answers, so that results can be checked and compared: PopQA (less-known facts), HotpotQA and 2WikiMultiHopQA (questions that need two Wikipedia articles), GSM8K (school maths word problems).

Term	What it means
Development, test and held-out sets	Questions used while building (development), for checks during the work (test), and kept aside until the end for a fair final check (held-out).
Exact match; judged	An answer is first counted correct if it contains the reference answer. Answers marked wrong were re-checked by the cloud model acting as a judge, because a right answer can be worded differently.
Precision; recall	Precision: of the answers given, the share that were right. Recall (in search): the share of the needed evidence that the search found.
Agent	A model that works through a task step by step, choosing tools: search memory, calculate, write an invoice, reply, escalate.
n8n workflow; node	A business automation built visually from connected steps (nodes), for example: read a ticket, answer it, then send the reply or escalate.
Gateway	A small local web service that gives every answering system the same four commands (reset, teach, forget, ask), so a workflow can switch systems without other changes.
CPU, RAM, GPU	The computer's main processor, its working memory, and a graphics card. Everything here ran on a 4-core CPU with 16 GB of RAM and no GPU.
Quantisation (Q4_K_M)	Storing each parameter in about 4 bits, so a model needs less memory and runs faster with a small loss of quality.
Baseline	The system something is compared against.
Requirement (FR, NFR)	From Chapter 3: functional requirements (what the system must do) and non-functional requirements (how well: speed, memory, privacy).
Defect (D-01 ... D-40)	A bug found during testing or experiments, numbered, fixed and retested.
Reproducible	Anyone can rerun the evaluation from the saved files and get the same numbers.

The work was done in increments, each ending with a measured result that decided the next step. Figure 5.1 shows the sequence as recorded in the repository's history.

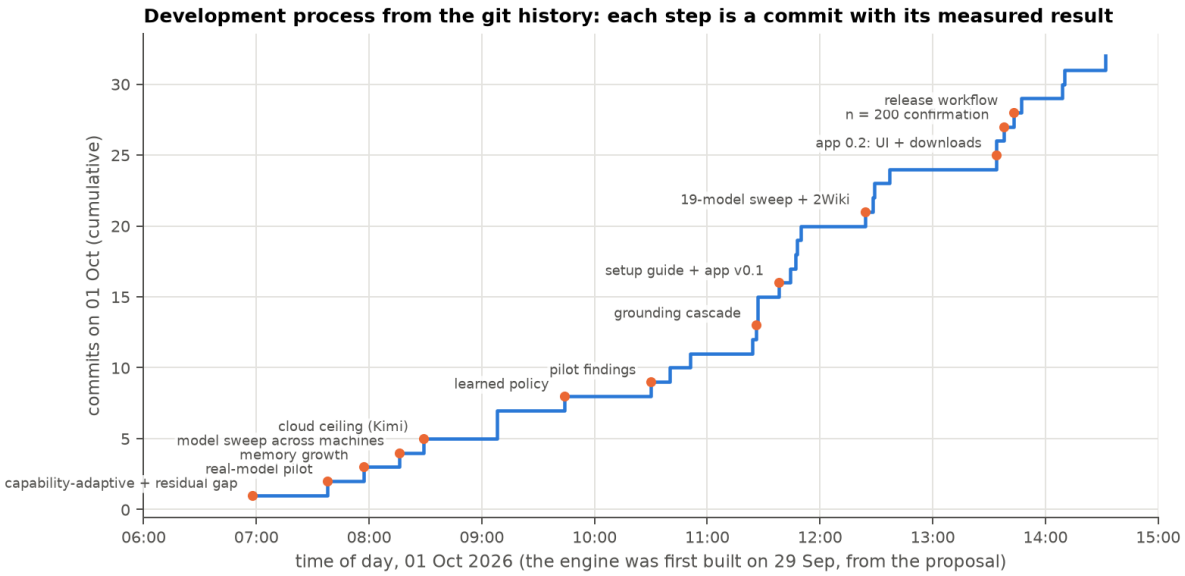


Figure 5.1: Development increments reconstructed from the git history; each marked step is a commit that added an implemented or measured increment

5.2 Implementation Environment

> **In plain words.** Everything ran on ordinary computers of the kind people already own: a 4-core processor, 16 GB of memory and no graphics card. That is deliberate, because the point is that a small model with memory should work on such a machine. The cloud model was used over the internet and paid per use.

Development, every experiment and every automation run took place on Linux machines with x86-64 CPUs and no GPU: four cores and 16 GB of memory, the consumer-laptop class that NFR-01 targets. The system is a local research prototype. CAMR Personal is distributed as downloadable files, but nothing was deployed as a public service. The n8n (a visual tool for building automations from connected steps) automations ran headless (from the command line, without the visual editor) on the same machine; no workflow was exposed to the internet.

5.2.1 Hardware and Deployment Environment

Table 5.3: Implementation environment and minimum requirements

Item or layer	Tested specification or version	Purpose and justification	Evidence
Primary machine (A)	Intel(R) Xeon(R) Processor @ 2.10GHz, 4 logical CPUs, 15.7 GB RAM (14.3 GB usable by processes), no GPU, Linux x86-64	All development; every experiment in this chapter except the 19-model sweep; the n8n automations	`results/pilot/tables/profile.json`
Sweep machines (B–E)	Same class: 4 vCPU Intel Xeon (2.10 GHz; machine C 2.80 GHz), 16 GB RAM	The 19-model sweep, one model family per machine, merged afterwards	FINDINGS F10
Minimum to run the engine	4 cores, 16 GB RAM. Engine process 712 MB peak; store 47.8 MB (pilot); 557 MB peak while bulk-ingesting 123,301 paragraphs	A 0.5–3B model and the engine fit with headroom on a 16 GB device	TC-14; `results/bulk/ingest.json`
Local models	Measured resident memory of the model runtime after loading: qwen2.5:0.5b 0.6 GB, qwen2.5:1.5b 1.3 GB, llama3.2:3b 2.8 GB, phi4-mini 3.6 GB, qwen2.5:7b 4.8 GB. A 7B model needs about 5 GB; a 20B model was killed for lack of memory until its weights were memory-mapped, then slowed to 0.4 tokens/s under pressure	Which model sizes a 16 GB device can run beside other software	`results/models/speed.json`; F6, F8
Cloud model	Moonshot Kimi K3 (and K2.6 in the pilot) over an OpenAI-compatible HTTPS API	Frontier reference, escalation target, second reader, judge; never required to run CAMR	`results/costs/kimi_costs.json`
End-user devices (CAMR Personal)	Windows 10/11 (x64, ARM64), macOS (Apple Silicon, Intel), Linux (x64, ARM64) with Ollama; ≥ 4 GB RAM for a	The application talks only to Ollama on the same machine	CI build and launch test on all six (Table 5.19)

Item or layer	Tested specification or version	Purpose and justification	Evidence
	1.5B model		
Specialised hardware	None: no GPU, NPU or accelerator anywhere	The study targets CPU-only consumer devices; every timing is a CPU timing	Profiles; CI runner labels

5.2.2 Software, Services and Configuration

Table 5.4: Software and services used (versions as tested)

Component	Version	Role in the system
Python	3.11.15	Engine, harness, gateway, experiment scripts
Ollama	0.35.0	Local model runtime on the loopback interface; quantised GGUF weights (Q4_K_M)
Local models	qwen2.5 0.5B / 1.5B / 7B, llama3.2 3B, phi4-mini 3.8B (+14 more in the sweep)	Frozen readers; greedy decoding, temperature 0, seed 13
SQLite / sqlite-vec	3.45.1 / 0.1.9	One-file store: notes, vectors (exact kNN), provenance, keyword index (FTS5), run logs
sentence-transformers / PyTorch (CPU)	6.1.0 / 2.14.1	BGE-small-en-v1.5 embedder (384-d); cross-encoder ms-marco-MiniLM-L-6-v2 for the RAG baselines
transformers / PEFT	5.19.0 / 0.21.2	The two LoRA fine-tuning baselines (CPU, fp32)
n8n / Node.js	2.43.6 / 24.22.0	Workflow automation: support desk, agent tasks, live news workflows, run headless with `n8n execute`
Moonshot API	kimi-k3 (list price $3 per million input tokens, $15 per million output tokens)	Cloud reference; the key is read from the `MOONSHOT_API_KEY` environment variable only
Streamlit, Playwright, axe-core	1.64.0, 1.63.0, 4.10	Read-only inspector; browser tests, screenshots and the accessibility audit
pytest	9.1.1	203 automated tests (section 5.6)

Each experiment is configured by one YAML file loaded into typed dataclasses that reject unknown keys, so a misspelt setting fails instead of silently using a default (NFR-07). Every run records the hash of its full configuration and is resumable. Docker was not used: the experiments ran directly on the hosts, and the application ships as one self-contained executable per platform. No password, key or personal data appears in the repository, the logs or any screenshot. The cloud key existed only in an environment variable, and it is to be rotated after the study.

5.2.3 Deployment and Reproducibility

Everything in this chapter can be rerun from the repository. The commands below were run as shown on machine A. Only the Kimi conditions need the cloud key; every other step runs offline once the models and data are downloaded.

```
git clone https://github.com/Satasdin/CAMR.git && cd CAMR
python -m venv .venv && source .venv/bin/activate
pip install -e ".[vec,embed,dev,inspect]"
python -m pytest -q                                    # health check: 203
tests, no network
ollama pull qwen2.5:0.5b && ollama pull qwen2.5:1.5b && ollama pull
llama3.2:3b
python scripts/fetch_data.py                           # PopQA, HotpotQA,
2Wiki, GSM8K (prints a hash per file)
export MOONSHOT_API_KEY=...                            # only for the Kimi
conditions
```

One command re-checks the key results. `scripts/reproduce.py` recomputes them from the frozen caches (every web response and model answer is stored by its exact request) and compares every number with the committed one; it exits with status 0 only if all match. It needs no key and no network.

```
python scripts/reproduce.py --twice                  # 8 checks, run twice:
16/16 must pass
```

The experiments of this chapter, each resumable:

```
python scripts/verified_qa.py answer --set heldout     # verified answering
(F24); judge, report
python scripts/rag_bench.py run --reader qwen2.5:0.5b  # nine RAG pipelines
and CAMR (F28)
python scripts/math_learning.py run --model qwen2.5:0.5b --mode calc   #
formulas from corrections (F27)
python scripts/large_data.py corpus && python scripts/large_data.py ingest
&& python scripts/large_data.py limits
python scripts/agentic/gateway.py &                    # automation gateway
on 127.0.0.1:8765
python scripts/agentic/n8n_workflows.py run --systems camr:kimi-k3   # n8n
support desk (F29)
python scripts/kimi_costs.py                           # US dollars spent on
the cloud model
python scripts/make_ch5_figures.py && python scripts/build_ch5.py --template
proposal.docx
```

The application only: download the file for the platform from the release page, or `pip install "camr[app] @ git+https://github.com/Satasdin/CAMR.git"`, then `camr app`, which opens http://127.0.0.1:8502.

5.3 Dataset Description

> **In plain words.** Results only mean something if the questions are fair. Most questions come from public research sets with known answers, so that anyone can check the results: less-known facts (PopQA), questions that need two Wikipedia articles (HotpotQA, 2WikiMultiHopQA) and school maths problems (GSM8K). Questions used to build and tune the system were kept apart from those used for the final test. The automations used an invented company (Helio Solar) and 30 days of real Wikipedia news pages.

Two kinds of data were used: public benchmarks with reference answers, and data built for this study where no benchmark could test the question. Nothing in either is generated by the system under test. Table 5.5 lists the public benchmarks and Table 5.6 the data built for the study.

5.3.1 Public Benchmarks

Table 5.5: Public benchmarks

Benchmark	Source and licence	Size used	Task	Metric
PopQA	Mallen et al. (2023); templates over Wikidata facts	14,267 questions; three sets of 200 drawn for this study (below)	Single-hop, long-tail facts	Answer contained; judged
HotpotQA (dev, distractor)	Yang et al. (2018); crowd-written over Wikipedia; CC BY-SA	7,405 questions, 10 paragraphs each	Multi-hop	Exact match, F1
2WikiMultiHopQA (dev)	Ho et al. (2020); Wikipedia and Wikidata	12,576 questions	Multi-hop (retrieval)	Full-support recall
GSM8K	Cobbe et al. (2021); human-written; MIT	1,319 test; 7,473 train	Arithmetic reasoning (control)	Final-number accuracy

Table 5.6: Data built for this study

Data	What one record is	Size	Why it was needed
PopQA development, test and held-out sets	A PopQA question; ids and subjects disjoint across sets	200 + 200 + 200 questions	Choose methods on one set, test once on another (section 5.3.2)
Frozen web cache	Every page the lookups read, stored by exact request	972 pages	The web changes; a frozen copy makes every lookup replayable
RAG benchmark corpus	The cached pages as notes: lead sentences and one note per infobox fact	9,989 documents → 10,443 notes, 46.9 MB	One store that every RAG pipeline and CAMR search
Formula families	A problem from one of ten formula families; four are invented (zorbit, glim, Kessler fee, Vantor score)	10 × (6 train + 6 held-out + 6 reworded + 4 derived)	Test learning from corrections on knowledge no model can have
Helio Solar (invented company)	A product sheet or policy; a customer ticket with its gold answer; an operations task with checks on the tool calls	14 documents, 3 policy changes, 60 tickets over 8 days, 20 agent tasks	Automations need private, changing knowledge that no model was trained on
Wikipedia Current events	One news item (a sentence) from a daily page; a question whose answer is copied from it	555 items over 30 days (2026-09-15 to 2026-10-14); 50 questions	Real events after every model's training data
Bulk corpus	Every paragraph of HotpotQA dev and 2Wiki dev, deduplicated	123,301 paragraphs → 195,735 notes	Large-data ingest and retrieval at scale

The Helio tickets were written in four kinds, in the proportions a support desk meets: answerable from the knowledge base (26), answerable only after a human agent (a model that

completes a task step by step using tools) has resolved the question once and the resolution has been taught (18), affected by a policy change (7), and unanswerable from anything the company published (9). The news questions were written by Kimi K3 (a large cloud model by Moonshot AI) from single items, with a rule that the answer must be copied exactly from the item; 50 of the first 100 sampled items passed it, at a cost of $0.24. Table 5.7 shows one record of each kind, verbatim.

Table 5.7: One record of each kind (verbatim)

Data	**Record**	**Reference answer**
PopQA (held-out)	Who was the screenwriter for Mississippi Burning?	Chris Gerolmo
HotpotQA	Were Scott Derrickson and Ed Wood of the same nationality?	yes
GSM8K	Janet's ducks lay 16 eggs per day. She eats three for breakfast … sells the remainder … for $2 per fresh duck egg. How much in dollars does she make every day?	18
Formula (invented, derived)	A Kessler fee of $30.20 was charged on an order of 6 items. What was the order value? (Kessler fee = 4% of the order plus $2.50 per item)	380
Helio ticket (learned)	Which cable connects the Lumen LB-13 battery?	CB-3 (taught after the first time a person answered it)
Helio agent task	Customer Amina Odhiambo orders 12 HS-400 panels with installation. Create her invoice.	Invoice total $5,640
Current events	How many civilians were killed during an armed attack by unknown individuals in Madegoua, Far North Region, Cameroon?	Fifteen (from the item of 2026-09-17)

5.3.2 Data Preparation and Partitioning

Draws. Every sample was drawn once from a fixed seed after sorting by question id, and saved with the hash of the source file, so every condition answered the same questions (paired comparisons). The PopQA (public questions about less-known facts) development set (200) was used to choose every method of verified answering; the test set (200) is the learning stream of the earlier increment; the **held-out set** (200) was drawn only after every choice was made, and answered once. The code asserts that ids and subjects do not overlap between the three. The formula problems come from two seeds: seed 31 for development and seed 47 for the reported test, with no problem shared.

Leakage guards. Memory was built from documents only, never from question–answer records, and a guard refused any document containing an evaluation question verbatim (DR-04). Thresholds of the RAG baselines (abstention, corrective RAG) were chosen on the development questions only, then applied to the held-out (kept aside until the end for a fair test) questions. The cross-encoder and embedder (the model that turns text into meaning-numbers for search) were used as published, without training.

Frozen external data. Web lookups were made once (`fetch`) and stored by exact request; answering never went online. The first reproducibility run caught answer runs that had fetched pages an earlier run had missed, so two variants saw slightly different pages; freezing fixed this. Wikipedia's Current events pages were fetched live by the n8n (a visual tool for building automations from connected steps) workflow and also cached, so the questions refer to exactly the text the systems were taught.

Preparation of a record. A document is normalised (Unicode NFKC, whitespace), split into notes with its title as a prefix, and each infobox (the summary box on a Wikipedia page) line becomes its own note (*Directed by: Walter West*). Table 5.8 follows one record from the raw files to what the model reads.

Table 5.8: One PopQA record before and after preparation

Stage	Content
Raw question	id 5939255 · subject *The Loudwater Mystery* · property *director* · answers ["Walter West", "Walter Alabaster West"]
Raw document	*The Loudwater Mystery is a 1921 British silent crime film directed by Walter West and starring Gregory Scott, Pauline Peters and Clive Brook …*
Stored note (pilot)	*The Loudwater Mystery (film): The Loudwater Mystery is a 1921 British silent crime film directed by Walter West …* (47 tokens, 384-d vector, provenance). Pages read by the later lookups also give one note per infobox line, e.g. *Directed by: Walter West*
Read at question time	Best similarity 0.77, next 0.62: only one note admitted (47 tokens instead of a 512-token budget)
Answer	0.5B alone: *John Sturges* (wrong). 0.5B with CAMR: *Walter West* (correct)

5.4 System Implementation

> **In plain words.** Think of CAMR as a helper in an open-book exam. When documents arrive, it cuts them into short notes and files them (the *write path*). When a question arrives, it finds the few notes that matter, checks that they are about the right subject, and gives the model only those, within a size limit (the *read path*). If the answer can be read straight from a note, the model is not asked at all, and an answer is stated only if a stored note supports it.

The implementation follows the package structure of Chapter 4: `camr.memory` (the engine, verification, learning, bulk ingest), `camr.models` (runners and prompts), `camr.harness` and `camr.eval` (experiments and scoring), `camr.learn` (the learned policy), `camr.device` (the governor), `camr.app` (CAMR Personal), and `scripts/agentic` (the automation gateway). Figure 5.2 shows how they fit together. A test enforces the package rules: the engine imports nothing from the harness, the interfaces or the scripts.

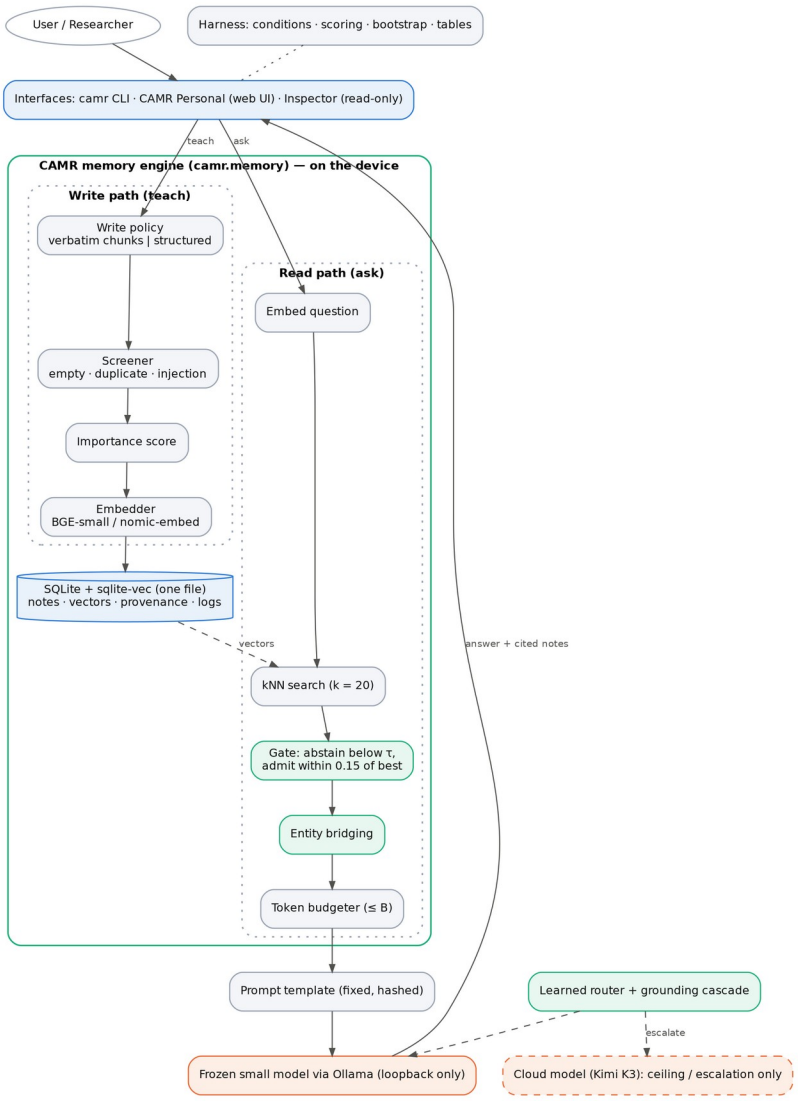


Figure 5.2: Implemented architecture: write path, one-file store, adaptive read path (green), frozen local model and optional cloud model

5.4.1 Write Path: From Documents to Notes

Ingestion turns a document into notes, screens them, scores their importance, embeds them and commits note, vector and provenance (a record of where each note came from) in one SQLite (a database kept in one ordinary file) transaction, so the store never holds a note without its vector or its source (FR-01–FR-05). The screener rejects empty, degenerate, duplicate, over-length and injection-like text and logs why. Duplicates are caught by a checksum under a UNIQUE constraint. Two additions came from the evaluation. Each line of an infobox (*Directed by: Walter West*) became a note of its own, because a short labelled fact can be read without a model (section 5.4.4). And when a large file arrives, notes are stored at once without vectors, findable through a keyword index, and their vectors are added in the background (section 5.4.6).

5.4.2 Read Path: Deciding How Much Memory a Model Reads

For each question the engine embeds it, finds the 20 nearest notes by exact search, and then applies the capability-adaptive steps that distinguish it from plain retrieval. Each step is switchable in configuration, and each was kept or dropped on measured evidence:

i. **Gating** abstains when even the best note is weak, and admits only notes close to the best one. It keeps irrelevant text away from a small reader.
ii. **Entity gate** abstains when the question names something no admitted note mentions. Similarity alone cannot tell a note about *this* film from a note about a similar one.
iii. **Entity bridging** follows a name mentioned in an admitted note to that entity's own note: the second hop of a multi-hop (needing two facts joined) question, without building a graph.
iv. **One note per source** (optional) admits at most one note per source title, so six near-copies of one correction do not fill the context.
v. **Hybrid order** (large stores) adds keyword (BM25) hits to the vector hits and lifts exact word matches.
vi. **The token budget** packs notes in rank order until the next would exceed B tokens (pieces of words; 100 tokens are about 75 words).

5.4.3 Formal Model of the Read Path

> **In plain words.** The equations below say in symbols what the read path does in words: every note gets a score for how well it matches the question, notes about the wrong subject are dropped, and the best notes are kept until the size limit is reached. A reader who prefers words can skip to the next section without losing the thread.

This subsection states precisely what the engine computes for a question q, so that every setting tuned or reported later has an exact meaning. The notation follows Chapter 4. The store holds notes n, each with its text, a token count $c(n)$, and an embedding (numbers that capture a text's meaning, used to search by meaning) produced by the embedder (the model that turns text into meaning-numbers for search) f. Embeddings are normalised to unit length, so cosine similarity is a dot product:

$$s(q,n)=\cos\big(f(q),f(n)\big)=\frac{f(q)\cdot f(n)}{\|f(q)\|\,\|f(n)\|}=f(q)\cdot f(n) \tag{5.1}$$

where

$f(\cdot) \in \mathbb{R}^{384}$ the embedder (BGE-small-en-v1.5 in the experiments; nomic-embed-text in the application), with the model card's query prefix on *q*;

$s(q,n) \in [-1,1]$ the similarity of note *n* to the question.

The candidate set is the *k* most similar notes, found by exact nearest-neighbour search in sqlite-vec (an add-on that lets the database search by meaning):

$$C_k(q) = \arg\max_{S \subseteq \mathcal{M}, |S|=k} \sum_{n \in S} s(q,n), k = 20 \tag{5.2}$$

The proposal ranked candidates by a weighted composite of similarity, recency and importance (Equation 4.1). As implemented, each term is min-max normalised over the candidate set:

$$\text{score}(n) = w_s \tilde{s}(q,n) + w_r \rho^{\Delta t(n)} + w_i \tilde{I}(n), w_s + w_r + w_i = 1 \tag{5.3}$$

where

$\tilde{s}, \tilde{I}$ min-max normalised similarity and importance;

$\rho = 0.99$ the recency decay per hour of logical time since the note was last read;

(w_s, w_r, w_i) (0.6, 0.15, 0.25) in the proposal's configuration, and (1, 0, 0) in the configuration the evidence selected (section 5.9.2), which reduces Equation 5.3 to ranking by similarity.

Gating is the first capability-adaptive step. Let $s_{(1)}$ be the best similarity in the candidate set. The engine abstains (gives the model no notes) when even the best note is weak; otherwise it keeps only notes within a margin δ of the best:

$$E(q) = \begin{cases} \varnothing & \text{if } s_{(1)} < \tau \\ \{n \in C_k(q) : s(q,n) \geq s_{(1)} - \delta\} & \text{otherwise} \end{cases} \tag{5.4}$$

where

τ the abstain threshold (0.50 for BGE, 0.55 for nomic-embed-text; calibrated in section 5.5.2);

δ the admission margin (0.15).

A second, optional gate (a check that drops notes about the wrong subject) was added after the comparison with standard RAG (section 5.9.5) showed that similarity cannot tell a note about *this* entity (a named person, place or thing) from a note about a similar one. Let *N(q)* be the names the question mentions (runs of capitalised words and quoted titles, `camr.memory.entities`). If the question names something and no eligible note mentions it, the engine abstains:

$$E'(q)=\begin{cases}\varnothing & \text{if } N(q)\neq\varnothing \text{ and } N(q)\cap \text{names}(n)=\varnothing \ \forall\, n\in E(q)\\ E(q) & \text{otherwise}\end{cases} \tag{5.5}$$

Questions that name nothing (*when is my dentist appointment?*) are unaffected. The gate is off in the benchmark runs reported before section 5.9.5 and on in the released application.

Entity bridging (following a name found in one note to another note about that name) then adds, for each of the top three eligible seed (the first note found, from which bridging continues) notes, up to two notes about other stored entities whose titles the seed's text mentions. These notes may have low similarity to the question; they are reached through the seed, which is the second hop of a multi-hop question:

$$B(q)=\left|m\in\mathcal{M}:\text{title}(m)\in\text{mentions}(n) \text{ for some } n\in\text{top}_3 E(q)\right|,\leq 2 \text{ per } n \tag{5.6}$$

Finally the budgeter admits notes in rank order, each bridged note directly after its seed, until the next note would exceed the token budget (a cap on how much text the model reads) *B*, and then stops:

$$A(q)=(n_1,\ldots,n_m),m=\max\left\{j:\sum_{i=1}^{j}c(n_i)\leq B\right\} \tag{5.7}$$

The prompt (the full text handed to the model) contains exactly the notes in *A(q)*, so the number of context tokens is at most *B* and is usually far smaller, because gating removes candidates before the budget binds.

Worked example (from the pilot's logs). For *Who was the director of The Loudwater Mystery?* the 20 candidates had best similarity $s_{(1)}$ = 0.771 (the film's own Wikipedia note). Since 0.771 ≥ τ = 0.50, the engine did not abstain. The admission floor was $s_{(1)} - \delta$ = 0.771 − 0.15 = 0.621, and the next-best note (*Klute*, 0.616) fell just below it, so only one note was eligible. That note was 47 tokens, so Equation 5.7 admitted it and the model read 47 tokens instead of filling the 512-token budget. The 0.5B model answered *Walter West* (correct); alone it had answered *John Sturges*. Figure 5.3 draws this decision from the logged scores.

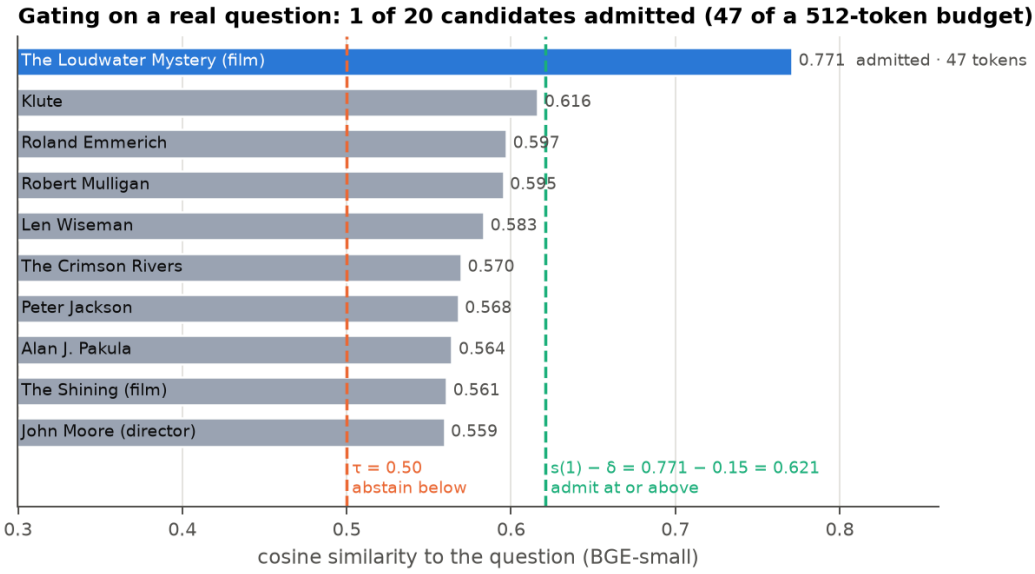


Figure 5.3: Gating on one real question: candidate similarities, the abstain threshold τ and the admission floor s(1) − δ; only the blue note was given to the model

Hybrid order, used for large stores, ranks by similarity plus a bounded bonus for the keyword match, scaled to the best match among the candidates; the gate still uses the plain similarity:

$$h(q,n)=s(q,n)+\beta\frac{\mathrm{bm25}(q,n)}{\max\limits_{m\in C(q)}\mathrm{bm25}(q,m)},\beta=0.10 \tag{5.8}$$

5.4.4 Verified Answering

> **In plain words.** An answer is stated as fact only if all of its words appear in one stored note, and that note is about the right subject and the right kind of fact. Otherwise CAMR says it cannot confirm the answer, or hands the question on.

A small model that reads the right note can still answer wrongly, and a model that reads nothing will answer anyway. Verified answering removes the second failure and exposes the first: an answer is stated as fact only if a stored sentence supports it. Figure 5.4 shows the flow.

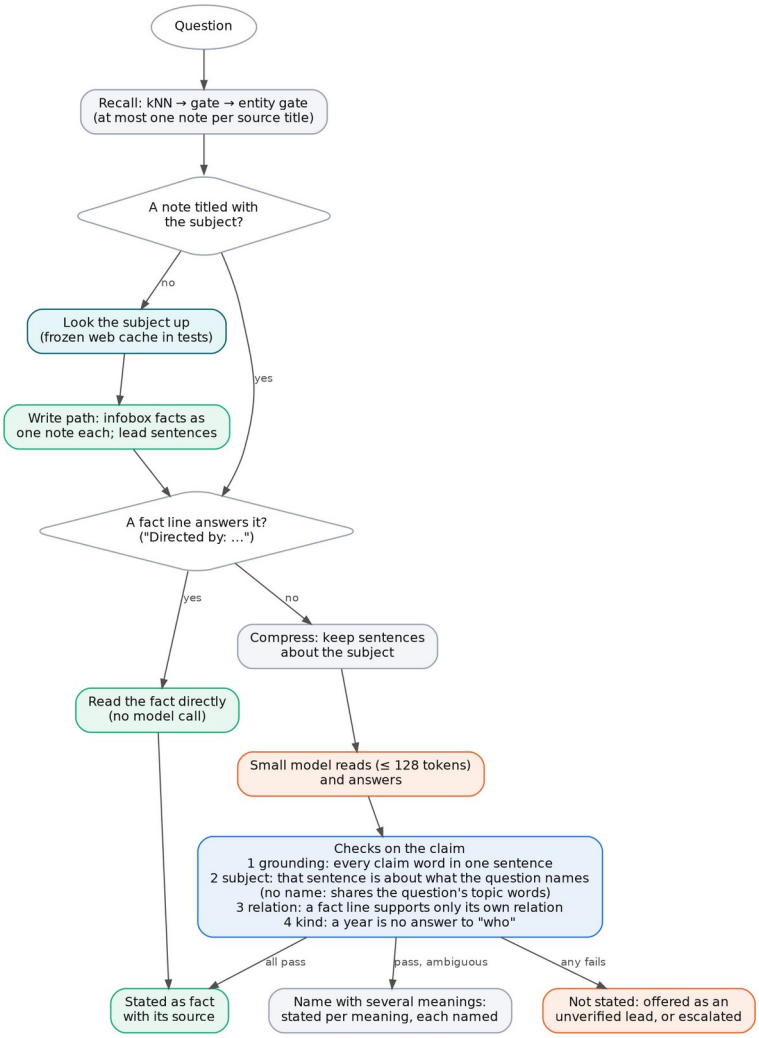


Figure 5.4: Verified answering: the fact reader answers without a model when a labelled fact matches; otherwise the model reads and four checks decide whether its answer is stated

Let *a* be an answer, *w(a)* its claim words (the words of its first sentence that are not in the question and are not framing words such as *directed by*), and *U* the units (sentences and fact lines) memory read. The answer is stated only when one unit satisfies all four checks:

$$V(a,q) = \exists\, u \in U : w(a) \subseteq \mathrm{tok}(u) \wedge \mathrm{about}(u,q) \wedge \mathrm{rel}(u,q) \wedge \mathrm{kind}(a,q) \qquad (5.9)$$

where

$w(a) \subseteq \mathrm{tok}(u)$ grounding: every claim word occurs in that one unit;
$\mathrm{about}(u,q)$ subject: the unit is about what the question names (titled with the name, or mentions it); when memory holds a note titled with the name, only such notes count;
$\mathrm{rel}(u,q)$ relation: a labelled fact supports only its own relation (*Directed by* is no evidence for *producer*), and a plain sentence must state the asked relation;
$\mathrm{kind}(a,q)$ the answer is the kind asked for (a year is no answer to *who*).

A question that names nothing (*What hours is the support desk open?*) cannot pass the subject check, and the first automation runs showed this withheld correct answers (defect D-33, section 5.9.9). Such a question is now anchored by its topic words *T(q)*, its content words of three or more letters other than question words, matched on their first five letters. The rule applies only when the question names no entity, asks no known relation, and no note is about its lookup phrase; every PopQA (public questions about less-known facts) question asks a known relation, so the benchmark results are untouched (a first version also applied it to *What is the religion of refuge?* and let a wrong answer through; the replay check caught it):

$$\text{about}(u,q)=1\left[\left|T(q)\cap \text{tok}_5(u)\right|\geq\frac{1}{2}\left|T(q)\right|\right]\text{when } q \text{ names nothing and asks no known relation} \tag{5.10}$$

Three further rules complete the answerer. **The fact reader**: when a fact line (a stored 'subject: relation: value' line read without the model) about the subject carries the asked relation, its value is the answer and no model is called. **Knowing a subject**: memory counts as knowing a subject only if a note is titled with it; a note that merely mentions it does not stop a lookup (this round-2 change added 3.5 points on the development set). **Several meanings**: a name that means several things is never resolved silently; the answer names the meaning it is about, or gives each. What fails the checks is offered as an unverified lead, or, in an automation, handed to a person.

Worked examples (held-out set, Qwen 0.5B). *Who was the screenwriter for Mississippi Burning?* Memory held the film's page with the fact line *Written by: Chris Gerolmo*. The asked relation matched the label, so the fact reader answered *Chris Gerolmo* with no model call, and the answer was stated with that line as its source. *Who is the author of The Disk?* has no labelled author, so the model read the compressed lead and answered *Jorge Luis Borges is the author of "The Disk"*; the claim words occur in the sentence *"The Disk" is a 1975 short story written by Argentine writer Jorge Luis Borges*, which is in the note titled *The Disk* and states the relation (*written by*), so all four checks passed. The checks also fail correct answers: *al-Ghazali* for *Who is the author of The Incoherence of the Philosophers?* was right, but the kind check expects a person's name to start with a capital letter, so it was withheld (section 5.9.17).

5.4.5 Learning Without Training

> **In plain words.** Two ways of learning without retraining the model: looking up what memory lacks and keeping it, and keeping corrections as worked examples to copy the next time.

Two mechanisms let a frozen model gain knowledge as it is used. Neither changes a weight.

Looking up what memory lacks (eager learning). When recall (the share of the needed evidence that the search found) finds nothing about the question's subject and the question is not about the user, the engine looks the subject up (DuckDuckGo, then the Wikipedia lead and infobox), stores what it read through the write path (how documents are stored as notes) with its link, and answers from memory. Questions about the user (*my*, *I*, *we*) are never sent to the web; the assistant asks to be taught instead. Figure 5.5 shows the loop.

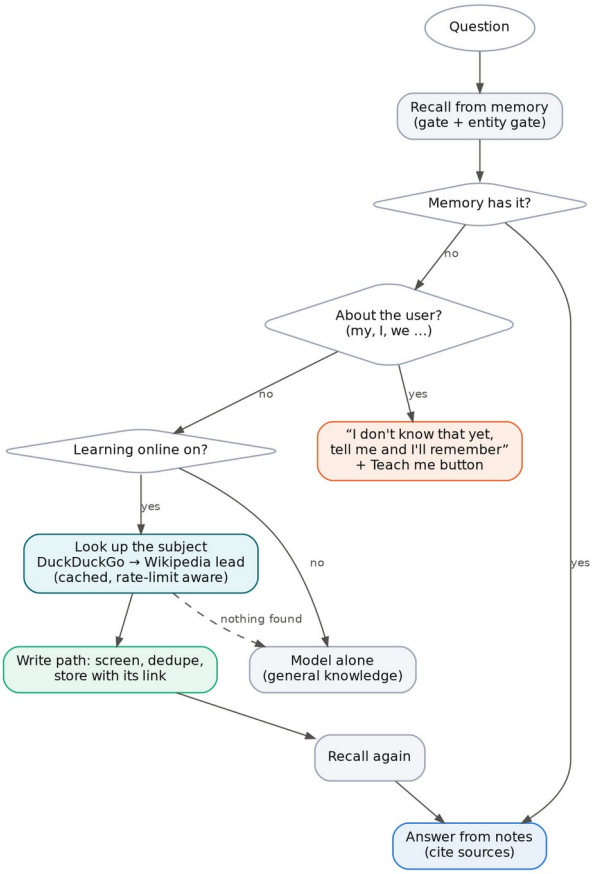


Figure 5.5: Eager learning: look up on a miss, store with the source, answer from memory; never for personal questions

Learning formulas from corrections. The model writes the formula as an expression, or as an equation with one unknown letter; a calculator evaluates or solves it. When an answer in a training round is wrong, one correction is stored: the rule and that problem worked through.

Later problems recall the correction (Figure 5.6). An answer is **grounded** only if its expression uses every number the problem gives:

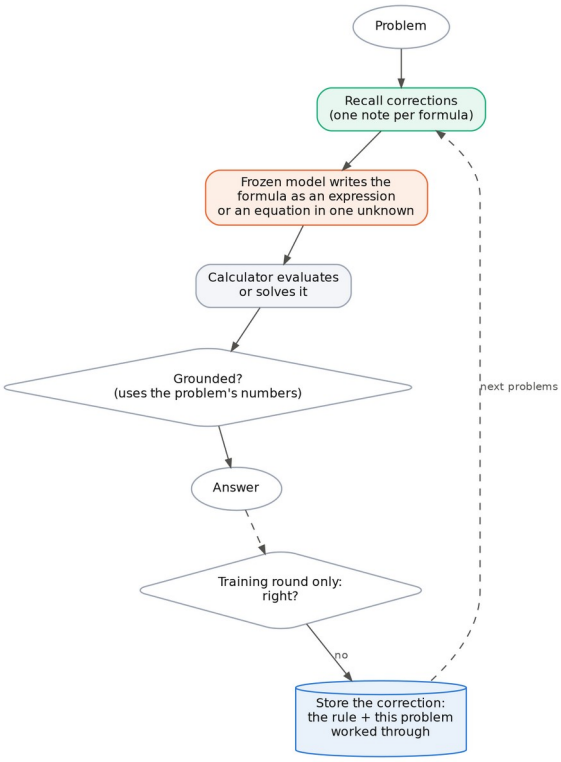


Figure 5.6: Learning a formula from corrections: the model sets the problem up, the calculator does the arithmetic, and a wrong answer in training stores the rule with a worked example

$$\text{grounded}(e, p) = 1\left[\text{nums}(p) \subseteq \text{nums}(e)\right] \tag{5.11}$$

A number the problem marks as a percentage may appear in the expression as a fraction (7% as 0.07). That rule is itself a correction: an earlier version also accepted *4 items* as *4%* (found on the development draw). **Sleep** (when the device is idle) tidies duplicates, looks up names memory mentions but lacks, rebuilds the index and trains a query adapter (a small learned adjustment to how questions are matched to notes), which is kept only if held-out (kept aside until the end for a fair test) recall rises; on the evidence of section 5.9.8 it never did.

5.4.6 Large Data and the Device Governor

> **In plain words.** How a large collection of documents is stored without running the computer out of memory: notes become searchable by keyword at once, and the slower search-by-meaning is added in the background, always within limits measured on the device.

On a CPU, embedding is the cost of ingestion (loading documents into memory): 30–47 notes per second for BGE-small on four cores, which would leave a 120,000-paragraph corpus (the

collection of documents) unusable for over an hour. Memory therefore has two tiers. Notes are committed at once **without** vectors and indexed by SQLite's FTS5 keyword index, kept in step by triggers, so they are searchable immediately through hybrid order (Equation 5.8). A background `drain` adds the vectors a batch at a time, pauses while other work keeps the CPU busy, and stops at the device's limits; the queue is its state, so it resumes after any stop.

A **governor** probes the device (free disk, available RAM, CPU load), calibrates it (embedding rate, search time per 100,000 notes, prompt-reading and generation speed) and enforces limits: a disk reserve, a RAM reserve, a store cap and a process cap. Every job checks it before each batch and stops cleanly, resumable, at the first limit. It also sets the read plan, choosing the largest token budget B whose expected time meets the latency target L:

$$B^{*} = \max_{B \in \{512,256,128,64\}} \left\{ B : t_{\text{search}}(N) + 30 + \frac{B+60}{r_{\text{read}}} \cdot 1000 + \frac{16}{r_{\text{gen}}} \cdot 1000 \le L \right\} \tag{5.12}$$

where

$t_{\text{search}}(N)$ exact-search time in ms for N notes, linear in N from the calibration;
$r_{\text{read}}, r_{\text{gen}}$ the measured prompt-reading and generation speeds in tokens per second (285 and 50.7 for the 0.5B model on machine A).

5.4.7 Model Runners and the Learned Routing Policy

> **In plain words.** A small learned rule decides how each question is answered (how much memory to read, or whether to ask the cloud model). It was trained on practice questions, rewarded for right answers and charged for time and cloud cost.

Every runner exposes one operation, `generate`, and none can change model weights (FR-09). The Ollama (free software that runs models on a computer) runner uses greedy decoding and refuses any host but loopback unless configured as a ceiling (IR-01, NFR-06). The cloud runner pins the model name, retries with backoff, and logs a failure instead of scoring it (IR-02, IR-07). Kimi accepts only temperature (how random the model's word choices are) 1, so its answers are single samples. Every answer from any model is cached by its exact request, which is what makes the results replayable without a key. Ollama aborts some generations with *token repeat limit reached*; the runner now records that as an empty answer instead of crashing the run (found in the formula experiment).

The learned policy chooses per question among five actions: the small model alone, with 128, 512 or 1,024 tokens of memory, or escalation (handed to a person or a stronger model) to the

cloud. No neural network is trained; the learned parameters are a ridge-regression vector per action and the cascade's grounding model.

Formally, the router (a small learned rule that decides how to answer each question) faces a contextual bandit. For question q with features $\varphi(q)$ it chooses an action a from the set $\mathcal{A}$ = {local, mem128, mem512, mem1024, cloud} and receives the reward

$$r(q,a) = y(q,a) - c_{\text{cloud}}\,1[a = \text{cloud}] - \mu\frac{T(q,a)}{1000} \tag{5.13}$$

where

$y(q,a) \in \{0,1\}$ whether action a answered q correctly;
$c_{\text{cloud}} = 0.3$ the price of one cloud call in units of one correct answer (a design preference, swept from 0 to 1.2 for the Pareto table);
$\mu = 0.1$ the price of the small model reading 1,000 prompt tokens, *T(q, a)*.

The offline learner fits one ridge regression per action on the training questions (design matrix X of feature rows, reward vector $\mathbf{r}_a$) and routes to the action with the highest predicted reward:

$$\hat{\theta}_a = \left(X^\top X + \lambda I\right)^{-1} X^\top \boldsymbol{r}_a,\ \pi(q) = \arg\max_{a \in \mathcal{A}} \hat{\theta}_a^\top \varphi(q) \tag{5.14}$$

The online learner, LinUCB (Li et al., 2010), sees only the reward of the action it took. It keeps $A_a = I + \Sigma x\, x^{\mathrm{T}}$ and $\mathbf{b}_a = \Sigma r\, x$ for each action and chooses optimistically:

$$a_t = \arg\max_{a \in \mathcal{A}} \hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t},\ \hat{\theta}_a = A_a^{-1}\boldsymbol{b}_a \tag{5.15}$$

The cascade (a chain: try the cheap way first and pass on only what it cannot handle) answers locally first and escalates when a ridge model of the answer's own signals ψ (are the answer's words in the notes it read? is it abstention-like? how long is it?) predicts that the answer is wrong:

$$\text{escalate}(q) = 1\left[\hat{w}^\top \psi\left(p_{\pi(q)}\right) < \theta\right] \tag{5.16}$$

The hyperparameters λ, α, the number of passes and θ are chosen by K-fold cross-validation (tuning on one part of the practice questions and checking on another, in turn) on the training split only (K = 5, folds stratified by task type), maximising the mean validation reward:

$$h^* = \arg\max_h \frac{1}{K}\sum_{k=1}^{K} \bar{r}\left(\pi_h^{(D \setminus D_k)}, D_k\right) \tag{5.17}$$

where

$\pi_h^{(D \setminus D_k)}$ the policy with hyperparameters *h* trained without fold *k*;

$\bar{r}(\pi, D_k)$ its mean reward (Equation 5.13) on the held-out fold.

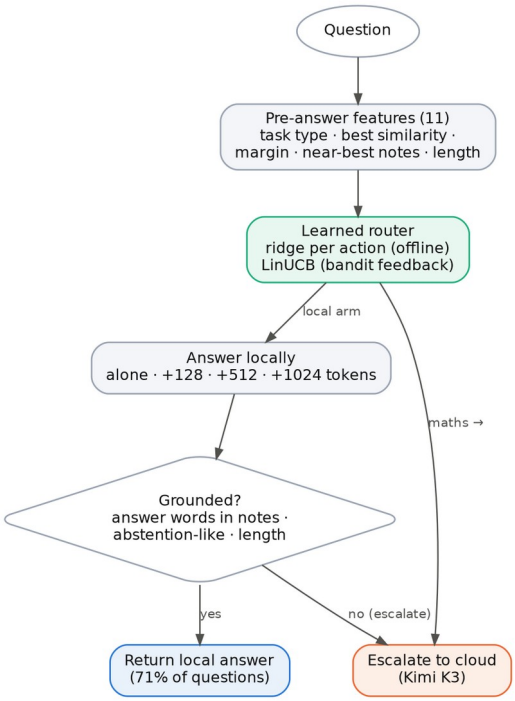


Figure 5.7: Learned routing: a router chooses a memory budget or the cloud, and a grounding check escalates local answers the notes do not support

5.4.8 CAMR Personal: the Engine as a Desktop Assistant

CAMR Personal gives any model the user has in Ollama a long-term memory. The user teaches it notes and files (.txt, .md, .pdf), states facts with *remember that* …, and approves good answers with a thumbs-up; what the user states in chat is remembered too, so a fact mentioned weeks ago is recalled after it has left the context window. Before the answer streams, a card shows which memories were read. Version 0.4 adds verified answers: a fact is shown as *Verified* with its source, or as an unverified lead. The server is a standard-library Python process bound to 127.0.0.1 that refuses foreign hosts and cross-site writes; the front end is a single page with no framework. Self-contained downloads are built by continuous integration for six platforms. Figure 5.8 and Figure 5.9 show it in use.

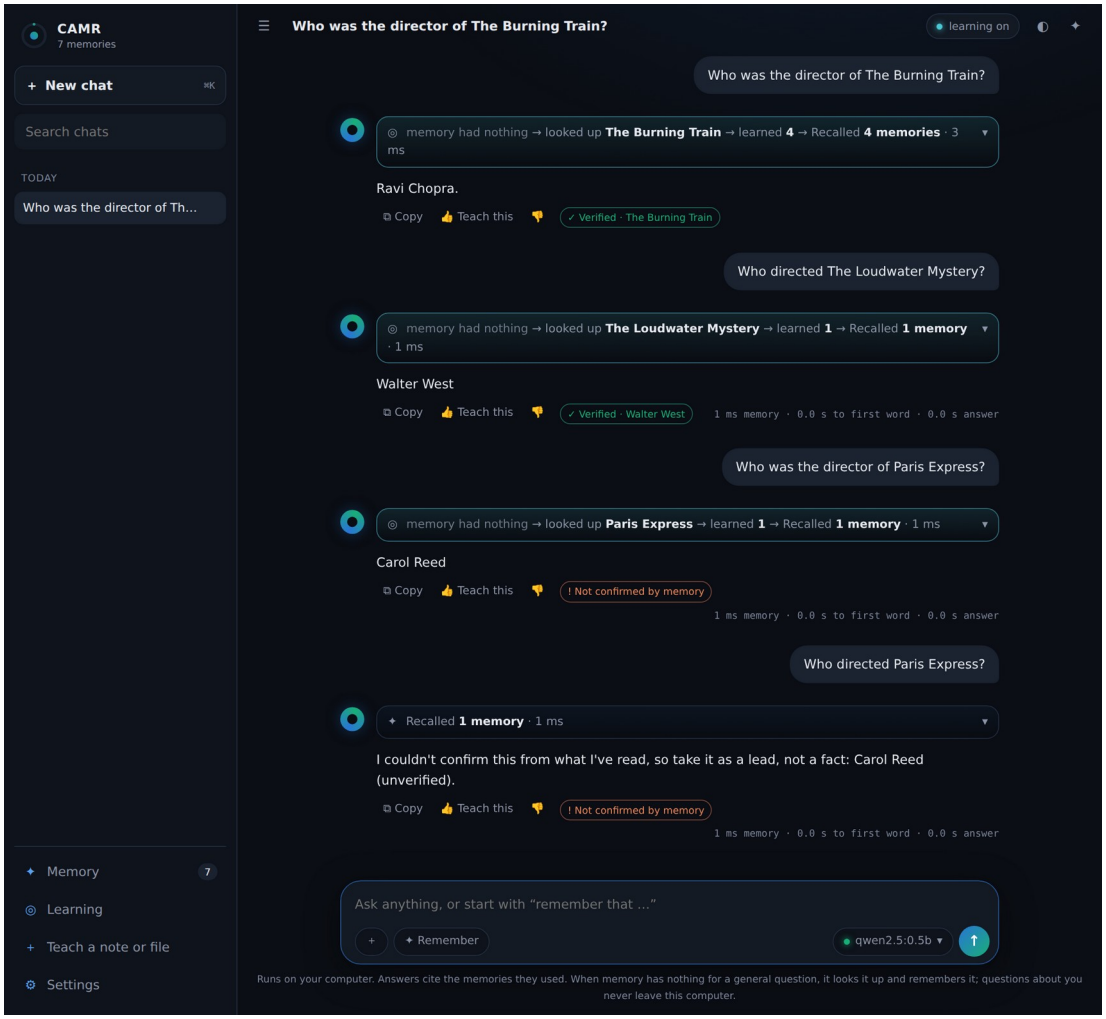


Figure 5.8: CAMR Personal 0.4: an answer stated as verified with its source, and a lead that could not be verified

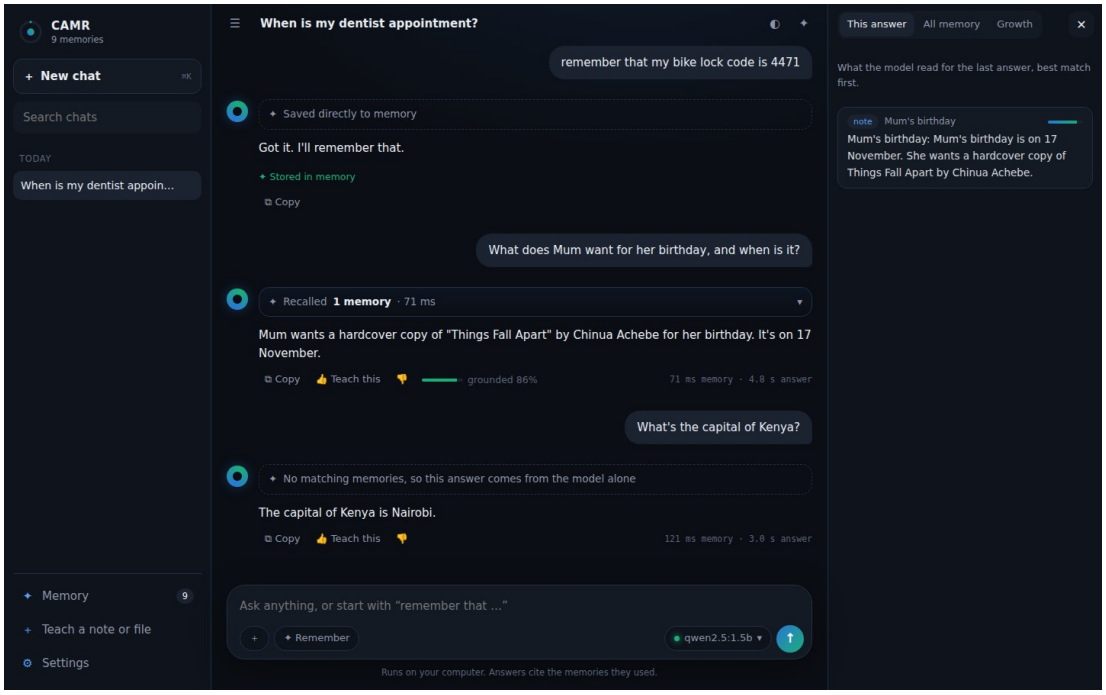


Figure 5.9: CAMR Personal: a remembered fact recalled, its grounding, and an abstention, with the memory panel open

5.4.9 Interface Design of CAMR Personal

The interface had one job beyond being a usable chat window: to make memory, the thing the research is about, visible and controllable. Its design followed six principles, each traceable to a feature and to a test (Table 5.9).

Table 5.9: Design principles of the interface and how each was realised and checked

Principle	Realised as	Checked by
Memory is visible, before the answer	A *Recalled N memories* card streams before the first word; an *answered by the model alone* label when gating abstains	TC-27, TC-25; browser test (TC-37)
Familiar conventions	ChatGPT's minimal composer and chat list; Ollama's model picker inside the composer and drop-anywhere files; Kimi's sources-first answers	Design review against the three apps
Honest provenance	Grounded % per answer; each memory labelled note / chat / learned / file; Forget on every item	TC-28, TC-29; memory panel test
The user stays in control	*remember that…* stores without a model call; 👍 teaches, 👎 only records; export hides text by default	TC-26, TC-39
Local and private by construction	Server bound to 127.0.0.1; Host and Origin checked; no telemetry	TC-31, TC-32
Accessible and responsive	WCAG 2.2 AA contrast in both themes; one heading structure; layout from 390 px phones to desktops	Contrast audit, axe-core audit, phone-width test (TC-40, TC-41)

Figure 5.10 shows how the interface is built. The front end is plain HTML, CSS and JavaScript with no framework or build step, so the downloadable executable stays small (16–43 MB) and starts in about a second. Answers arrive as Server-Sent Events in a fixed order:

recall (which notes were read), then *token* events, then *done* with timings and the grounding score. That order is what puts memory in front of the answer.

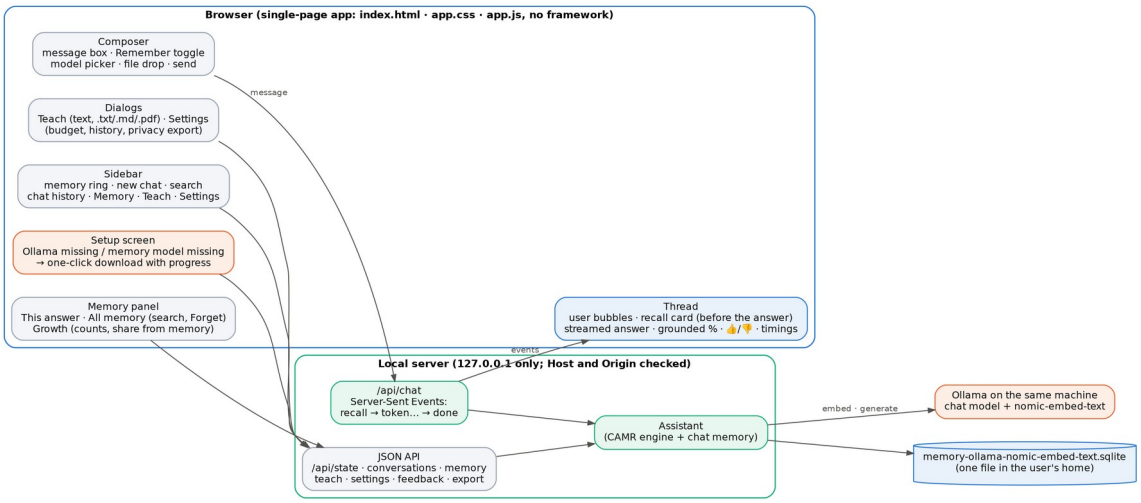


Figure 5.10: Interface components of CAMR Personal and the local server they talk to

Layout. A three-column CSS grid: a 272 px sidebar, a fluid thread capped at 768 px for comfortable line length, and a 360 px memory panel that slides in. Below 900 px the sidebar becomes an overlay and the panel covers the screen, so the same interface works at phone width (TC-41). **Typography.** The system UI font stack (Inter where installed) at 15 px with 1.6 line height; monospace for codes and timings. **Colour.** Dark-first, with a single blue-to-green accent taken from the study's chart palette, and a light theme. Every colour is a named design token, so the two themes differ only in token values.

Text must stand out from its background for readers with low vision. WCAG (the international guidelines for accessible web pages) 2.2 defines the contrast ratio of two colours from their relative luminance L, computed from the linearised sRGB channels:

$$\mathrm{CR} = \frac{L_{\mathrm{light}} + 0.05}{L_{\mathrm{dark}} + 0.05}, \quad L = 0.2126\,R_{\mathrm{lin}} + 0.7152\,G_{\mathrm{lin}} + 0.0722\,B_{\mathrm{lin}} \tag{5.18}$$

Level AA requires CR ≥ 4.5 for normal text and ≥ 3 for large text and interface marks. Figure 5.11 shows every token pair the stylesheet uses, measured with this formula after the audit in section 5.6.5 corrected the four tokens that failed.

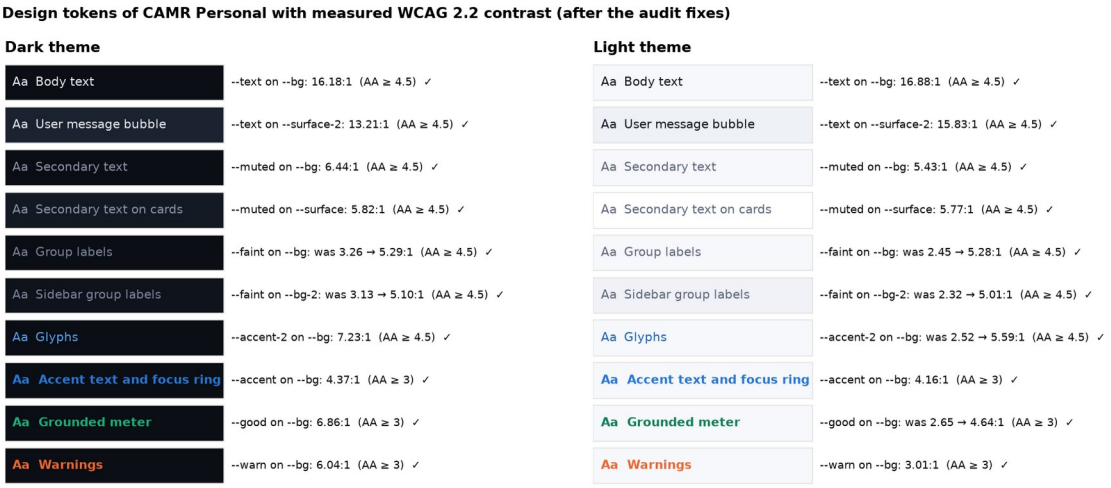


Figure 5.11: Design tokens of both themes with their measured contrast ratios

5.4.10 Automation Gateway, n8n Workflows and the Agent

> **In plain words.** A gateway is a small local web service. Every answering system (the model alone, context stuffing, RAG and CAMR) sits behind the same four commands, so an n8n workflow can switch systems without any other change. The screenshots show the workflows running, with what went into and came out of each step.

To test whether the engine is needed in the automations companies actually build, it was put behind a small HTTP (the protocol that web pages and web services use) gateway (a small local service that gives every system the same commands) that any workflow tool can call, next to the alternatives such a workflow would otherwise use (Figure 5.12). The gateway serves four systems behind one interface (*reset*, *teach*, *forget*, *ask*): the model **alone**; **context stuffing**, which pastes the whole knowledge base into every prompt; **modern RAG**, hybrid search with a cross-encoder reranker and LangChain's standard RAG prompt; and **CAMR**, verified memory. Each answer records its model calls, tokens, seconds and, for the cloud model, its cost in US dollars at Moonshot's list price:

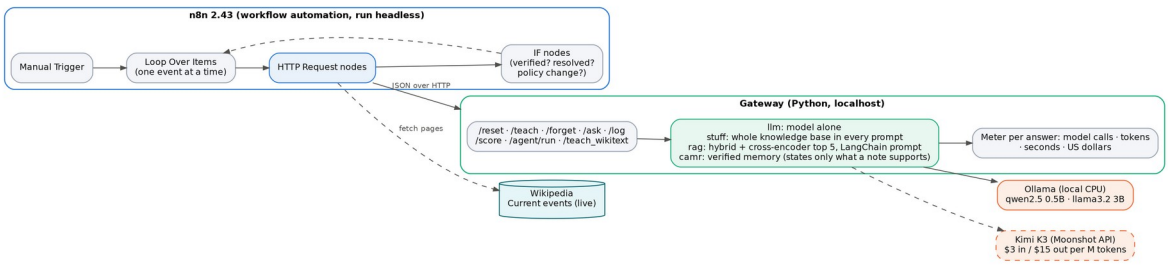


Figure 5.12: Automation set-up: n8n workflows call the gateway over HTTP; the gateway serves four answering systems on a local or a cloud model and meters each answer

$$\text{cost} = \frac{3.00\,T_{\text{in}} + 15.00\,T_{\text{out}}}{10^{6}}\ \text{USD} \tag{5.19}$$

where

$T_{\text{in}}, T_{\text{out}}$ prompt and generated tokens reported by the API (Kimi K3 reasoning tokens count as output); cached-input discounts were not applied, so costs are upper bounds.

Three n8n (a visual tool for building automations from connected steps) workflows were built with standard nodes only (Manual Trigger, HTTP Request, Loop Over Items, IF, No Operation) and run headless (from the command line, without the visual editor) with `n8n execute`; one workflow file per system is in `results/agentic/n8n/`. The **support desk** (Figure 5.13) replays eight days of Helio Solar: each event is either a policy change, which withdraws the old document and publishes the new one, or a ticket, which is answered and then either sent or escalated. When a person resolves an escalated ticket, the resolution is taught, so the next customer who asks gets it automatically. The **current-events digest** fetches 30 daily Wikipedia pages live, teaches every item, and answers 50 questions about them; when Wikimedia refused a fetch (it began to after repeated runs, D-36), the gateway taught the same page as cached when the questions were written. The **operations agent** workflow hands each of 20 tasks to an agent and records whether it completed it or handed it to a person.

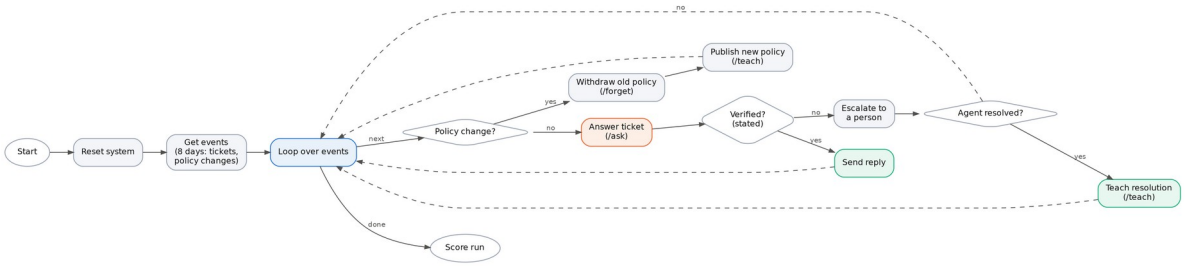


Figure 5.13: Support-desk workflow as built in n8n (node names as in the workflow file)

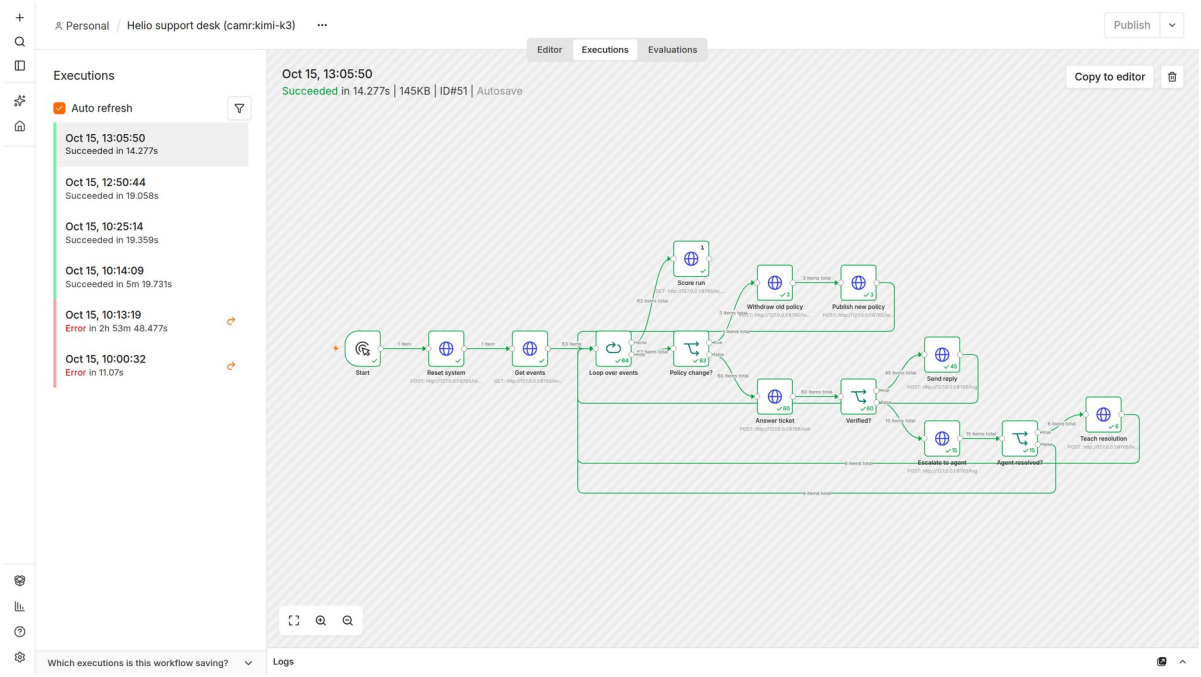


Figure 5.14: A support-desk run in n8n (Kimi K3 + CAMR): 60 tickets answered, 45 replies sent, 15 escalated, with the resolutions a person gave taught back and 3 policy changes applied; each link shows how many items passed (screenshot)

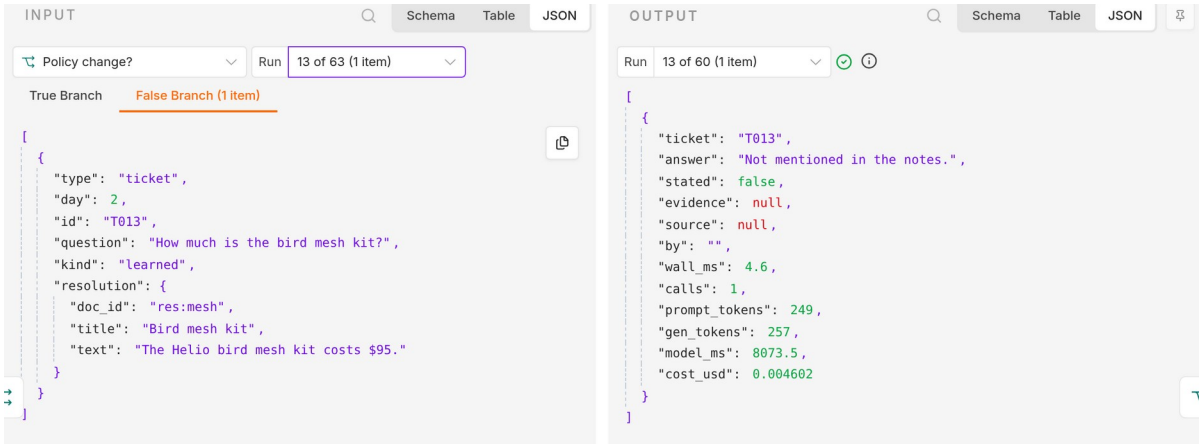


Figure 5.15: Inside that run, day 2, ticket T013 “How much is the bird mesh kit?”: nothing in memory, so the answer is not stated (stated: false) and the ticket goes to a person; the input (left) carries the resolution the person gives, which the workflow then teaches (screenshot)

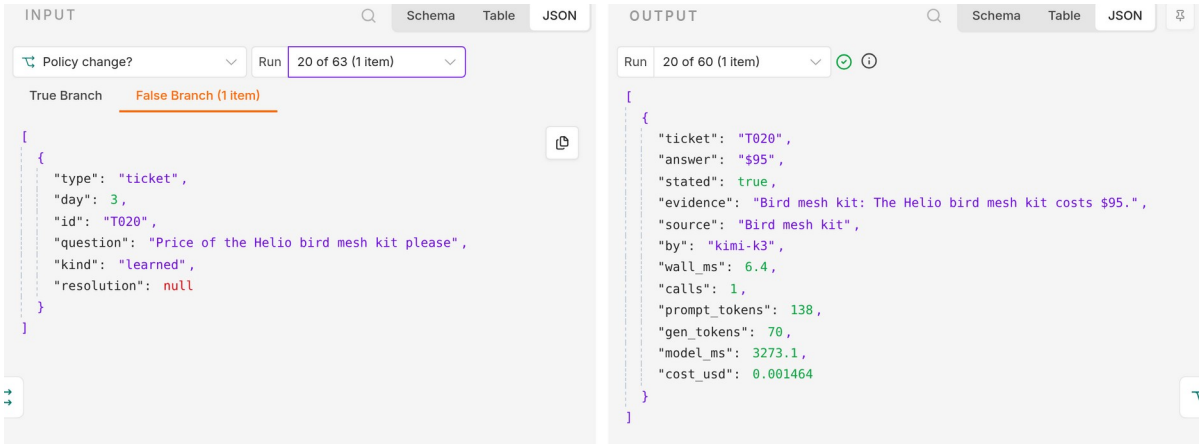


Figure 5.16: Day 3, ticket T020 “Price of the Helio bird mesh kit please”: answered “$95” from the taught resolution, with the evidence sentence and its source; one model call, $0.0015 (screenshot)

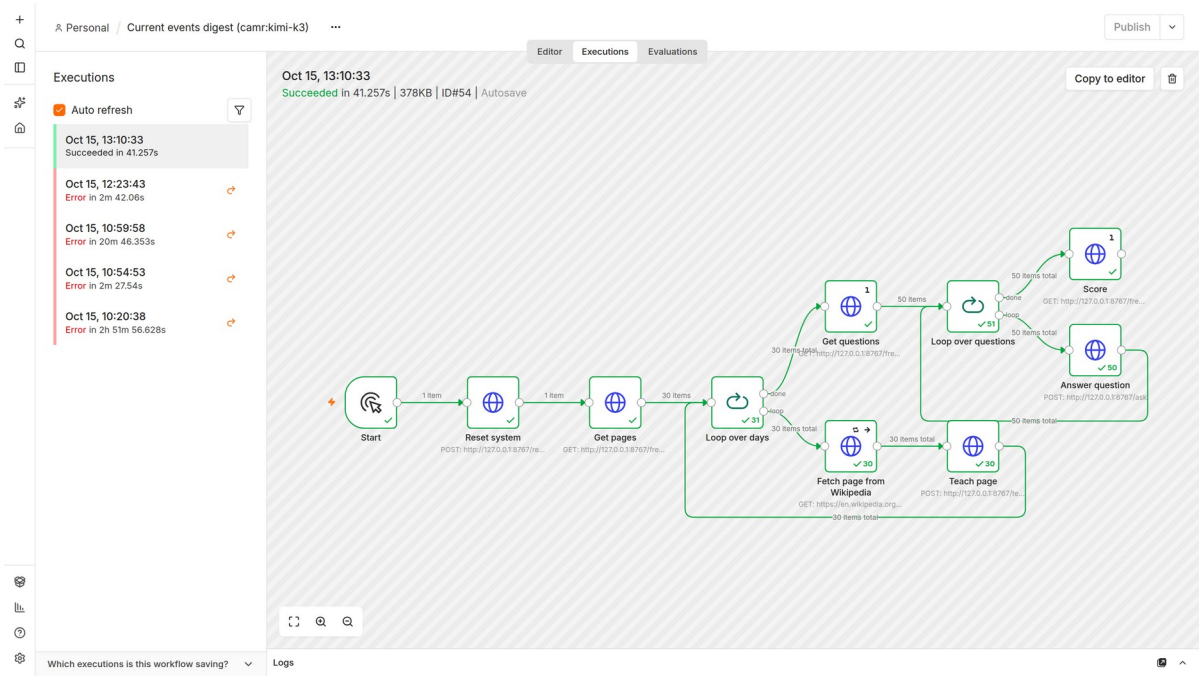


Figure 5.17: A current-events run in n8n (Kimi K3 + CAMR): 30 pages fetched live from Wikipedia and taught (556 items), 50 questions answered, one score (screenshot)

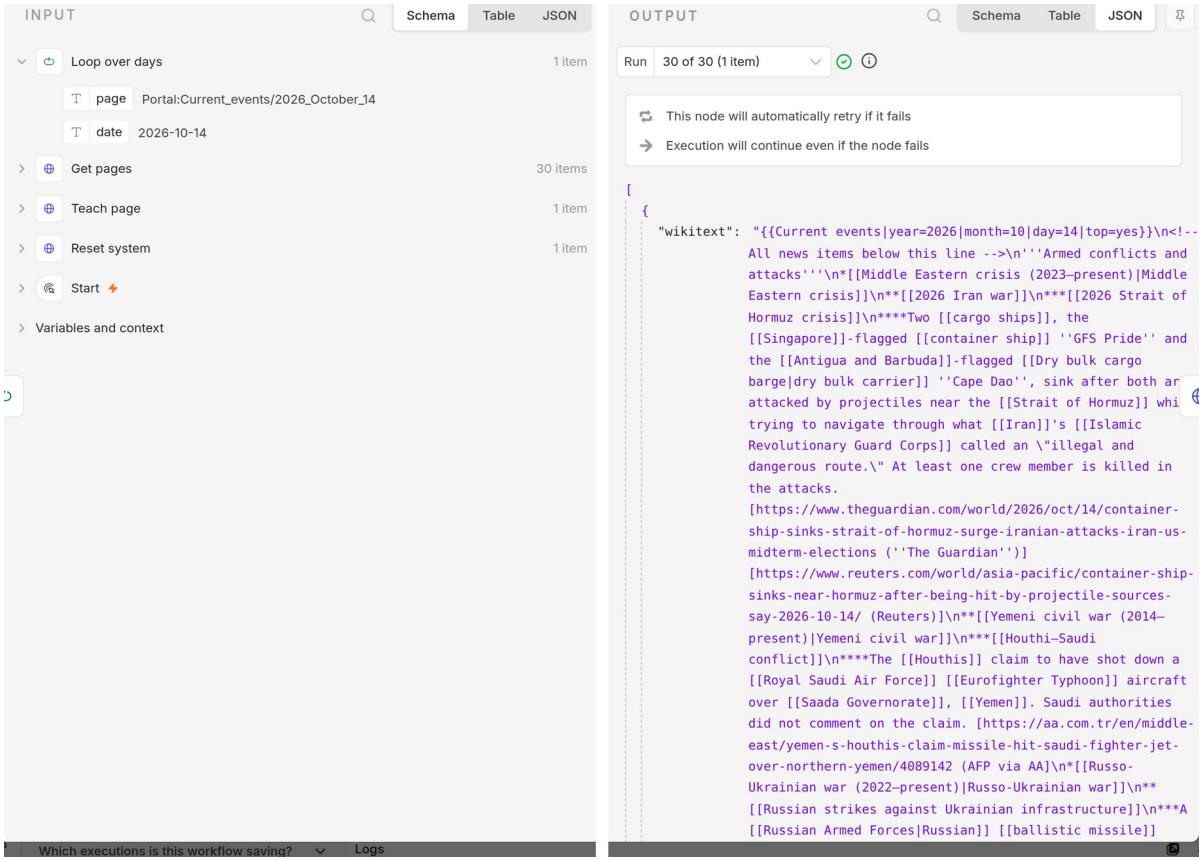


Figure 5.18: The fetch node's output: the raw wikitext of one day's “Current events” page as fetched live (the node retries once and continues on error, D-36) (screenshot)

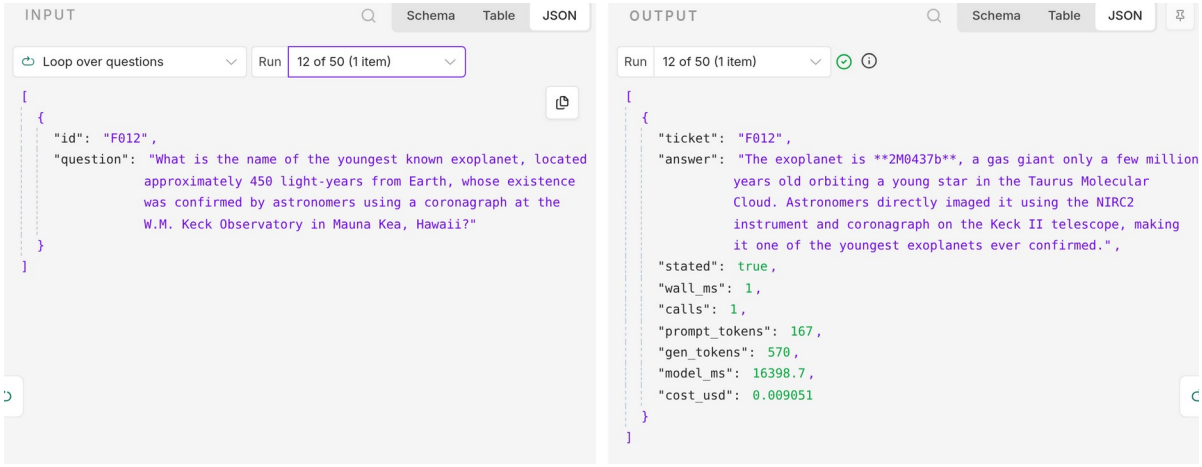


Figure 5.19: Question F012 to Kimi K3 alone: a confident answer from its training data that is wrong for this news (“2M0437b”); 16.4 s of model time, $0.0091 (screenshot)

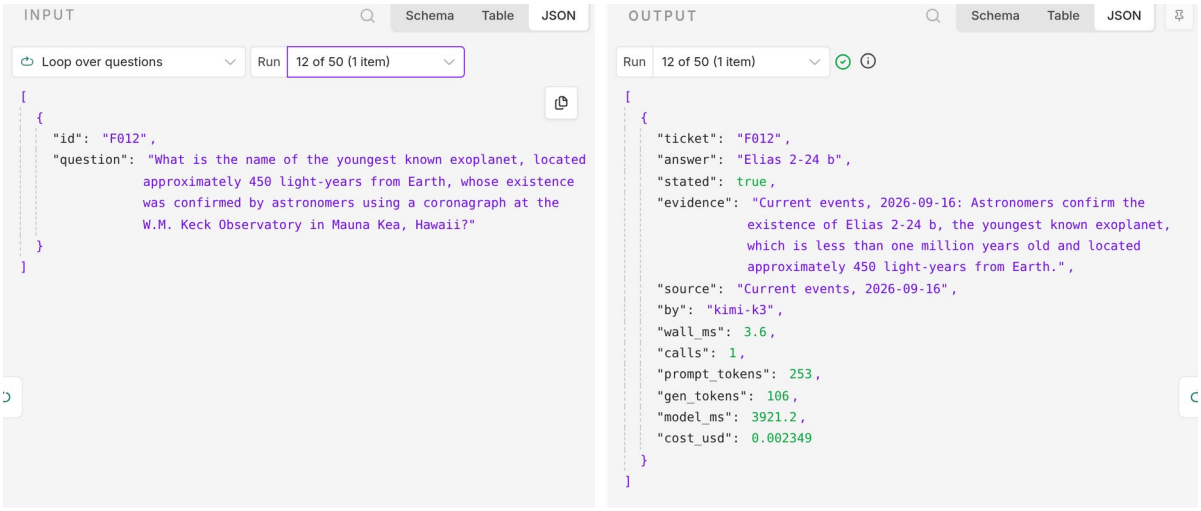


Figure 5.20: The same question to Kimi K3 + CAMR: “Elias 2-24 b”, with the news item it came from as evidence; 3.9 s, $0.0023 (screenshot)

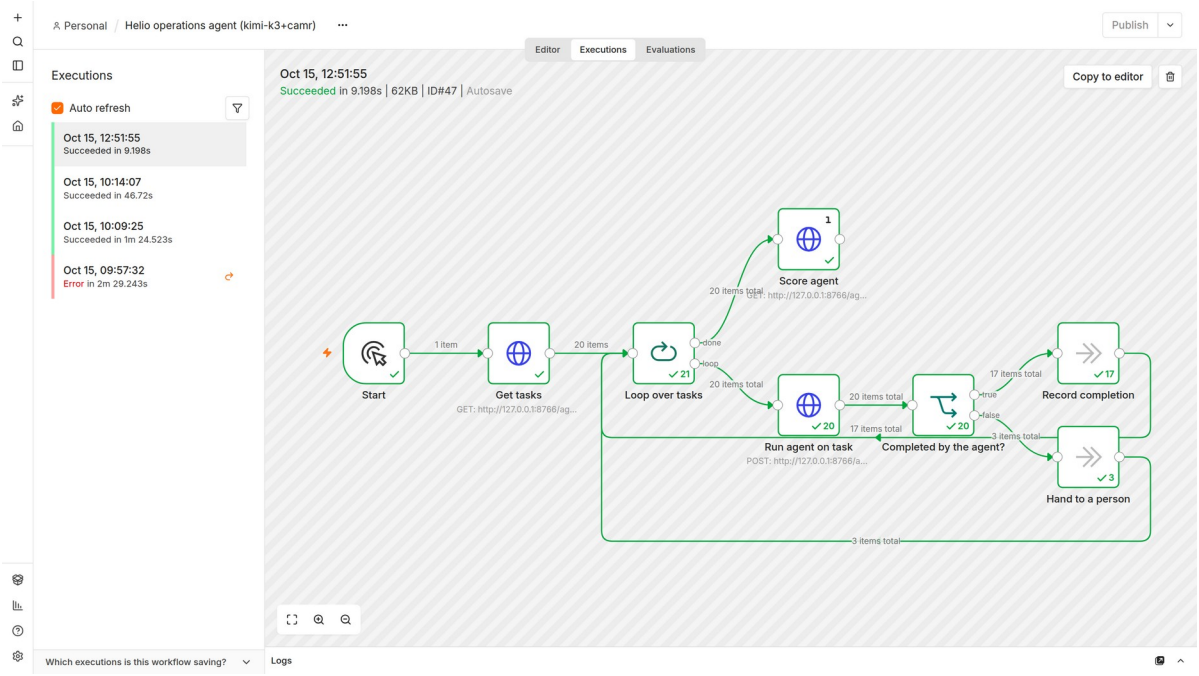


Figure 5.21: An operations-agent run in n8n (Kimi K3 + CAMR): 20 tasks, 17 completed by the agent, 3 handed to a person (screenshot)

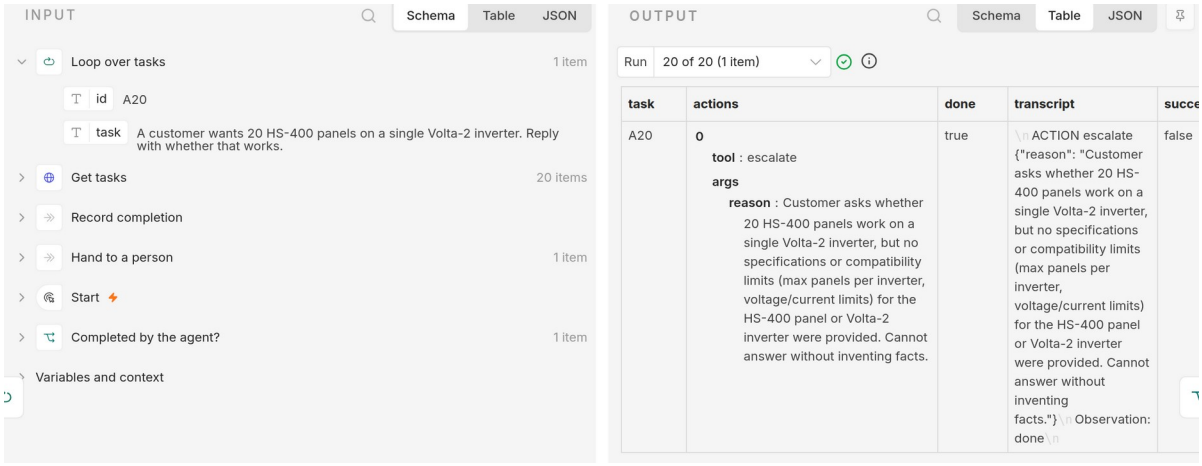


Figure 5.22: Task A20 (“20 HS-400 panels on a single Volta-2 inverter?”) given to Kimi K3 without memory: escalated, “cannot answer without inventing facts” (screenshot)

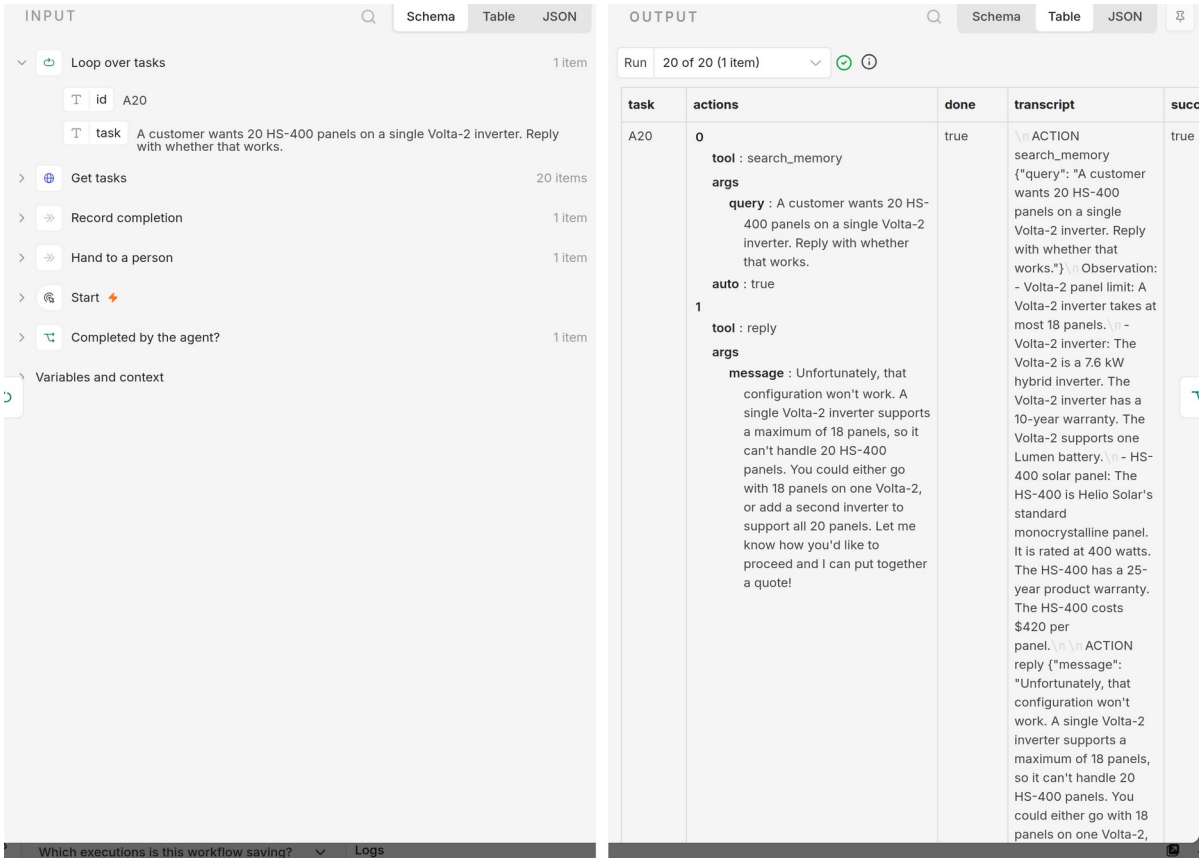


Figure 5.23: The same task with CAMR: the agent searches memory, reads that a Volta-2 inverter takes at most 18 panels, and replies with the limit and two options (screenshot)

The agent (Figure 5.24) is a tool-using loop in the ReAct (think, use a tool, read the result, repeat) style: each turn the model writes one line, *ACTION* <*tool*> <*json*> or *DONE*; the tool runs and its result is appended. Its tools are `search_memory` (absent without memory), `calculate`, `create_invoice`, `reply` and `escalate`. A task succeeds only when every

check on the tool calls holds; an invoice or reply with a number or fact that is not the company's counts as a **hallucinated action**. The first runs with the 3B model showed it answering *DONE* at once or never consulting memory, so the agent (a model that completes a task step by step using tools) now searches memory with the task before the first turn, and the prompt ends with an explicit cue for the next line.

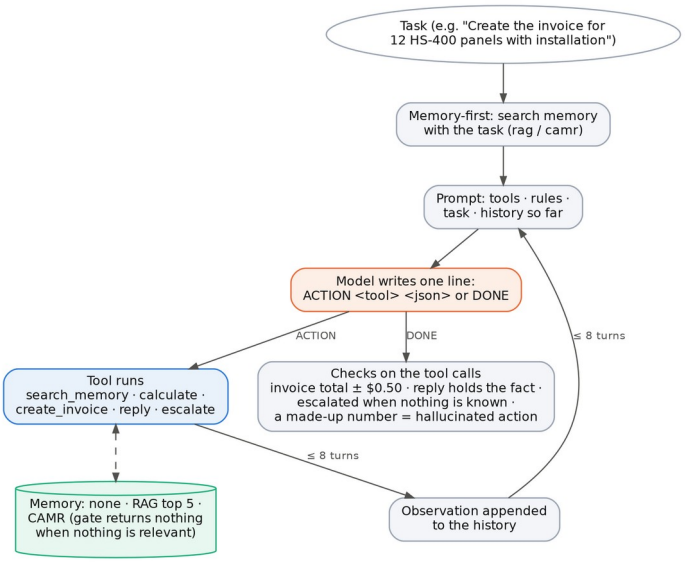


Figure 5.24: The operations agent: one tool call per turn, at most eight turns, checked on what it did, not on what it said

5.5 Model Training, Selection and Evaluation Protocol

> **In plain words.** No language model was trained in this project; they are used as downloaded. What was chosen is which models to test, what to compare CAMR with (the model alone, the cloud model, eight popular RAG designs, pasting all documents in, and retraining) and how answers are scored. The few small settings inside CAMR were tuned on practice questions, never on the final test.

5.5.1 Baselines and Candidate Models

Because CAMR's answering models are frozen, *model selection* meant two things: which frozen model to recommend for the engine, and which engine configuration to use. Both were chosen on evidence kept apart from the final tests. Table 5.10 lists every baseline (what it is compared against) the engine was compared with.

Table 5.10: Baselines

Baseline	**What it is**	**Used in**
Model alone (floor)	The same frozen model, no memory	Every experiment
Kimi K3 closed-book	Frontier cloud model answering from its weights	5.9.3, 5.9.6, 5.9.9
Kimi K3 with the same	The cloud model reading what CAMR retrieved	5.9.6

Baseline	What it is	Used in
notes	(ceiling_rag)	
Eight modern RAG pipelines	Dense, BM25, hybrid, reranked, HyDE, corrective, abstaining, and reranked with CAMR's check (section 5.5.4)	5.9.5
Context stuffing	The whole knowledge base in every prompt	5.9.9
LoRA fine-tuning (facts)	Qwen2.5-0.5B trained on the 237 documents CAMR stores	5.9.8
LoRA fine-tuning (formulas)	The same model trained on the same corrections memory stores	5.9.8
Token memory (kNN-LM)	22,554 (hidden state, next token) pairs beside the frozen model	5.9.8

Candidate models. Nineteen quantised models from 0.5B to 14.8B parameters across ten families were each run with the same store, questions and prompts (greedy decoding, seed 13). The pilot questions served as validation evidence; the 200-question HotpotQA (public questions that need two Wikipedia articles) study then served as a held-out (kept aside until the end for a fair test) test, with its configuration written before it ran. For verified answering, five readers (0.5B, 1.5B, 3B, 3.8B, 7B) answered the held-out set with the pipeline that had been chosen on the development set with the 0.5B model.

5.5.2 What Was Trained, and How It Was Selected

No weight of an answering model changes in CAMR. Four components were fitted to data, and each was selected on data its final test never saw (Table 5.11).

Table 5.11: Trained and tuned components and how each was selected

Component	Search space	Selected on	Chosen	Tested on
Router (ridge per action)	$\lambda \in \{0.01 \ldots 100\} \times 3$ feature sets	5-fold CV, 144 training questions	task-only features, $\lambda = 0.01$	72 held-out questions, once
LinUCB (online router)	$\alpha \in \{0 \ldots 2\}$, 1–10 passes	same	$\alpha = 1$, one pass	same
Grounding cascade	threshold $\theta \in [0, 1]$, step 0.05	same	$\theta = 0.05$	same
Read path (gate margin δ, budget B)	$\delta \in$ {None, 0.05 … 0.25}, B ∈ {256, 512, 1024}	validation half of 1,000 retrieval questions	δ = None, B = 1,024 (retrieval only)	other half
LoRA on documents	format × lr × epoch	60 validation questions	raw, 3e-4, epoch 1	60 held-out questions
LoRA on corrections	epoch 1–6	2 problems per family	epoch 6	fresh draw (seed 47)
RAG abstain and CRAG thresholds	cross-encoder score	200 development questions	abstain 5.3; CRAG 4.77 / 7.57	200 held-out questions
Verified answering (methods)	infobox facts, fact reader, compression, strict checks, subject look-up, second readers, hybrid	200 development questions	facts + compression + strict + subject look-up, 0.5B reader	200 held-out questions, once

The two LoRA (a cheap way of retraining a model) baselines were trained on CPU in full precision (rank 16; micro-batch 2 with 4-step gradient accumulation and gradient checkpointing after batch 8 ran out of memory at 13.9 GB). Their training and validation curves are in section 5.9.8.

5.5.3 Evaluation Protocol and Measures

The scores are defined as follows. Let p be the model's answer, G the set of accepted gold answers, and norm(·) the SQuAD normalisation (lower case, punctuation and articles removed, whitespace collapsed):

$$\mathrm{EM}(p,G)=\max_{g\in G}1\left[\mathrm{norm}(p)=\mathrm{norm}(g)\right],\mathrm{Contains}(p,G)=\max_{g\in G}1\left[\mathrm{norm}(g)\sqsubseteq\mathrm{norm}(p)\right] \tag{5.20}$$

$$\mathrm{F}_1(p,g)=\frac{2PR}{P+R},P=\frac{\left|t_p\cap t_g\right|}{\left|t_p\right|},R=\frac{\left|t_p\cap t_g\right|}{\left|t_g\right|} \tag{5.21}$$

where

$\sqsubseteq$ occurrence as a whole-word substring;

t_p, t_g the multisets of normalised tokens of the answer and of the gold answer.

For GSM8K (public school-maths word problems) the final number in the answer is compared exactly with the gold final number (fractions and thousands separators parsed). For a condition with per-question scores y_i the accuracy is their mean, and the headline measure of the proposal (Equation 3.1) is the fraction of the gap closed, with the residual (what is left over) gap beside it:

$$G=\frac{\mathrm{Acc}_{\mathrm{CAMR}}-\mathrm{Acc}_{\mathrm{floor}}}{\mathrm{Acc}_{\mathrm{ceiling}}-\mathrm{Acc}_{\mathrm{floor}}},R=\mathrm{Acc}_{\mathrm{ceiling+notes}}-\mathrm{Acc}_{\mathrm{CAMR}} \tag{5.22}$$

G = 1 means the small model with memory matches the large model; G > 1 means it exceeds a closed-book (from the model's own knowledge, with no documents) ceiling (the large model's score, used as the upper reference), which is why the frontier model is used as the anchor. R is what model size still buys when both models read the same notes. Uncertainty comes from a paired bootstrap over questions: for b = 1, …, 2,000, draw Q questions with replacement and compute the mean paired difference; the 95% interval is the 2.5th and 97.5th percentile of these means:

$$\hat{\Delta}_b^*=\frac{1}{Q}\sum_{i=1}^{Q}\left(y_{j_{b,i}}^{\mathrm{CAMR}}-y_{j_{b,i}}^{\mathrm{floor}}\right),\mathrm{CI}_{95}=\left[\hat{\Delta}_{(0.025)}^*,\hat{\Delta}_{(0.975)}^*\right] \tag{5.23}$$

At the retrieval level, without any language model, the measure is full-support recall (the share of the needed evidence that the search found): the share of questions for which every gold supporting paragraph *S*_q reached the context:

$$\mathrm{SR} = \frac{1}{|Q|} \sum_{q \in Q} 1\left[S_q \subseteq \mathrm{titles}\left(A(q) \right) \right] \tag{5.24}$$

For verified answering every answer falls in one of four outcomes: **correct**; **wrong, stated as fact** (the hallucination rate); **about another meaning, named** (a true statement about a meaning the answer names, e.g. the 2011 film when the question meant the 1992 one); and **not confirmed** (withheld or offered as a lead). Exact matching misses aliases (*Garowe* for *Garoowe*), so every answer exact matching (the answer must contain the reference answer) marked wrong was judged again by Kimi K3 with a fixed prompt (the full text handed to the model), and both scores are kept. For a system that may decline, the precision of what it states is reported beside its accuracy:

$$\mathrm{precision} = \frac{¿\{\text{stated and correct}\}}{¿\{\text{stated}\}}, \mathrm{coverage} = \frac{¿\{\text{stated}\}}{¿\{\text{questions}\}} \tag{5.25}$$

In the automations, each of the *N* tickets ends in exactly one outcome: a correct reply, a wrong reply (including a superseded value after a policy change) or an escalation (handed to a person or a stronger model) to a person. Unknowable tickets (and the first occurrence of a question nobody has answered yet) should be escalated; all others should be answered:

$$R = \frac{n_{\text{correct}}}{N}, W = \frac{n_{\text{wrong}}}{N}, E = \frac{n_{\text{escalated}}}{N}, R + W + E = 1; c_{1000} = \frac{1000}{N} \sum_{t=1}^{N} \mathrm{cost}_t \tag{5.26}$$

An agent (a model that completes a task step by step using tools) task *succeeds* only if every check on its tool calls holds. With 200 questions one question is 0.5 points; with 60 tickets, 1.7 points; with 20 tasks, 5 points.

5.5.4 The Modern RAG Pipelines Compared

> **In plain words.** All eight designs search the same store and give the model the top five passages. They differ in how they search (by meaning, by keywords, or both), whether a second model re-orders the results, and whether they check the results before answering.

CAMR's verified pipeline (a fixed sequence of steps) was compared with eight retrieval-augmented generation pipelines in current use. All nine read the same store (the RAG corpus of section 5.3), retrieve from the same notes, and are scored on the same 200 held-out

questions. Every RAG (retrieval-augmented generation: search for passages and paste them into the prompt) pipeline uses LangChain's standard RAG prompt (*rlm/rag-prompt*) and five passages. Figure 5.25 shows what each one does.

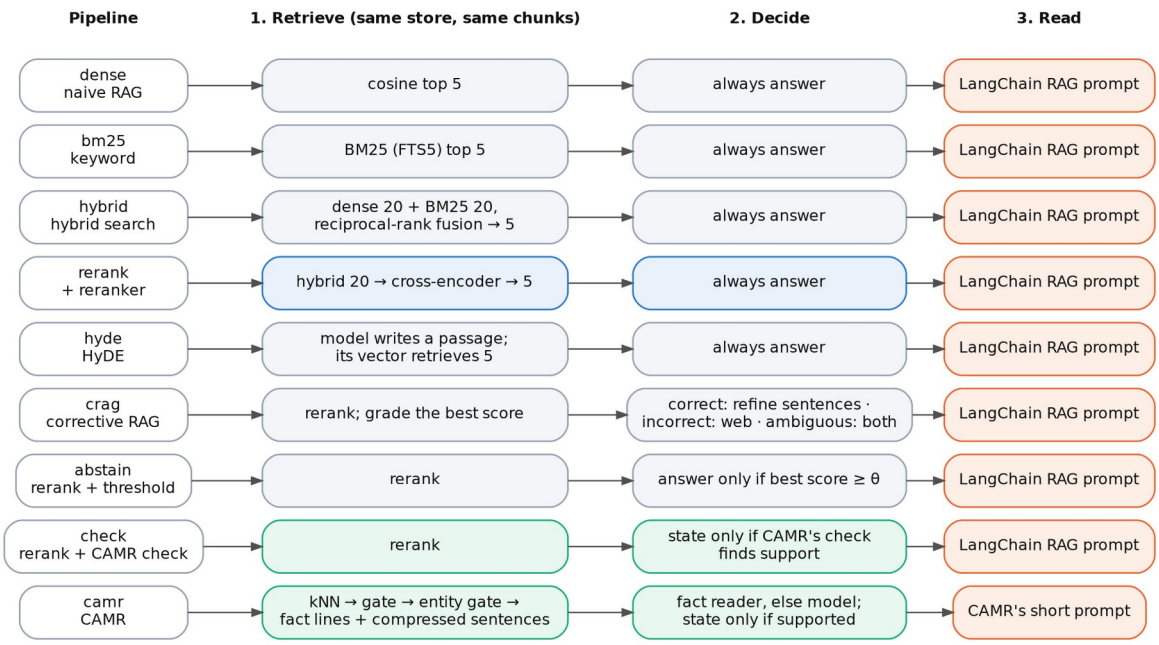


Figure 5.25: The nine pipelines: what each retrieves, how it decides whether to answer, and what reads the result

Hybrid search fuses the dense and BM25 (keyword search) rankings by reciprocal rank (Cormack et al., 2009), and the reranker (a second model that re-orders search results) rescores the fused top 20 with a cross-encoder (Nogueira & Cho, 2019):

$$\mathrm{RRF}(n) = \sum_{r \in \{\mathrm{dense},\,\mathrm{bm25}\}} \frac{1}{k + \mathrm{rank}_r(n)}, k = 60 \tag{5.27}$$

HyDE (Gao et al., 2022) embeds a passage the reader writes instead of the question. Corrective RAG (Yan et al., 2024) grades retrieval with the cross-encoder's best score: above the upper threshold it keeps the relevant sentences; below the lower one it searches the web (here, the same frozen lookups CAMR uses); in between it does both. The abstaining pipeline answers only when the best score clears its threshold. The *check* pipeline is a hybrid: modern RAG retrieval and reading, followed by CAMR's verification check, which separates the value of CAMR's check from the value of its retrieval.

5.6 Testing

> **In plain words.** Testing checked two things: that each part does what the requirements ask (204 automated tests, all passing), and that every problem found during the experiments was fixed and checked again. Every bug has a number (D-01 to D-40), a cause, a fix and a retest.

5.6.1 Test Strategy and Acceptance Criteria

Testing worked at four levels, each answering a different question. **Unit and integration tests** (203, pytest, white-box, fully offline with test doubles for models and the web) check that each component does what its requirement says. **System tests** run the real pipeline (a fixed sequence of steps) on real data with real models. **Replay tests** recompute the key results from frozen caches and require every number to match. **Measurements** check the non-functional requirements against thresholds fixed before the runs: retrieval ≤ 10% of answer time (NFR-02), byte-identical prompts on a repeat (NFR-05), fits in 16 GB (NFR-01). Browser tests (Playwright) and an accessibility audit (axe-core, WCAG AA contrast) cover the interface. A verdict is assigned only by comparing an observed result with an expected one written in advance. Table 5.12 shows where the 203 automated tests sit.

Table 5.12: Automated tests by file (pytest 9.1.1, 203 tests, all passing)

File	Tests	What they cover
`test_memory.py`	48	Write and read paths, gate, entity gate, bridging, budget, per-title limit
`test_learning.py`	27	Lookups, rate limits, privacy rule, packs, harvest, sleep
`test_verify.py`	25	Grounding, subject, relation, kind, fact reader, topic anchoring, replay
`test_harness.py`	22	Conditions, records, failures, resume, runner errors
`test_bridging.py`	16	Entity bridging and its budget interaction
`test_app_learning.py`	15	Application learning features
`test_app.py` / `test_app_server.py`	19	Assistant core; server API, host and origin checks
`test_device.py`	11	Governor limits, resume, read plan
`test_app_e2e.py`	11	Browser tests with Chromium
`test_architecture.py` / `test_learn.py`	9	Package dependency rules; learned policy

5.6.2 Test Cases and Results

The test cases are summarised by requirement below. Detailed cases for the earlier increments (TC-01 to TC-58) are in the test files and the repository's earlier chapter; TC-59 to TC-68 cover this increment.

Table 5.13: Test cases and results

Test ID	Traceability	Scenario and data	Expected result	Actual result and verdict
TC-01–06	FR-01–FR-08, IR-03/04	Write path, screener, store, ranking, budget (fixtures)	Notes with provenance; rejects logged; kNN equal to NumPy; budget never exceeded	Pass
TC-07–08	FR-09/10, IR-01/02	Runner interface; non-loopback host; failing cloud transport	Only `generate`; remote host refused; bounded retries then a logged failure	Pass
TC-09–13	FR-11–FR-15, DR-02	Sampling, scoring, records, ablation plan, bootstrap	Same sample after shuffling; scores equal hand values; CI correct	Pass
TC-14	FR-16, NFR-01	Device profile, 3 × 10 questions	Fits 16 GB	Engine 712 MB + runtime 1,987 MB. Pass
TC-17	NFR-02	Retrieval share of answer time, 0.5B	≤ 10%	56% (contention). **Fail** (D-05)
TC-19	NFR-05	Same configuration twice	Byte-identical prompts	30/30 prompts; 0.5B answers 20/30 (D-02)
TC-20–22	NFR-06, DR-04, IR-07	Sockets blocked; leaked question; runtime down mid-run	No external connection; ingestion refused; failures logged, run continues	Pass
TC-23–36	Application	Teach, ask, remember, 👍, forget, export; empty input; foreign host; path traversal; six platform builds	As specified in Chapter 4	Pass (13 + 6 server tests, CI on six platforms)
TC-37–58	Learning (0.3)	Lookups, privacy of personal questions, rate limits, packs, harvest, sleep	Personal questions never sent; caches replay offline; packs install without a model	Pass (27 + 15 tests)
TC-59	Verified answering	A claim in memory about another subject / relation / of the wrong kind	Not stated	Pass (25 tests)
TC-60	Verified answering	A fact line answers the question	Answered without a model call	Pass
TC-61	Verified answering (D-33)	Questions that name nothing (*What hours is the support desk open?*)	Stated only from a sentence sharing the topic words	Failed first in the n8n desk; fixed. Pass
TC-62	Large data	Bulk ingest; search before any vector exists	Searchable at once through keywords	123,301 paragraphs in 335 s; hybrid recall@10 74.3% with no vectors. Pass
TC-63	Governor	Store cap, disk reserve, RAM reserve; resume	Stops before each limit; resumes with no duplicate	Stopped at all three; 0 duplicates in 14,104 notes. Pass (11 tests)
TC-64	Formulas	A wrong training answer; a correction stored; held-out problems	Correction recalled; calculator evaluates or solves; grounding flagged	Pass (replayed offline)
TC-65	Entities (D-31, D-32, D-37)	Instructions at the start of a sentence; terse ticket phrasing; titles with *St.*, *A.I.*, *Hello*	Instructions not taken for names; titles kept whole	D-37 found by the replay check; fixed. Pass (4 regression tests)
TC-66	Runner	Ollama aborts with *token repeat limit*	Empty answer recorded; run continues	Pass
TC-67	Automation gateway	n8n workflows: 60 tickets, 20 tasks, 30 live pages	Every workflow completes (exit 0); every answer metered	Failed when Wikimedia refused fetches (D-36); fixed. Pass (n8n 2.43.6)
TC-68	Reproducibility (NFR-04/05)	`reproduce.py --twice` without key or network	Every recomputed number equals the committed one	16/16. Pass

5.6.3 Defects Found and Retests

Every defect below was found by a test or a real run, and every one was either fixed and retested or is reported as open. Failed results were kept, not deleted. Most defects were found by running the system on real data, not by the unit tests, which is why the system tests matter.

Table 5.14: Defects found and retests

ID	Defect	Correction	Retest
D-01	Screener rejected a real song title (*I Can't Get Next to You*) as a model refusal	Refusal check only on model-written notes	Pass
D-02	0.5B answers differed on 10/30 questions between identical runs (wrong → wrong)	Engine shown deterministic; floors reported as a range	Open (runtime)
D-03	20B model killed for lack of memory at 13.3 GB	Memory-mapped weights	Pass
D-04	A configuration field added mid-study changed the hash; resume re-ran finished runs	Schema frozen before final runs	Pass
D-05	Retrieval took 56% of the 0.5B model's answer time (NFR-02)	Diagnosed as CPU contention; thread partitioning	Open for 0.5B
D-06	Recency + importance (Eq. 4.1) lowered full-support recall by 4.6 points	Similarity + gating + bridging	Pass
D-07	A run whose model server died mid-sweep was still marked complete	Resume re-runs runs with failed queries	Pass
D-08	gemma2:9b killed three times at 13.4 GB (prompt cache)	Prompt cache capped at 1 GiB	Pass
D-09	App: store used across threads; file watcher crashed	Lock; watcher off	Pass
D-10	App warm-up loaded the model with other options, so it loaded twice	Same options	Pass
D-11, D-12	Interface contrast below WCAG AA; no top-level heading	Tokens per theme; hidden heading	Pass (0 axe violations)
D-13, D-16	App over-refused general questions (*capital of Australia*)	Prompt chosen on held-out cases (21/21)	Pass
D-14	Chat memory stored plain questions	Only statements kept	Pass
D-15	Entity gate: possessive normalised differently in question and note	One normalisation	Pass
D-17, D-18	Lookups: *The* stripped mid-title; an empty reply (rate limit) cached as *nothing known*	Article kept; empty replies never cached	Pass; stream re-run
D-19	*Ask to be taught* prompt led with *I don't know* although the notes answered	Prompt F	Pass
D-20, D-21	Pack harvest drifted off topic; lookups sent no User-Agent, so Wikimedia refused them	Relevance floor; CAMR User-Agent	Pass
D-22, D-23	Sleep read names across sentence ends; two chips below AA contrast	Names within a sentence; darker green	Pass
D-24, D-29	Ambiguous names: the wrong meaning fetched first; for *Spain* a list instead of the country	Meanings named exactly as asked first; the name's own article first	Pass
D-25, D-26	Searched for *director of Emma* (finds people called Emma); fetched fragments of a name	Relation phrases look up their subject; fragments skipped	Pass
D-27, D-30	Rate-limited search and summary APIs cost two minutes a question	Ordinary search page and article page instead	Pass
D-28	*genre is Covers* searched as a name	The phrase only when no name exists	Pass
D-31	Entity gate read instructions (*Give the amount…*, *Find its area*) as names, withholding the right correction for 33 of 60 reworded problems	Instructions at any sentence start are not names	Pass; verified replays identical
D-32	Entity gate read terse ticket phrasing (*Recommended cleaning interval?*, *E57. Any idea?*) as names, so the agent's memory returned nothing	Sentence openers are not names; a name ends with its sentence	Pass; agent 80% → 100%

ID	Defect	Correction	Retest
D-33	Verified answering withheld correct answers to questions that name nothing: 14 of 45 answerable desk tickets escalated	Topic anchoring (Eq. 5.10)	Pass; desk 51.7% → 75.0%, still 0 wrong
D-34	The news workflow reused the support persona, so the cloud model refused news questions as off topic	Persona per scenario	Pass; re-run
D-35	The decline detector missed *I can't identify …*, so a RAG decline was sent as a reply	Pattern widened; every desk system re-run	Pass
D-38	Kimi K3 writes *I don't know* with a curly apostrophe, which the decline detector missed, so 13 of its honest declines on the news questions were scored as wrong answers	Apostrophes normalised before detection (gateway and RAG benchmark)	Every result file re-checked: only that run was affected; re-run
D-37	The D-32 fix itself cut titles at abbreviations and greetings (*St. Louis Blues → Louis Blues*, *A.I. Artificial Intelligence, the composer of Hello*), and its opener rule dropped names that start a statement (*Josh decides to …*), so PopQA questions and the GSM8K interference check recalled different notes	Abbreviations, dotted initials and ellipses do not end a sentence; greetings are openers only at a sentence start; the opener rule applies only to a sentence that asks	Found by the replay check; extraction identical to before D-32 on all 600 PopQA questions and 440 formula problems; replays re-run
D-36	After repeated runs Wikimedia refused the workflow's live page fetches (HTTP 403), and the news workflow stopped	The fetch node continues on error; the gateway then uses the page as cached when the questions were written, and records which pages came live	Pass; re-run
D-39	In the news workflow a loop's *done* output fed all 30 looped pages to the next node, which n8n runs once per item: each question was asked 30 times and 1,500 score requests reset the gateway's connection, so n8n marked the runs as failed	The nodes after a loop run once (*Execute Once*)	Found in the screenshots; scores and costs unchanged (answers per question, repeats were cache hits); re-run clean
D-40	The D-37 opener rule applied only to sentences ending in *?*, so the desk request *Price of the Helio bird mesh kit please* read *Price of the Helio* as a name and the ticket was escalated	Requests (*please*) and fragments without a full stop also open with a common word; the test uses the ticket as written	Found by re-running the desk on the final code; every CAMR desk answer identical to the reported run; PopQA and formula extraction unchanged

5.6.4 Application and Interface Testing

CAMR Personal was tested as a user meets it: valid, empty, out-of-scope and unsupported input, the runtime missing, a foreign host. Browser tests (Chromium, Playwright) cover remember-then-ask with the recall card, an empty message, thumbs-up teaching, theme and panel, phone width without horizontal scrolling, and a keyboard-only path with accessible names. The axe-core (an automatic accessibility checker) audit first reported a *serious* contrast violation and a missing page heading (D-11, D-12); after the fixes it reported **0 violations** in both themes at desktop and phone width, with every colour pair at WCAG (the international guidelines for accessible web pages) AA or better. These tests show that the

application behaves correctly and fails safely. They do not show that people find it useful: no user study was run (section 5.9.18).

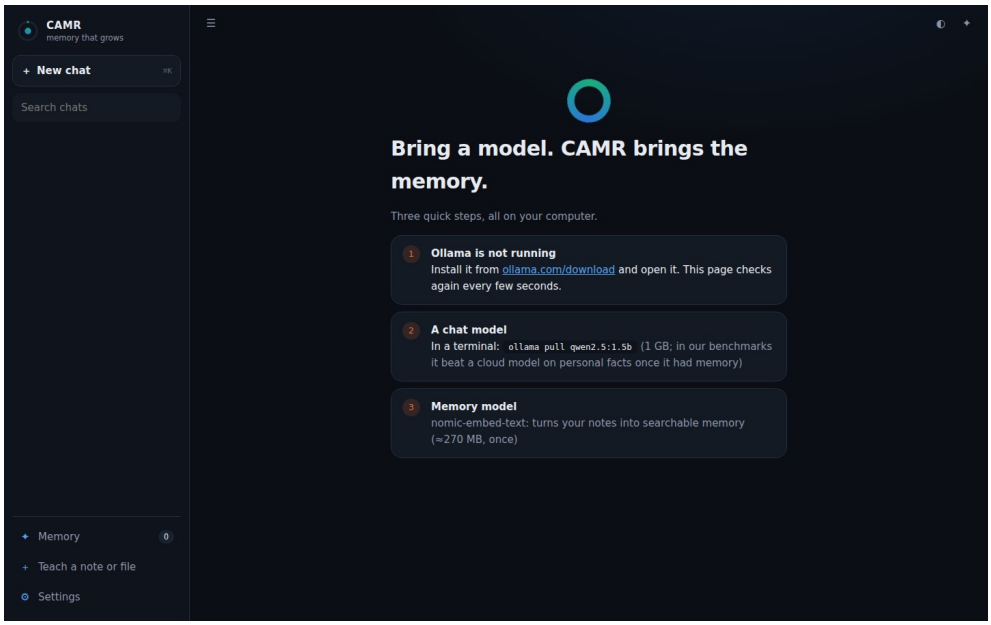


Figure 5.26: Failing safely: with Ollama not running, the application says what is missing and offers the download

5.6.5 Interface Testing: Browser, Accessibility and Layout

The interface was tested in three further ways. **End-to-end browser tests** (`tests/test_app_e2e.py`) drive the real interface in Chromium through Playwright (a tool that drives a web browser automatically for testing), as a user would, with the deterministic test doubles in place of the model so that a failure means the interface is wrong, not the model. An **automated accessibility audit** (`scripts/ui_audit.py`) runs axe-core 4.10 (Deque Systems) against WCAG 2.0/2.1/2.2 A and AA rules and best practices, in both themes, at desktop (1,366 × 860) and phone (390 × 844) size, on a populated chat with the memory panel open. A **contrast audit** computes Equation 5.18 for every token pair.

Table 5.15: Automated interface tests (Chromium, Playwright)

Test ID	What the browser does	Expected	Result
TC-37	Types *remember that my bike lock code is 4471*, then asks for the code	Stored without a model call; the recall card shows the memory before the answer; the answer contains 4471; memory counter increases	Pass
TC-38	Submits a message of spaces	Nothing is sent; no chat created	Pass
TC-39	Clicks 👍 on an answer	Button reads *Learned*; memory counter increases	Pass
TC-40	Toggles the theme and opens the memory panel	Theme changes; panel opens	Pass
TC-41	Loads the app at 390 × 844 px	Composer fully on screen; no horizontal	Pass

Test ID	What the browser does	Expected	Result
		scrolling	
TC-42	Uses only the keyboard: Tab to the message box, type, Enter	Message box reachable by Tab; message sent; every visible control has an accessible name	Pass

The first audit found real defects. axe-core reported 2 rule violations (color-contrast, page-has-heading-one) and the contrast audit found 6 token pairs below AA, all of them in small secondary text (sidebar group labels, keyboard hints, memory-kind chips) and, in the light theme, the accent and success colours. The page also had no top-level heading once a chat had started, which hurts screen-reader navigation. They were logged as defects D-11 and D-12, fixed by darkening or lightening the failing tokens (pieces of words; 100 tokens are about 75 words) per theme and adding a persistent (visually hidden) page heading, and re-audited:

Table 5.16: Accessibility audit with axe-core: violations (impact, elements) before and after the fixes

Theme and size	Violations before	Rules passed before	Violations after	Rules passed after
Dark, desktop	color-contrast (serious, 2), page-has-heading-one (moderate, 1)	32	none	35
Dark, phone	page-has-heading-one (moderate, 1)	32	none	35
Light, desktop	color-contrast (serious, 4), page-has-heading-one (moderate, 1)	32	none	35
Light, phone	color-contrast (serious, 2), page-has-heading-one (moderate, 1)	32	none	35

Table 5.17: Colour-contrast failures found and fixed (WCAG 2.2 AA)

Theme	Token pair	Used for	Before	After	Required
dark	faint on bg	Group labels, keyboard hints (11–12 px)	3.26:1	5.29:1	4.5:1
dark	faint on bg-2	Sidebar group labels	3.13:1	5.10:1	4.5:1
light	faint on bg	Group labels, keyboard hints (11–12 px)	2.45:1	5.28:1	4.5:1
light	faint on bg-2	Sidebar group labels	2.32:1	5.01:1	4.5:1
light	accent-2 on bg	Glyphs, links	2.52:1	5.59:1	4.5:1
light	good on bg	Grounded meter, success	2.65:1	4.64:1	3:1

After the fixes the audit reported no violations in either theme at either size, and every token pair met AA. Keyboard-only use of the main path is covered by TC-42. Automated tools catch only part of what makes an interface accessible: the interface was **not** tested with a screen reader or with users who rely on one, and a usability study with real users remains future work (Chapter 6).

As a structured design review, the interface was also checked against Nielsen's ten usability heuristics (Table 5.18). This is the author's own expert review, not a user study; its value is that each heuristic is tied to a concrete feature or to a gap that is admitted.

Table 5.18: Heuristic review of the interface (Nielsen, 1994)

Heuristic	How the interface meets it	Known gap
Visibility of system status	*Searching your memory…*, recall card, streaming, time to first word, download progress	—
Match with the real world	Plain words: *Teach*, *Remember*, *Forget*, *Learned*	—
User control and freedom	Forget any memory; rename or delete chats; 👎 never teaches	No undo after Forget
Consistency and standards	ChatGPT/Ollama conventions; one token system across themes	—
Error prevention	Empty messages rejected; duplicates detected; local-only server	—
Recognition rather than recall	Suggestions on the empty chat; model picker shows installed models	—
Flexibility and efficiency	⌘K new chat; Enter to send; *remember that…* shortcut	Few keyboard shortcuts
Aesthetic and minimalist design	One accent colour; memory details in a panel, not in the thread	—
Help users recover from errors	Setup screen names the missing step (TC-30); toasts explain failures	Model errors shown as raw text
Help and documentation	docs/APP.md, illustrated setup guide, About in Settings	No in-app tour

5.7 Version Control and Implementation Evidence

> **In plain words.** All work is kept in Git, which records every change with its date and a description, so the order of the work and the origin of every result can be traced.

Project repository: **https://github.com/Satasdin/CAMR**. The repository holds the README, a setup walkthrough (`docs/SETUP.md`), the findings log naming the results file behind every number (`docs/FINDINGS.md`), the dependency file (`pyproject.toml`), every experiment configuration (`configs/`), the test suite, the n8n (a visual tool for building automations from connected steps) workflow files, and the scripts that regenerate every figure and this chapter. Results that are small enough (summaries, per-question records, answer and web caches) are committed so that the replay checks run from a fresh clone; downloaded datasets and large databases are excluded by `.gitignore` and regenerated by the scripts. The history holds 253 commits, each an implemented or measured step whose message names what it added.

The work followed GitHub flow: one branch per round of work, merged through a pull request that describes the change and its test plan. Merged so far: Satasdin/CAMR#2 (engine,

harness, findings), Satasdin/CAMR#3 (CAMR Personal 0.2 and the n = 200 study), Satasdin/CAMR#4 (application speed), Satasdin/CAMR#5 (0.2.1), Satasdin/CAMR#6 (GitHub evidence), Satasdin/CAMR#7 (equations, RAG comparison, tuning, LoRA, interface tests) and Satasdin/CAMR#8 (CAMR 0.3, learning as it goes). The work in this chapter (verified answering, large data, the governor, formulas, RAG pipelines, automations) is on branch `claude/engine-build-docs-yfw9u0`. The sweep machines pushed results to their own branches, which were merged into the results database rather than into the code. Continuous integration builds and launch-tests the application on six platforms for every change to it; releases v0.2.0 and v0.2.1 attach the six downloads. The code is MIT-licensed.

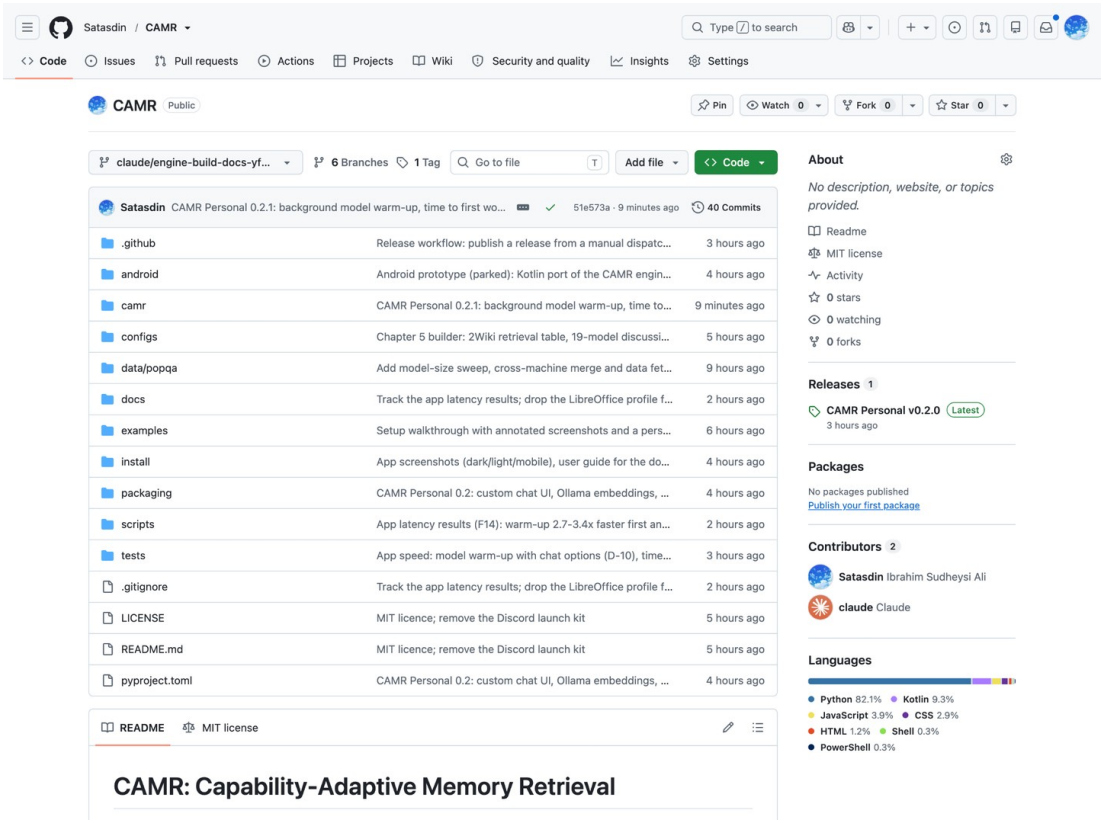


Figure 5.27: Project repository on GitHub (screenshot)

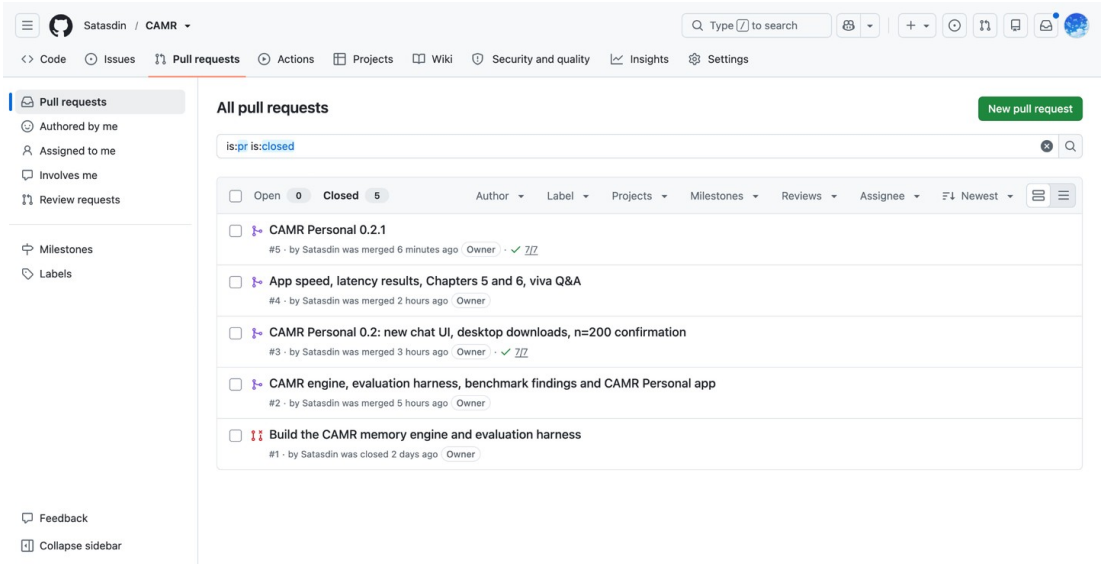


Figure 5.28: Pull requests that merged each round of work (screenshot)

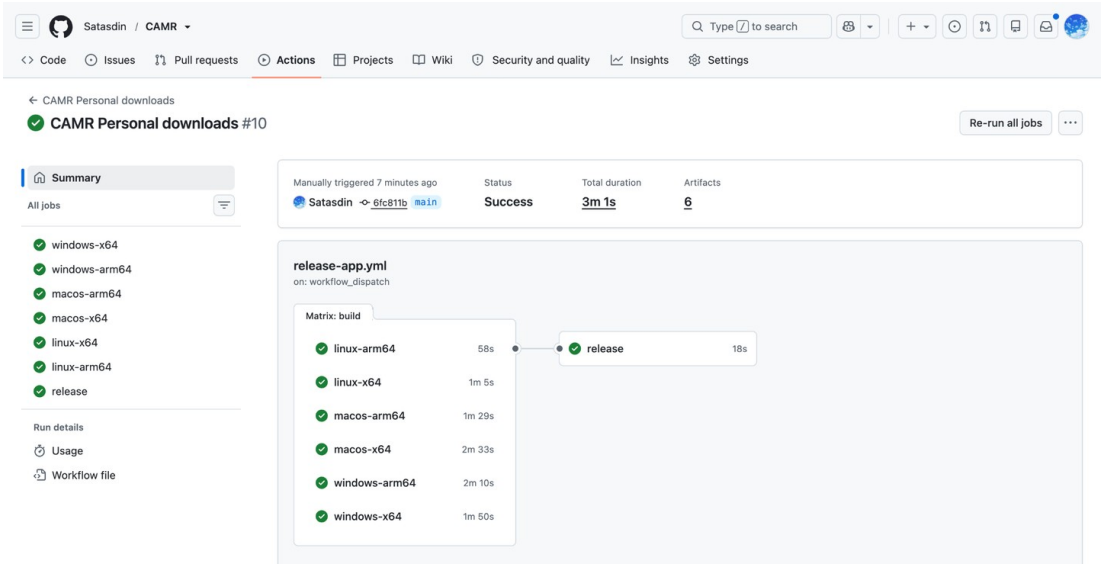


Figure 5.29: Continuous integration: tests, build and launch test on six platforms (screenshot)

Table 5.19: Continuous-integration evidence (GitHub Actions run 36899712846, release v0.2.1)

Target	Runner	Tests	Build	Launch test
Windows x64	windows-2025	pass	pass	pass
Windows ARM64	windows-11-arm	pass	pass	pass
macOS Apple Silicon	macos-15	pass	pass	pass
macOS Intel	macos-15-intel	pass	pass	pass
Linux x64	ubuntu-24.04	pass	pass	pass
Linux ARM64	ubuntu-24.04-arm	pass	pass	pass

Secrets. The Moonshot key was supplied only through an environment variable, masked in every log, and never written to a configuration file, a commit or a screenshot. Cached cloud answers are stored by request text, which contains no key.

5.8 Author Oversight and Use of AI-Assisted Tools

Claude Code, an AI coding assistant provided by Anthropic, was used during implementation and evaluation. Under the student's direction it helped write the engine, the gateway, the experiment scripts and their tests; ran the experiments on the cloud machines; produced the figures and screenshots; and drafted this chapter from the logged results. Kimi K3 (Moonshot AI) was used as a measured system and as the judge of answers that exact matching (the answer must contain the reference answer) marked wrong; its role in each experiment is stated where it appears. The student defined the problem, objectives, requirements and design (Chapters 1–4), chose which experiments to run, including the comparison with modern RAG (retrieval-augmented generation: search for passages and paste them into the prompt) pipelines, the automation scenarios and the cloud-cost accounting, and set the budget for cloud use.

The output was checked against the source code, the automated tests (203), the per-question records and the replay harness, which recomputes the key results and fails if any number differs. The student required that only experiments actually run be reported, and that failures be reported as measured; an earlier draft written before results existed was discarded for that reason. The student made the final decisions and takes full responsibility for the submitted work.

5.9 Evaluation Results, Discussion and Limitations

> **In plain words.** This section answers one question at a time. Most numbers are percentages of a set of questions: *correct* is a right answer, *wrong* is a wrong answer stated as fact (the harmful kind), and *declined* means the system said it did not know (safe, but not useful). Each part ends with where the system fell short.

The results are organised by what the study set out to show, not by the order of the experiments. Each subsection gives the measurement, what it permits the study to conclude, and where the system fell short. Sources are in `results/`; every table is also in `docs/FINDINGS.md`.

5.9.1 Results at a Glance: Where It Was Perfect and Where It Was Not

> **In plain words.** The table gives each claim a verdict: met, partly met, or not met. In short, memory helps whenever the problem is missing knowledge, and does not help when the problem is reasoning, such as maths.

Table 5.20: Scorecard: each claim, its evidence and whether it was met

Claim	Evidence	Result	Verdict
Long-tail facts: a small model with memory matches the cloud model	22-model sweep; PopQA	20 of 22 models reach or beat Kimi K3 closed-book (66.7%) with memory	**Met**
Multi-hop: memory closes part of the gap	HotpotQA, n = 200	3B + CAMR 54.0% vs Kimi K3 51.8%; 0.5B gains +23 points but stays far below	**Met for ≥ 3B; partial for 0.5B**
Second hop found without a graph	2Wiki retrieval, n = 500	Full-support recall 35.8% → 78.4% at the same budget	**Met**
Reasoning gaps closed by memory	GSM8K	No: residual 83.3 points; worked examples hurt the 0.5B model	**Not met**
Few wrong facts stated	Held-out PopQA, n = 200, judged	3.0% wrong stated (Kimi K3 alone 31.5%), 60.5% correct (Kimi 67.5%)	**Met, at a cost in coverage**
Automations: no wrong reply to a customer	n8n desk, 60 tickets, Kimi K3 + CAMR	100.0% of answerable tickets resolved, 0.0% wrong replies, 100.0% of unknowable tickets escalated	**Met (after D-33)**
Agents act only on company facts	20 operations tasks, Kimi K3 + CAMR	100.0% success, 0.0% hallucinated actions	**Met**
Learning without training: formulas	Fresh draw, invented formulas	Kimi K3 100% after 4 corrections; 0.5B 0% → 70.8% on invented formulas, 50% overall	**Met for Kimi; partial for 0.5B**
Memory beats fine-tuning	Same documents / same corrections	Facts: 71.7% vs 13.3%. Formulas: LoRA higher on trained wording (88.3% vs 50.0%), lower on derived problems and lost GSM8K	**Met for facts; mixed for formulas**
Large data within device limits	123,301 paragraphs; governor	Searchable in 335 s; stops at every limit; 0 duplicates	**Met**
Retrieval ≤ 10% of answer time (NFR-02)	Device profile	56% for 0.5B (contention); met only for ≥ 1.5B models	**Not met for 0.5B**
Reproducible on demand	`reproduce.py --twice`	16/16 checks, no key, no network	**Met**

Three patterns run through the table. Memory works when the gap is **knowledge** (facts, private documents, recent events, invented formulas) and does not when the gap is **reasoning**. Verification turns a small model's confident errors into withheld answers, at the price of answering less. And a stronger model reading the same memory does better wherever reading is the hard part, which is why the engine is more useful beside a strong model than a replacement for one.

5.9.2 Getting the Evidence to the Model (Objective ii)

> **In plain words.** Before a model can answer from memory, the right notes have to be found. For questions that need two facts, CAMR follows the name found in the first note to a second note; at the same reading budget this more than doubled how often both facts were found (35.8% to 78.4%).

Before any model answers, the first question is whether the right text reaches it. This was measured with no language model at all: 500 questions per benchmark over one store of 38,054 notes from 23,524 real paragraphs, counting a question as served only if **every** gold paragraph reached the context (Equation 5.24). Table 5.21 gives the result.

Table 5.21: Full-support recall by read path, 500 questions each (results/retrieval_eval/tables/retrieval_eval.json)

Read path (512 tokens unless stated)	**2Wiki: all evidence found**	**Tokens**	**HotpotQA: all evidence found**	**Tokens**
Similarity only (baseline)	35.8%	471	70.8%	470
+ recency and importance (proposal's Eq. 4.1)	34.6%	470	65.4%	467
+ gating	33.4%	408	64.6%	384
+ entity bridging	75.2%	472	70.0%	470
Similarity + gating + bridging (CAMR)	**78.4%**	**439**	70.8%	420
Standard RAG, top 10	39.2%	767	78.6%	800
CAMR at 1,024 tokens	**84.0%**	821	**83.0%**	728

Where it was strong. On 2WikiMultiHopQA, entity bridging lifted full-support recall from 35.8% to 78.4% at the same budget with 7% fewer tokens (pieces of words; 100 tokens are about 75 words): twice the recall of top-10 RAG (retrieval-augmented generation: search for passages and paste them into the prompt) with 43% fewer tokens. These questions are compositional (*Who is the father of the director of film X?*); the second paragraph shares almost no words with the question, so similarity cannot find it, but the first paragraph names it, and bridging follows the name; section 5.9.7 follows two real questions through it.

Where it was not. On HotpotQA (public questions that need two Wikipedia articles), where both entities are usually named in the question, CAMR is plain RAG with a token cap (70.8% against 68.2% for top-5 RAG at equal precision), and top-10 RAG finds more (78.6%) by reading twice the text. The proposal's composite score (Equation 4.1, recency and importance) **lowered** recall (the share of the needed evidence that the search found) on both benchmarks (−1.2 and −5.4 points), because benchmark questions are independent and

recency carries no information about them; it was dropped (D-06). Bridging also costs recall at a 256-token budget (−6.4 points), where bridged notes crowd out direct hits.

5.9.3 Answers That Are Verified or Not Stated (No Hallucination)

> **In plain words.** Alone, the smallest model (0.5B) got 11.5% of the 200 test questions right and stated a wrong answer for 88.5%. With CAMR's verified answering it got 60.5% right and stated a wrong answer for only 3.0%. The cloud model alone got 67.5% right but stated a wrong answer for 31.5%. The price of being careful is that some questions get no stated answer.

A small model with memory still states wrong facts: in the earlier pipeline (a fixed sequence of steps) the 0.5B model read memory and answered every question, and half of its answers were wrong. Verified answering (section 5.4.4) was built to change that, and every method was chosen on the 200 development questions (Table 5.22).

Table 5.22: Development set: each step of verified answering (200 questions, judged; results/verified/answers_dev*.json)

Method	Correct	Wrong answer stated	Same-name mix-up (named)	Not confirmed (no answer stated)
Before: 0.5B reads, every answer stated	45.0%	**51.5%**	–	–
The same answers, checked	42.0%	9.0%	20.5%	28.5%
+ infobox facts as notes	48.0%	5.0%	20.0%	27.0%
+ fact reader and compression	55.5%	3.5%	18.0%	23.0%
+ strict checks (kind, relation)	55.5%	2.5%	17.5%	24.5%
+ look up the subject when no note is titled with it (chosen)	**59.0%**	**2.5%**	20.5%	18.0%
chosen + Kimi K3 second reader and cross-check	60.0%	3.0%	21.0%	16.0%
strict + 3B second reader	57.0%	3.0%	18.0%	22.0%
chosen + hybrid (keyword) recall	59.0%	2.5%	20.5%	18.0%

Two findings from development shaped the design. A larger second reader (3B or Kimi K3) added at most one point of correct answers and up to 1.5 points of wrong ones: when the answer is not in what memory read, a stronger reader cannot find it either. And *knowing* a subject had to mean having a note titled with it; notes that merely mention it had stopped lookups (+3.5 points once fixed). The chosen pipeline was then run **once** on the held-out (kept aside until the end for a fair test) set (Figure 5.30, Table 5.23).

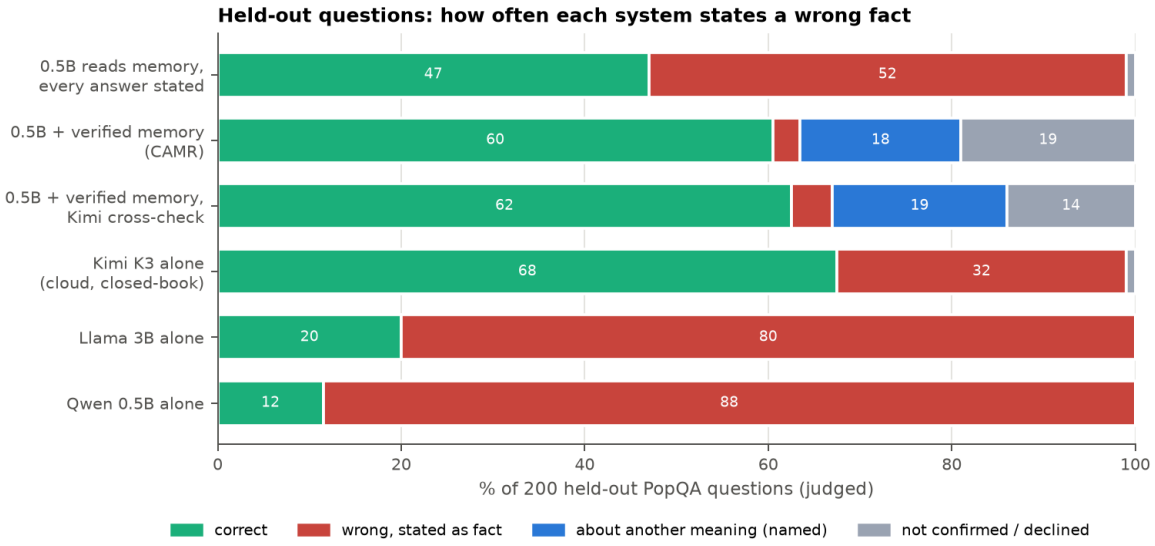


Figure 5.30: Held-out set (200 questions no choice was made on): outcomes of each system, judged

Table 5.23: Held-out set: verified memory against the models alone (results/verified/judged_heldout.json)

System	Correct	Wrong, stated as fact	Same-name mix-up (named)	Not confirmed (no answer stated)
0.5B + verified memory (CAMR)	**60.5%**	**3.0%**	17.5%	19.0%
0.5B + verified memory, Kimi K3 cross-check	62.5%	4.5%	19.0%	14.0%
Before: 0.5B reads, every answer stated	47.0%	52.0%	–	–
Kimi K3 alone (cloud)	67.5%	31.5%	–	1.0%
Llama 3B alone	20.0%	80.0%	–	0.0%
Qwen 0.5B alone	11.5%	88.5%	–	0.0%

What this shows. On questions no choice was made on, the frozen 0.5B model with verified memory states a wrong fact about once in thirty-three answers (3.0%), against about once in three for the cloud model answering from its weights (31.5%), and it is correct 60.5% of the time against the cloud model's 67.5%. That is the trade the design makes: less coverage for far fewer wrong statements. Of the correct answers, 102 were read straight from a fact line (a stored 'subject: relation: value' line read without the model) with no model call, and 19 were read by the model and confirmed by the checks.

Where it was not perfect. Table 5.24 shows one real case of every outcome.

Table 5.24: One held-out case of each outcome (Qwen 0.5B, verbatim)

Outcome	Question	What was stated	Why
Correct, no model call	Who was the screenwriter for Mississippi Burning?	Chris Gerolmo	Fact line *Written by: Chris Gerolmo*
Correct, model read	Who is the author of The Disk?	Jorge Luis Borges is the author of "The Disk"	Sentence *"The Disk" is a 1975 short story written by … Jorge Luis Borges*
Other meaning, named	Who was the producer of Happy? (gold: Allu Aravind)	Happy (Pharrell Williams song): Pharrell Williams; Happy (2011 film): …	The name has many meanings; each answer names its own
Wrong, stated	In what country is Almedina? (gold: Portugal)	Spain	The stored line *Almedina: Country: Spain* is about another Almedina
Wrong, stated	What is Hermann Ahlwardt's occupation? (gold: journalist)	Teacher, opinion writer	Quoted from the infobox; the benchmark's answer differs
Correct, withheld	Who is the author of The Incoherence of the Philosophers?	– (lead: al-Ghazali)	The kind check expects a capitalised name
Wrong, withheld	Who was the screenwriter for Boring?	– (lead: Steven Spielberg)	Not in memory: the model guessed, and the check stopped it

All six wrong stated answers on the held-out set repeat a stored line word for word: the model did not invent them. They come from a page about another meaning of the name, or from a source that disagrees with the benchmark. The check guarantees that a stated answer is *supported by memory*, not that memory is right. The other costs are coverage: 19% of questions got no stated answer, and 3.5% of those had a correct lead that the checks were too strict to accept. On the 200-question test stream (seen once before round 2) the same pipeline scored 52.5% correct with 5.0% wrong; Kimi K3 (a large cloud model by Moonshot AI) alone scored 68.0% with 31.0% wrong.

5.9.4 Which Model to Run CAMR With

> **In plain words.** 22 models from 0.5 to 14.8 billion parameters were tested alone and with CAMR on the same questions. With memory, small models know as much as big ones: from about 1.5 billion parameters, models with CAMR beat the cloud model on less-known facts. Bigger models still read and reason better (two-step questions and maths), but on a laptop they are much slower. For a 16 GB laptop the best choice is a 1.5B model for facts and a 3B model for questions that need several documents.

Two experiments answer this. The 22-model sweep measured what memory adds to each model on three task types; the held-out re-test swapped the reader inside the verified pipeline, keeping everything else fixed. The sweep covers 22 models from 0.49B to 14.8B parameters, every one on the same store and the same questions. Table 5.25 lists all of them, smallest first, and Table 5.26 sums them up by size class. No 5B model was in the sweep; the size classes cover those sizes by their neighbours.

Table 5.25: All 22 models of the size sweep: accuracy alone → with CAMR (bold: at or above Kimi K3), decode speed with CAMR and seconds per answer with CAMR (results/pilot/tables/model_sweep.json; F10, F10b)

Model (parameters, quantisation)	Long-tail facts (30 q)	Multi-hop (30 q)	Arithmetic (12 q)	Writing speed (tokens/s)	s / answer	Machine
qwen2.5:0.5b (0.49B, Q4_K_M)	6.7 → **66.7%**	10.0 → 30.0%	25.0 → 16.7%	48.9	1.4	E
gemma3:1b (1.00B, Q4_K_M)	6.7 → **66.7%**	16.7 → 40.0%	33.3 → 16.7%	23.9	3.5	C
llama3.2:1b (1.2B, Q8_0)	10.0 → 63.3%	16.7 → 40.0%	58.3 → 33.3%	33.7	1.9	E
qwen2.5:1.5b (1.5B, Q4_K_M)	10.0 → **80.0%**	16.7 → 50.0%	58.3 → 41.7%	19.9	2.1	E
smollm2:1.7b (1.7B, Q8_0)	20.0 → **73.3%**	13.3 → 36.7%	33.3 → 41.7%	13.8	6.4	C
granite3.3:2b (2.5B, Q4_K_M)	16.7 → **76.7%**	26.7 → 43.3%	58.3 → 58.3%	12.2	8.5	C
qwen2.5:3b (3.1B, Q4_K_M)	6.7 → **80.0%**	33.3 → 43.3%	25.0 → 33.3%	11.0	4.2	E
falcon3:3b (3.2B, Q4_K_M)	10.0 → **70.0%**	13.3 → 50.0%	33.3 → 66.7%	14.5	6.6	C
llama3.2:3b (3.2B, Q4_K_M)	23.3 → **73.3%**	16.7 → **56.7%**	75.0 → 75.0%	10.4	4.2	E
phi4-mini:3.8b (3.8B, Q4_K_M)	13.3 → **80.0%**	10.0 → 40.0%	83.3 → 91.7%	10.7	9.4	C
gemma3:4b (4.3B, Q4_K_M)	20.0 → **73.3%**	20.0 → **53.3%**	75.0 → 50.0%	9.1	5.6	E
yi:6b (6.0B, Q4_0)	13.3 → 63.3%	16.7 → 43.3%	66.7 → 41.7%	10.2	9.4	F
mistral:7b (7.2B, Q4_K_M)	30.0 → **76.7%**	13.3 → 43.3%	33.3 → 50.0%	5.1	10.8	D
olmo2:7b (7.3B, Q4_K_M)	23.3 → **76.7%**	36.7 → **58.6%** †	63.6 → 83.3% †	5.3	8.9	D
qwen2.5:7b (7.6B, Q4_K_M)	23.3 → **70.0%**	36.7 → **60.0%**	83.3 → 83.3%	6.0	8.3	B
llama3.1:8b (8.0B, Q4_K_M)	20.0 → **73.3%**	36.7 → **56.7%**	83.3 → 58.3%	5.4	9.3	B
granite3.3:8b (8.2B, Q4_K_M)	16.7 → **80.0%**	33.3 → 50.0%	58.3 → 72.7% †	4.6	13.1	D
gemma2:9b (9.2B, Q4_0)	33.3 → **76.7%**	33.3 → 50.0%	75.0 → 66.7%	6.5	16.5	B
falcon3:10b (10.3B, Q4_K_M)	20.0 → **70.0%**	26.7 → **56.7%**	16.7 → 50.0%	3.9	14.1	F
gemma3:12b (12.2B, Q4_K_M)	33.3 → **79.3%** †	30.0 → **63.0%** †	91.7 → 90.0% †	3.7	16.2	D
olmo2:13b (13.7B, Q4_K_M)	30.0 → **70.0%**	36.7 → **53.3%**	75.0 → 91.7%	2.8	18.1	F
qwen2.5:14b (14.8B, Q4_K_M)	13.3 → **73.3%**	46.7 → **63.3%**	66.7 → 91.7%	3.0	17.7	B
Kimi K3 (cloud), no memory	*66.7%*	*53.3%*	*100.0%*	–	*7.7*	cloud

Machine is the 4-core cloud machine the model ran on (B, D, E, F: Xeon at 2.1 GHz; C: Xeon at 2.8 GHz). Accuracy is comparable across machines, speed only within one. One question is 3.3 points on the facts and multi-hop (needing two facts joined) sets and 8.3 points on arithmetic, so a gap of one or two questions between two models is noise. † One to three questions of that run failed with a runtime error; accuracy is over the answered ones (F10).

Table 5.26: The sweep by size class: mean accuracy alone → with CAMR, models at Kimi K3's level, and time per answer

Size class	Models	Facts	Multi-hop	At Kimi on multi-hop	Arithmetic helped / hurt	s / answer (median, 2.1 GHz)
About 1B	3	7.8 → 65.6%	14.4 → 36.7%	0 of 3	0 / 3	1.7
1.5–2.5B	3	15.6 → 76.7%	18.9 → 43.3%	0 of 3	1 / 1	2.1
3–4B	5	14.7 → 75.3%	18.7 → 48.7%	2 of 5	3 / 1	4.2

Size class	Models	Facts	Multi-hop	At Kimi on multi-hop	Arithmetic helped / hurt	s / answer (median, 2.1 GHz)
6–9B	7	22.9 → 73.8%	29.5 → 51.7%	3 of 7	3 / 3	9.4
10–15B	4	24.2 → 73.2%	35.0 → 59.1%	4 of 4	3 / 0	16.9

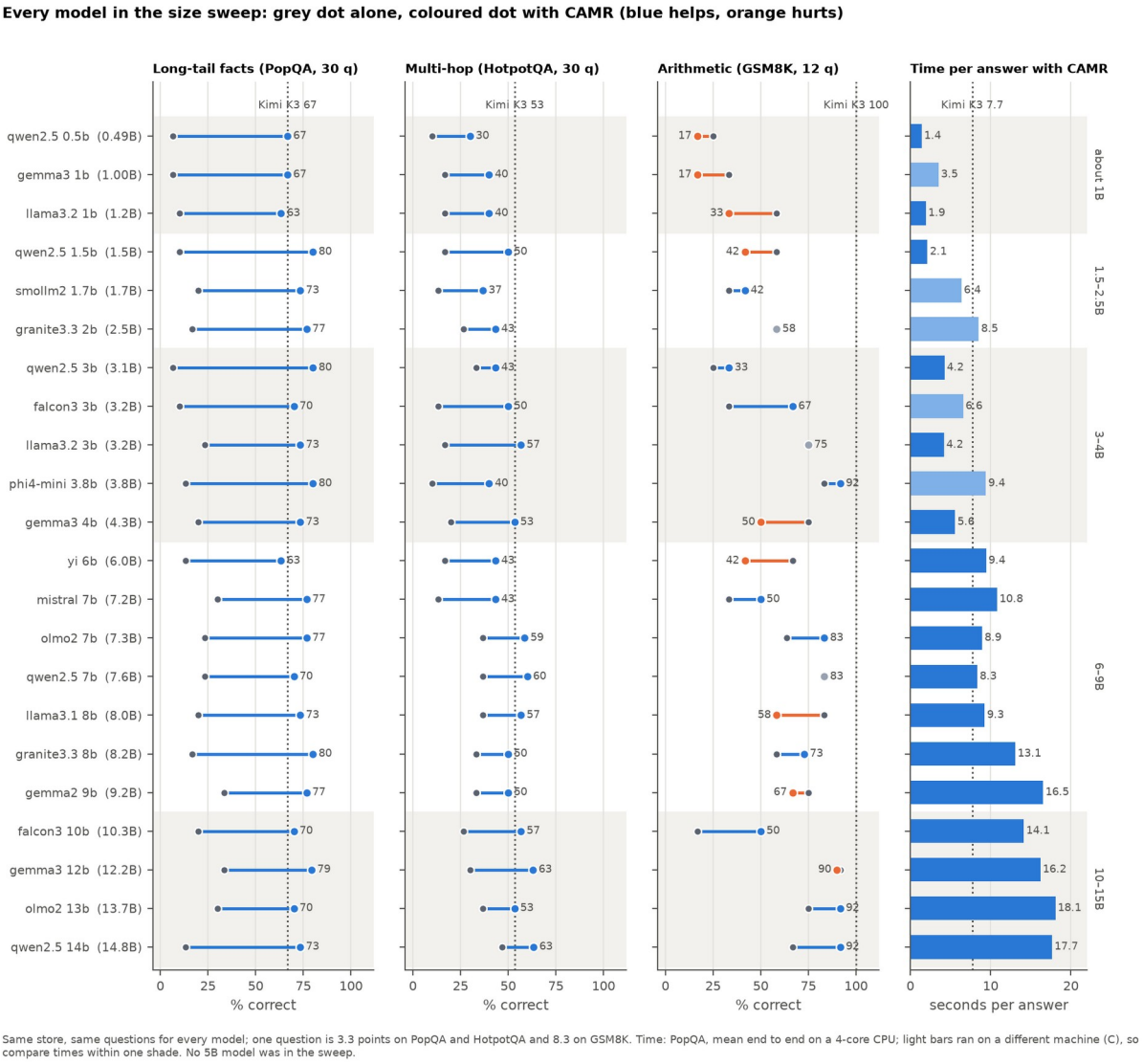


Figure 5.31: Every model of the sweep by name and size: accuracy alone (grey) and with CAMR (blue helps, orange hurts) on three task types, and time per answer

Facts stop improving at about 1.5B; reading across documents keeps improving; time grows fastest. Memory lifts every model on long-tail (about less famous subjects) facts, from a mean of 8% alone to 66% for the smallest class, and from 1.5B upwards every class averages 73–77% with CAMR: 20 of 22 models reach or beat Kimi K3's 66.7%, and a 14B model with memory is no better than a 1.5B one. Multi-hop reading is different: the class means with CAMR climb 37 → 43 → 49 → 52 → 59%, and only 9 of 22 models reach Kimi K3's 53.3% (none below 3B). Time per answer climbs faster than either: on the 2.1 GHz

machines the class medians are 1.7 → 2.1 → 4.2 → 9.4 → 16.9 s, and all 11 models of 6B and above measured there took longer than Kimi K3's 7.7 s. **Where small models fall short:** worked examples lowered arithmetic for all 3 models of about 1B, and the 0.5B model gains least on multi-hop. **Where large models fall short:** speed. Their extra parameters paid off on multi-hop reading and arithmetic, not on facts.

Table 5.27: Held-out set, verified pipeline with the reader swapped (judged); speed and memory measured alone on the 4-core CPU (results/models/speed.json)

Reader	Correct	Wrong answer stated	Same-name mix-up	Not confirmed (no answer stated)	s / answer	RAM, GB
qwen2.5:0.5b	60.5%	3.0%	17.5%	19.0%	0.31	0.6
qwen2.5:1.5b	60.5%	3.5%	18.0%	18.0%	0.52	1.3
llama3.2:3b	61.0%	3.0%	18.5%	17.5%	1.01	2.8
phi4-mini (3.8B)	60.0%	3.0%	18.0%	19.0%	0.99	3.6
qwen2.5:7b	60.5%	3.5%	19.5%	16.5%	2.04	4.8

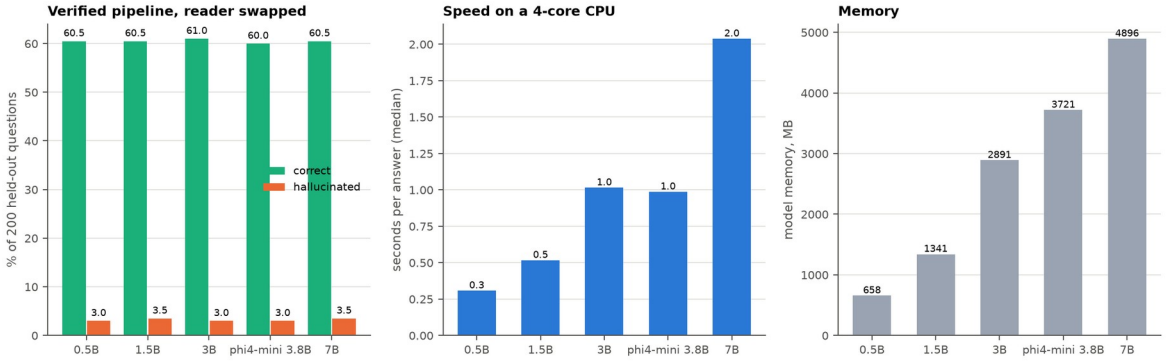


Figure 5.32: The verified pipeline with five readers: accuracy and wrong statements, speed and memory

What the reader swap shows. Every reader from 0.5B to 7B landed within 1.0 points on correct answers (60.0%–61.0%) and within 0.5 points on wrong stated ones (3.0%–3.5%). The reason is structural: 140 of the 200 questions were answered from a fact line with no model call, whatever the reader. Of the other 60 (with the 0.5B reader), 19 were answered correctly, and 36 of the remaining 41 had no answer anywhere in what memory read, which no reader can fix. In this pipeline a bigger model buys nothing but time and memory. Measured alone on the 4-core CPU, a short answer took 0.31 s with the 0.5B model, 1.01 s with the 3B model and 2.04 s with the 7B model, whose runtime held 4.8 GB of RAM against 0.6 GB. **The choice therefore depends on the task.** For verified fact answering, the smallest model is enough (0.5B or 1.5B). For multi-hop reading, 3B is the smallest model

that matched the cloud model, and the gain from memory grows with the reader (section 5.9.6). For arithmetic, phi4-mini (3.8B) with worked examples reached 91.7%, but memory helps arithmetic only in some model families, so it is a per-model choice. From 6B up, every model with memory answered more slowly than the cloud model on this hardware. The recommended configuration for a 16 GB CPU-only device is a **1.5B model for facts and a 3B model when questions need reading across documents**, with the cloud model reserved for what memory cannot close.

5.9.5 CAMR Against Modern RAG Pipelines

> **In plain words.** RAG is today's standard way to give a model documents. Eight RAG designs and CAMR answered the same 200 questions from the same documents. With a small model, CAMR was right more often (62.5% against 45.5% for the best RAG design) and almost never stated a wrong answer (3.5% against 47.0%). With the cloud model, the best RAG design answered more questions correctly (71.5% against 63.0%) but stated about 5 times as many wrong answers (21.0% against 4.0%).

All nine methods of section 5.5.4 (eight RAG designs and CAMR) answered the same 200 held-out questions from the same store of 10,443 notes. How to read the tables: *Correct* and *Wrong answer stated* are judged (re-checked by the cloud model, since right answers can be worded differently) as in section 5.9.3. *Same-name mix-up* is an answer about another person or thing with the same name, which CAMR labels (only CAMR does this). *Said it did not know* means the method gave no answer (for CAMR and the check: it did not state one). *Answer was in what it read* shows whether the right answer was anywhere in the text the model was given: when it was not, no model could answer. *Text read*, *Model time* and *Search time* show the cost of each answer.

Table 5.28: RAG pipelines and CAMR, 200 held-out questions, reader Qwen 2.5 0.5B (results/ragbench/run_*_qwen2_5_0_5b.json)

Method	Correct	Wrong answer stated	Same-name mix-up (named)	Said it did not know	Answer was in what it read	Text read (tokens)	Model runs	Model time (s)	Search time (ms)
Dense (naive RAG)	43.5%	50.0%	–	6.5%	64.0%	259	1.00	3.88	12
BM25	29.0%	64.0%	–	7.0%	40.5%	176	1.00	1.52	3
Hybrid (RRF)	37.0%	56.0%	–	7.0%	56.5%	191	1.00	1.70	13
Hybrid + reranker	45.5%	47.0%	–	7.5%	72.0%	290	1.00	2.48	979
HyDE	33.0%	59.0%	–	8.0%	54.5%	318	2.00	6.56	354
Corrective RAG	42.0%	46.0%	–	12.0%	71.5%	267	1.00	1.86	1305
Reranker + abstain	45.0%	38.0%	–	17.0%	72.0%	262	0.90	2.24	700
Reranker + CAMR check	17.0%	8.5%	–	74.5%	72.0%	290	1.00	2.48	701
CAMR (verified)	62.5%	3.5%	18.5%	15.5%	64.0%	52	0.27	0.21	14

Table 5.29: RAG pipelines and CAMR, 200 held-out questions, reader Llama 3.2 3B (results/ragbench/run_*_llama3_2_3b.json)

Method	Correct	Wrong answer stated	Same-name mix-up (named)	Said it did not know	Answer was in what it read	Text read (tokens)	Model runs	Model time (s)	Search time (ms)
Dense (naive RAG)	53.5%	14.0%	–	32.5%	64.0%	247	1.00	5.96	12
Hybrid + reranker	56.0%	15.5%	–	28.5%	72.0%	277	1.00	6.26	993
Corrective RAG	55.0%	15.5%	–	29.5%	71.5%	255	1.00	5.81	1102
Reranker + CAMR check	37.0%	9.5%	–	53.5%	72.0%	277	1.00	6.26	628
CAMR (verified)	63.5%	3.5%	20.0%	13.0%	64.0%	49	0.27	0.57	13

Table 5.30: RAG pipelines and CAMR, 200 held-out questions, reader Kimi K3 (results/ragbench/run_*_kimi-k3.json)

Method	Correct	Wrong answer stated	Same-name mix-up (named)	Said it did not know	Answer was in what it read	Text read (tokens)	Model runs	Model time (s)	Search time (ms)
Dense (naive RAG)	68.0%	20.0%	–	12.0%	64.0%	305	1.00	5.40	12
Hybrid + reranker	71.5%	21.0%	–	7.5%	72.0%	336	1.00	5.69	2922
Corrective RAG	71.5%	20.5%	–	8.0%	71.5%	314	1.00	5.30	5539
Reranker + CAMR check	36.0%	7.0%	–	57.0%	72.0%	336	1.00	5.69	2951
CAMR (verified)	63.0%	4.0%	20.5%	12.5%	64.0%	65	0.27	1.80	373

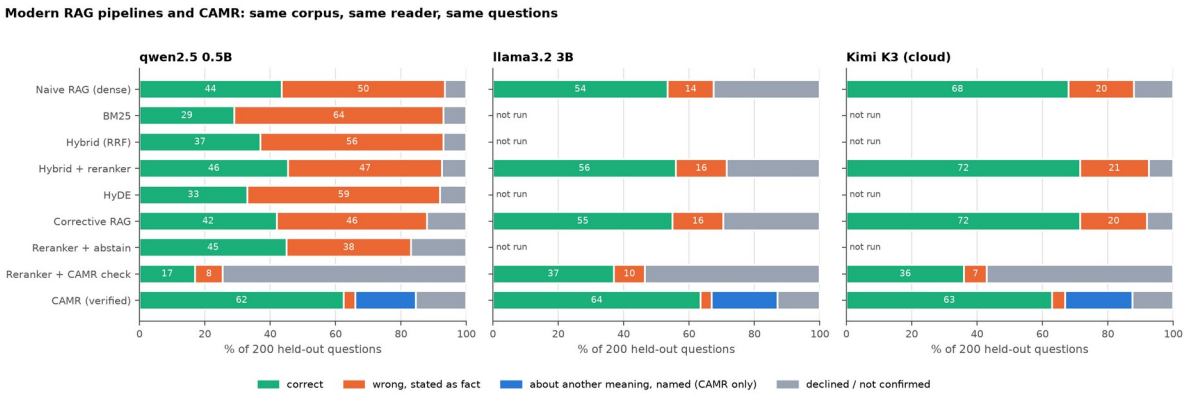


Figure 5.33: Outcomes of every pipeline per reader: correct, wrong but stated, declined

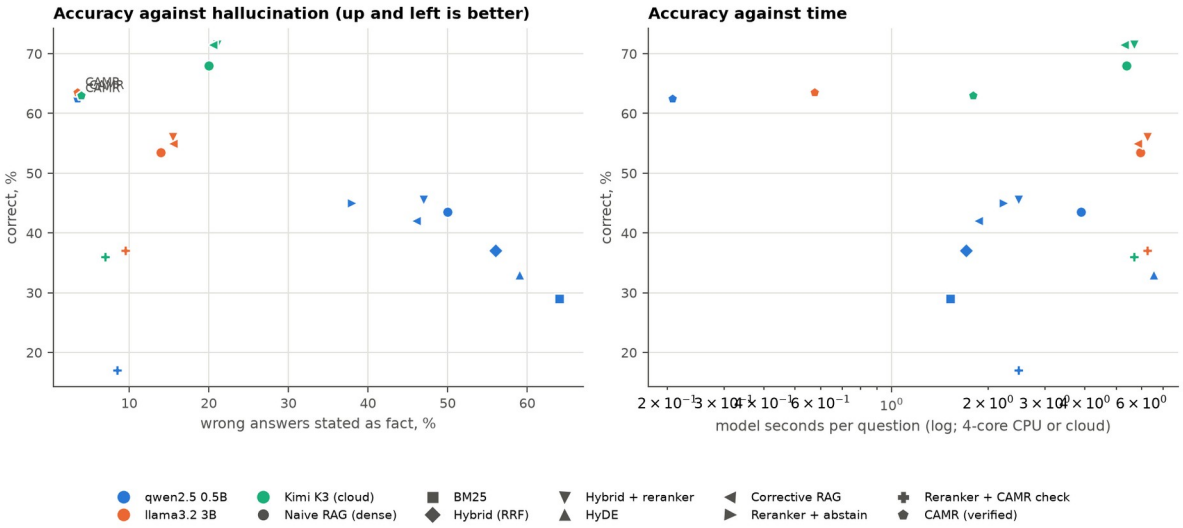


Figure 5.34: Accuracy against wrong statements, and accuracy against model time, for every pipeline and reader

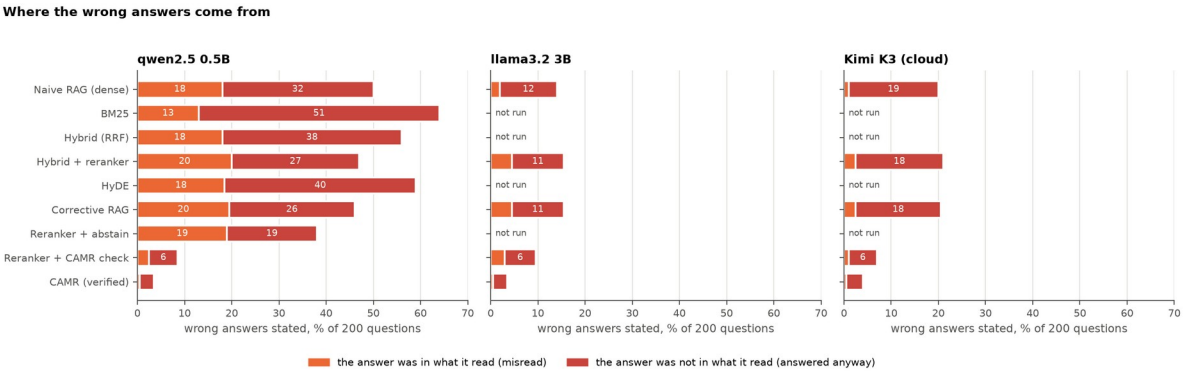


Figure 5.35: Where the wrong answers come from: the answer was in what the reader got (misread) or it was not (answered anyway)

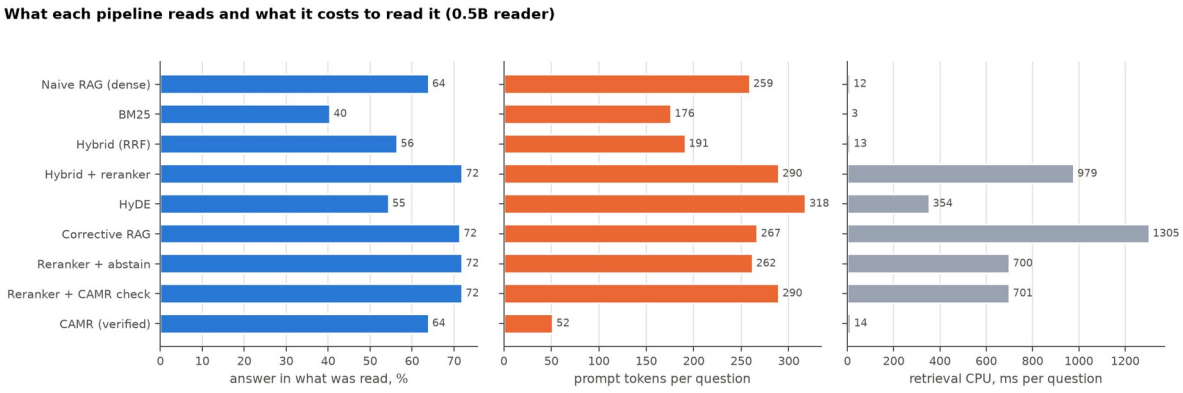


Figure 5.36: What each pipeline reads and what reading costs (0.5B reader): answer in context, prompt tokens, retrieval CPU time

With Kimi K3 as the reader, the most accurate RAG pipeline was hybrid + reranker (71.5% correct), but it also stated a wrong answer to 21.0% of questions; CAMR was correct on

63.0%, stated a wrong answer to 4.0%, gave an answer about a named other meaning for 20.5%, and declined 12.5%, with 0.27 model calls and 65 prompt (the full text handed to the model) tokens per question against 1.00 and 336. **With the 0.5B model as the reader**, the most accurate RAG pipeline was hybrid + reranker (45.5% correct), but it also stated a wrong answer to 47.0% of questions; CAMR was correct on 62.5%, stated a wrong answer to 3.5%, gave an answer about a named other meaning for 18.5%, and declined 15.5%, with 0.27 model calls and 52 prompt tokens per question against 1.00 and 290. **With the 3B model as the reader**, the most accurate RAG pipeline was hybrid + reranker (56.0% correct), but it also stated a wrong answer to 15.5% of questions; CAMR was correct on 63.5%, stated a wrong answer to 3.5%, gave an answer about a named other meaning for 20.0%, and declined 13.0%, with 0.27 model calls and 49 prompt tokens per question against 1.00 and 277. **With a small reader the order reverses.** CAMR with the 0.5B model was correct on 62.5%, more than any of the eight RAG pipelines (best: hybrid + reranker, 45.5%), and stated a wrong single claim for 3.5% against 47.0%. With CAMR the reader hardly matters: the 0.5B model and Kimi K3 gave the same outcome on 187 of 200 questions (62.5% against 63.0% correct), because most answers come from a fact line with no model call. With RAG the reader is everything: reranked RAG went from 45.5% with the 0.5B model to 71.5% with Kimi K3. **CAMR's check bolted onto RAG** cut Kimi K3's wrong answers from 21.0% to 7.0% but declined 57.0%, and 72 of those 114 declined answers were correct: under the standard RAG prompt the model answers in full sentences (… *is al-Ghazali, a Muslim polymath and student of the Asharite school* …), and the check requires every word of the claim in one stored sentence. The check is built for the short answers CAMR's own prompt asks for; it is not a filter for any answer. **What the comparison means.** Modern RAG answers more questions, and with a strong reader it answers more of them correctly. It does so by stating whatever its reader concludes, so roughly one answer in five from the cloud model, and one in two from a small model, is wrong but presented as fact. CAMR answers fewer questions and states very few wrong ones, because it reads labelled facts directly and states nothing a stored sentence does not support; its remaining errors are answers about another meaning that it names, which a reader can see. Where a wrong answer costs more than an unanswered one (a customer reply, an invoice, an agent's action), that trade is the point; where coverage matters most and the reader is a strong cloud model, a reranked RAG pipeline is the better choice; with a small on-device reader, CAMR is better on both counts. Part of CAMR's lower coverage is retrieval, not caution: the answer was in what the reader received for 72.0% of questions with reranked RAG and 64.0% with CAMR, the same as plain dense retrieval;

CAMR has no reranker (a second model that re-orders search results), and adding one is an obvious next step. Corrective RAG's web fallback added nothing on this corpus, because the corpus already held the pages the lookups would fetch.

Table 5.31: The same held-out questions through three pipelines (reader in brackets; verbatim, shortened)

Question (gold)	**Hybrid + reranker**	**Corrective RAG**	**CAMR**
Who was the screenwriter for For Better or Worse? (*Jeff Nathanson*)	Based on the context provided, the screenwriter for For Better or Worse was Brent Maddock, who is described as [✗]	I don't know. The provided context only discusses "Worse" as a disambiguation term (referring to a creek and a [declined]	Jeff Nathanson [declined]
What genre is Nancy Drew and the Hidden Staircase? (*mystery film*)	Nancy Drew and the Hidden Staircase is a **Mystery** novel. It is the second volume in the Nancy Drew Mystery [✗]	Nancy Drew and the Hidden Staircase is a **mystery** novel. It is the second volume in the Nancy Drew Mystery [✗]	Mystery [✓]
Who is the author of The Incoherence of the Philosophers? (*Al-Ghazali*)	The author of The Incoherence of the Philosophers is al-Ghazali, a Muslim polymath and student of the Asharite [✓]	The author of The Incoherence of the Philosophers is al-Ghazali, an 11th-century Muslim polymath and student o [✓]	al-Ghazali [declined]

5.9.6 Multi-Hop Questions and the Residual Gap (Objectives iii and iv)

> **In plain words.** Questions that need two facts are hard for small models even when both facts are found, because the model has to combine them. From about 3 billion parameters, a small model with memory matched the cloud model on these questions; the smallest model improved but stayed well behind.

The pilot (30 questions per task) showed what memory can and cannot close. With the same notes, the gap between a 0.5B model and a 20B model was 10.0 points on single facts, 23.3 on multi-hop questions and 83.3 on arithmetic reasoning: **memory closes knowledge gaps, not reasoning gaps**. Most of the remaining multi-hop gap was reading: 16 of the 0.5B model's 21 misses (76%) had the answer in its context. The multi-hop result was then confirmed on 200 HotpotQA questions with the configuration fixed in advance.

Table 5.32: HotpotQA, 200 held-out questions: exact match alone and with CAMR, paired bootstrap 95% CI (results/hotpot200/camr.sqlite)

Model	**Alone**	**With CAMR**	**Gain (95% CI)**	**s / answer**
qwen2.5:0.5b	3.5%	**26.5%**	+23.0 pts (+17.0 to +29.0)	2.4
qwen2.5:1.5b	10.5%	**41.0%**	+30.5 pts (+23.0 to +38.0)	2.8
llama3.2:3b	19.5%	**54.0%**	+34.5 pts (+27.0 to +42.0)	5.8
Kimi K3 (cloud)	51.8%	**66.3%**	+14.6 pts (+9.0 to +20.6)	9.5

With CAMR, a 3B model (54.0%) matched the cloud model closed-book (51.8%) at 5.8 s instead of 18.4 s per answer, and every gain excluded zero. The cloud model with the same notes reached 66.3%: the 12-point residual (what is left over) is what a larger reader still buys when both read the same evidence. Gains grew with the reader (+23, +30.5, +34.5 points), which is the first argument against the smallest model for multi-hop work (section 5.9.4).

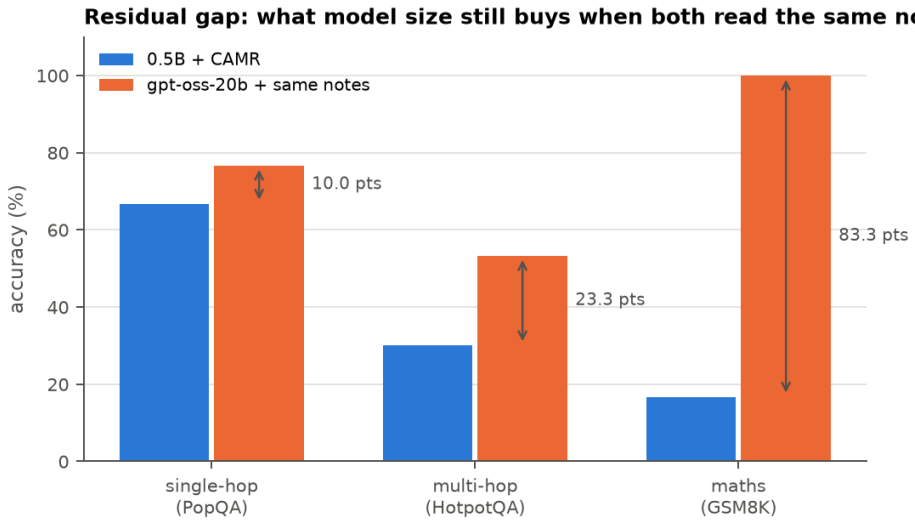


Figure 5.37: Residual gap: the 0.5B model and a 20B model reading the same notes, per task type (pilot)

Budget (objective v). For the 0.5B model, 128 tokens of memory was best for single facts (76.7%, against 66.7% at 512 and 1,024): more notes distracted it. Multi-hop peaked at 512 tokens. Arithmetic never benefited from memory at any budget, and worked examples **lowered** the 0.5B model's GSM8K (public school-maths word problems) score from 33.3% to 16.7%.

5.9.7 What the Models Actually Answered: Worked Examples

> **In plain words.** Real answers, word for word, with and without memory, so the numbers above can be seen in practice.

Aggregate accuracy hides how an answer is won or lost. This subsection follows single questions through the engine, with and without memory, using the logged prompts and answers verbatim (abridged only where marked …). The examples were chosen to show each mechanism and each failure mode, not to flatter the engine; `scripts/extract_examples.py` pulls them from the result databases.

Example 1: a long-tail fact, and an ambiguous name. *What genre is Fahrenheit?* PopQA's intended entity (a named person, place or thing) is the Taiwanese band *Fahrenheit*, and the accepted answers are mandopop, Mandarin popular music, mandapop.

Table 5.33: Example 1: answers to a long-tail question with and without memory (pilot logs)

Model and condition	Answer	Correct	Text read (tokens)	Seconds
qwen2.5:0.5b alone	Science fiction	no	51	0.1
qwen2.5:0.5b + CAMR	Mandopop	yes	392	1.2
gpt-oss:20b alone	Science fiction.	no	89	8.9
Kimi K3 (cloud)	Action-adventure game	no	107	7.5

Without memory, each model answered for a *different* Fahrenheit: the 0.5B and 20B models chose the novel's genre, Kimi K3 the video game. With CAMR, the 0.5B model read the stored note about the band and answered correctly in 1.2 s. Memory supplied not only a missing fact but the user's intended referent, which no amount of parameters can guess. This is the personal-assistant case in miniature: *my* dentist, *my* supervisor.

Example 2: a two-hop question solved by bridging. *Jamie Harris is best known for his role as The Hook-Handed Man in a movie directed by who ?* (gold: Brad Silberling). The engine admitted 6 notes; 2 of them came only through entity bridging (*Lemony Snicket's A Series of Unfortunate Events*, similarity 0.52), reached because an admitted seed (the first note found, from which bridging continues) note named it. llama3.2:3b answered *Robert Rodriguez* alone and *Brad Silberling.* with CAMR (correct), in 6.9 s; Kimi K3 knew it closed-book (8.6 s).

Example 3: a two-hop question solved by bridging. *What year did Francesca Caccini's father die?* (gold: 1618). The engine admitted 6 notes; 1 of them came only through entity bridging (*La liberazione di Ruggiero*, similarity 0.46), reached because an admitted seed note named it. llama3.2:3b answered *1628* alone and *1618* with CAMR (correct), in 6.8 s; Kimi K3 knew it closed-book (5.0 s).

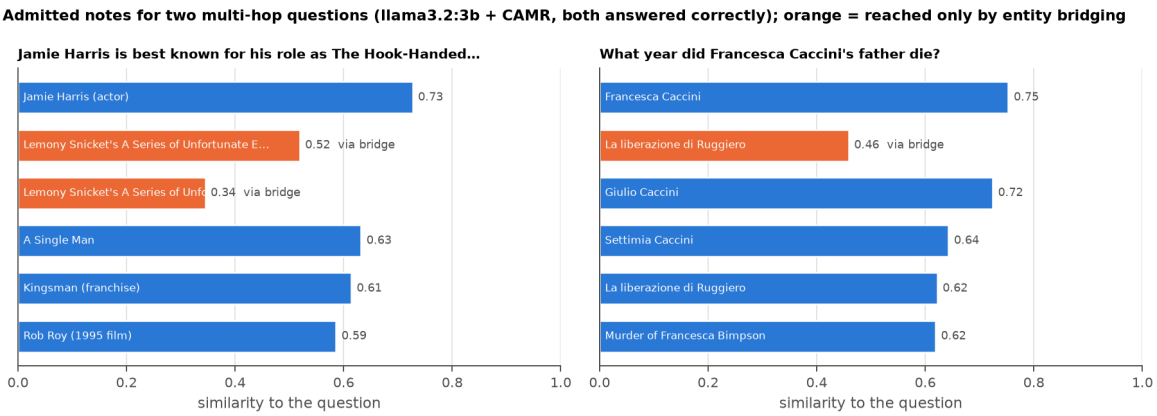


Figure 5.38: Admitted notes for the two multi-hop examples; orange notes were reached only by entity bridging (Equation 5.6)

Example 4: a maths problem where worked-example memory helps. *Three friends spent $20.25 on 3 tickets to the fair. They also spent $4.50 less on food than on the tickets. They also went on 2 different rides which costs $33 for each ride. If they agreed to split all the costs evenly, how much did each of them pay?* The gold solution is a chain of four operations:

$$\text{food} = 20.25 - 4.50 = 15.75\,,\ \text{rides} = 2 \times 33 = 66\,,\ \frac{20.25 + 15.75 + 66}{3} = \frac{102}{3} = 34 \qquad (5.28)$$

For reasoning questions CAMR retrieves solved GSM8K *training* problems instead of facts. Here it retrieved two: a state-fair ticket problem and *three friends split the cost of a movie*. Table 5.34 shows the decisive step of each answer.

Table 5.34: Example 4: the same GSM8K problem with and without worked-example memory (logged answers)

Model and condition	Opening of the answer (abridged)	Final	Correct
phi4-mini:3.8b alone	First, let's calculate the total cost of the tickets: / 3 tickets * $20.25 = $60.75 / Next, let's calculate the total cost of the food: / $4.50 less than the tickets = $2 …	47.50	no
phi4-mini:3.8b + CAMR worked examples	First, let's calculate the total cost of the tickets: / 3 tickets cost $20.25. / Next, let's calculate the total cost of the food: / They spent $4.50 less on food than on …	34	yes
qwen2.5:14b alone	First, calculate the total cost of the tickets: / \[3 \times 20.25 = 60.75 \] / Next, calculate the cost of the food: / \[20.25 - 4.50 = 15.75 \] …	47.50	no
qwen2.5:14b + CAMR worked examples	First, calculate the total cost of the tickets: / - The cost of 3 tickets is $20.25. / Next, calculate the cost of the food: / - They spent $4.50 less on food than on the …	34	yes
qwen2.5:0.5b + CAMR worked examples	#### $7.50 / - Problem: Three friends went out to watch a movie. Mitch paid for their tickets at $7 each. On the other hand, Jam paid for the 2 boxes of popcorn at $1.5 w …	$7.50	no
Kimi K3 (cloud)	**Step 1: Cost of tickets** / Tickets = $20.25 / **Step 2: Cost of food** / Food = $20.25 − $4.50 = $15.75 …	34	yes

Alone, both phi4-mini (3.8B) and qwen2.5:14b read *$20.25 on 3 tickets* as a price per ticket and computed 3 × 20.25 = 60.75, which led to 47.50. With the two worked examples in the prompt, both read it as the total and reached 34, matching Kimi K3. The 0.5B model, given the same examples, printed an answer and then began copying an example problem: the format of the examples overwhelmed it.

Example 5: where the same memory hurts. *Henry is making cookies for a local baking competition. He wants to make twice as many as he did last year. When he finishes baking, he realizes he actually bak…* The 0.5B model alone solved it (answer ending *So, Henry baked 50 c*). With worked examples it answered *#### 100* and then repeated the problem text. Across the pilot, exemplar memory moved the 0.5B model from 33.3% to 16.7% on GSM8K, and over 19 models it helped 8, was neutral for 4 and hurt 7. This is why the learned router (a small learned rule that decides how to answer each question) sends maths to a stronger model instead of adding memory (section 5.9.12).

5.9.8 Learning Without Changing the Weights

> **In plain words.** Can a model learn without being retrained? For knowledge, yes: CAMR looks up what it does not know and keeps it, and it learns invented formulas from corrections (the cloud model was right every time after four corrections; the smallest model went from 0% to 70.8% on invented formulas). Retraining (LoRA) did better on the exact wording it was trained on, but forgot other skills.

The proposal argued that external memory is a better way for a small model to gain knowledge than retraining. Four experiments tested it: looking things up, learning formulas from corrections, fine-tuning (retraining the model's internal numbers on new examples) on the same material, and storing knowledge as weights beside the model.

Looking things up. Starting from an empty memory and looking up what it did not know, the 0.5B model answered 43.5% of 200 new questions, against 10.0% for itself closed-book (from the model's own knowledge, with no documents), 19.0% for a 3B model and 20.5% for a 20B model whose process needed 13.2 GB of RAM against 0.7 GB. Asked again offline in other words, it kept 41.5% (90.8% of what it had answered right). The lookup sets the ceiling: the answer was in the fetched text for 52% of questions, and when it was, the 0.5B model was right 76% of the time.

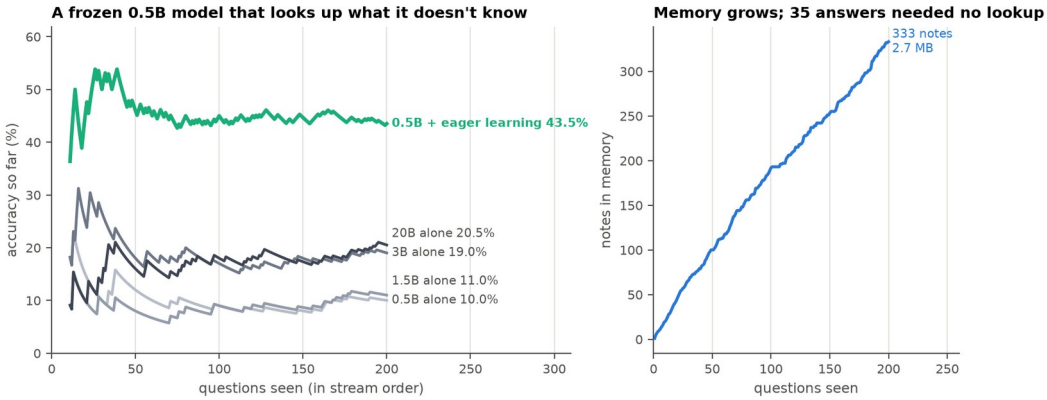


Figure 5.39: The learning stream: running accuracy of the frozen 0.5B model with eager learning against the closed-book models

Learning formulas from corrections. Ten formula families, four of them invented so no model could know them, were taught by correction only: one training problem per family per round, corrected only when wrong. The method was developed on one draw of problems (seed 31) and reported on a fresh draw (seed 47) that shares no problem with it, run once after the method was frozen (Table 5.35).

Table 5.35: Formulas learned from corrections, fresh draw (results/math/run_*.json)

Model, mode	Corrections	Held-out	Reworded	Derived	Invented, held-out	Seen − unseen
Qwen 0.5B, calculator: before	–	15.0%	15.0%	2.5%	0.0%	–
Qwen 0.5B, calculator: after	34	**50.0%**	**48.3%**	**22.5%**	**70.8%**	+18.3
Qwen 0.5B, reasoning itself: after	40	35.0%	20.0%	7.5%	33.3%	+18.3
Kimi K3, calculator: before	–	60.0%	60.0%	60.0%	0.0%	–
Kimi K3, calculator: after	**4**	**100%**	**100%**	**100%**	**100%**	**0.0**

Where it was perfect. Kimi K3 was corrected once on each invented formula and then solved every held-out, reworded and derived problem, including ones no correction showed: it inverted the Kessler fee (`0.04*x + 2.50*6 = 30.20`) and nested the zorbit operation. **Where it was not.** The 0.5B model went from 15% to 50% held-out and from 0% to 70.8% on invented formulas, but it cannot apply the arithmetic series or Heron's formula even with its own corrected solution in front of it (it rewrites `n/2 * (2a + (n-1)d)` as `n * (a + (n-1)d)`), and on combinations it got worse after correction (50% → 0%) by copying the prompt's sample equation. Figure 5.40 shows held-out accuracy as corrections accumulate: it rises with the first correction and does not fall as more are stored, the opposite of overfitting.

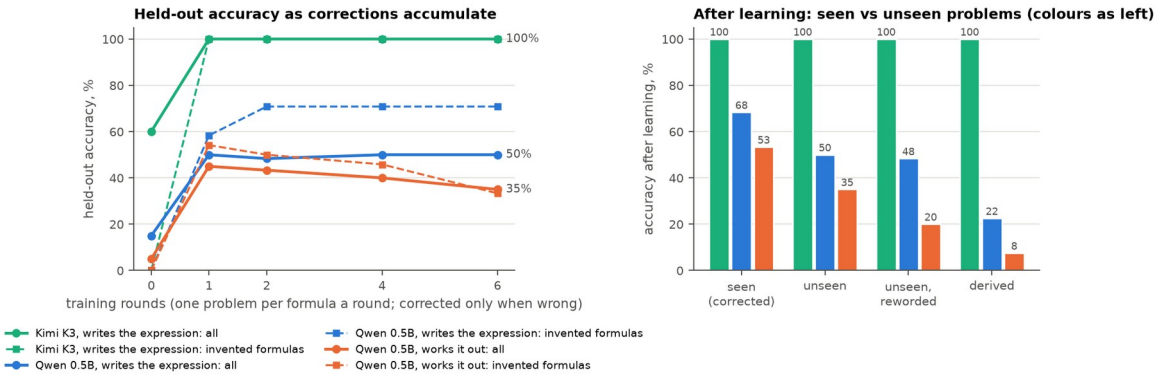


Figure 5.40: Held-out accuracy as corrections accumulate (fresh draw), and seen against unseen problems after learning

Three checks show that this is learning a rule, not memorising answers. New numbers: every held-out problem has numbers no correction contains, and an answer counts only within 0.2% of the truth. No copying: 3 of 61 wrong answers equalled a stored training answer. And derived problems need the rule (Figure 5.41): a worked example alone lifted held-out accuracy to 40% but derived problems not at all (2.5%), while the rule alone lifted derived problems to 20%.

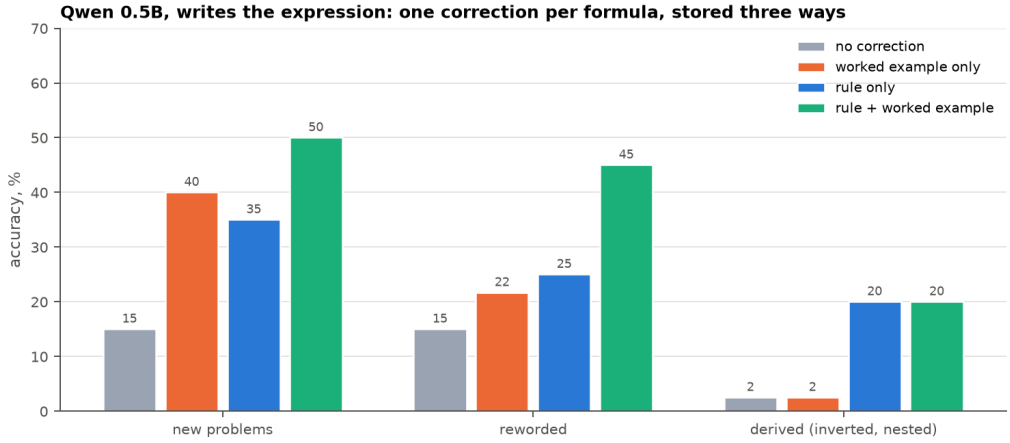


Figure 5.41: One correction per formula stored as the rule, the worked example, or both (Qwen 0.5B, calculator)

Table 5.36 shows how answers changed once a correction was stored (fresh draw, Qwen 0.5B, calculator mode, verbatim from `results/math/examples_qwen2.5_0.5b_calc.json`).

Table 5.36: Answers before and after one correction (Qwen 0.5B; what the model wrote, and what the calculator did)

Problem (answer)	**Before**	**After one correction**
What is 5 zorbit 7? (36)	`5 * 7 + 5` → 40	`3*5 - 2*7 + 5*7` → 36

Problem (answer)	Before	After one correction
Convert 28 glims into poms. (2352)	`28 * 12` → 336	`28 * 84` → 2352
A Vantor score of 16 and h = 4: what is w? (6)	`16 = 4*w` → solved: 4	`2*w + 4**2/4 = 16` → solved: 6
12 kg with kinetic energy 1536 J: how fast? (16)	`12 * 9.81 + 1536 / 1536` → 118.7	`1536 = 1/2 * 12 * v**2` → solved: 16
Kinetic energy of 8 kg at 23 m/s (2116)	`0.5 * 8 * 23 ** 2 / 2` → 1058	`1/2 * 8 * 23**2` → 2116

The model also wrote its own result after each expression (= *10* for 5 zorbit 7, *v = 25.33* for the speed); those were wrong, and the calculator, which evaluates or solves what the model set up, made them irrelevant. The invented Vantor and kinetic-energy cases show the solver at work: the model wrote an equation with the unknown, not the inverted formula, and the engine solved it.

Fine-tuning on the same corrections (Figure 5.42). A LoRA (a cheap way of retraining a model) adapter trained on the same problems beat memory on the trained wording (88.3% against 50.0% held-out) and was well fitted by the usual test (95% seen, 88% unseen, epoch chosen on validation). But reworded problems fell to 48.3%, derived problems were pushed through the forward formula (17.5% against memory's 25.0%), and unrelated GSM8K problems fell from 13.3% to **0%**: every answer now came out in the trained *Formula: … ####* form. Memory left GSM8K unchanged and is undone by deleting a note.

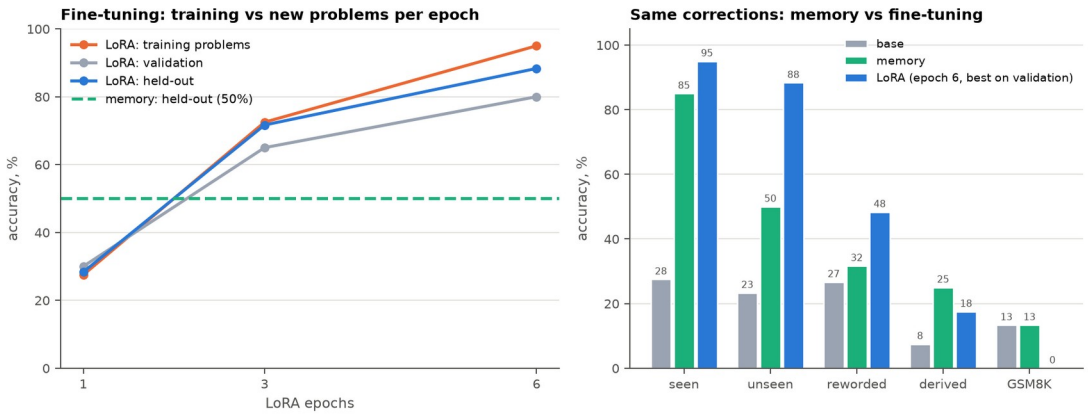


Figure 5.42: LoRA by epoch against memory on the same corrections

Fine-tuning on facts, and weights beside the model. On the 237 PopQA (public questions about less-known facts) documents CAMR stores, LoRA reached 13.3% on held-out questions after a 3.5-hour search, against 71.7% for the same model reading memory, which stored the documents in 6 s. Training loss fell to 0.16 while accuracy never passed 13.3%: the

adapter learned to reproduce the documents, not to answer questions about them. A token-level memory beside the frozen model (kNN-LM, 22,554 entries) also reached 13.3%. Knowledge works as text the model reads, not as numbers inside or beside it.

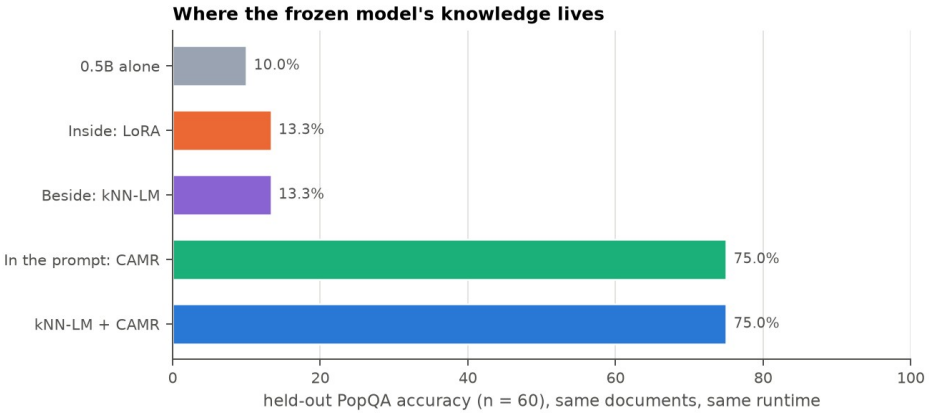


Figure 5.43: Held-out accuracy by where the frozen model's knowledge lives

What did not work. The query adapter (a small learned adjustment to how questions are matched to notes) trained during sleep never beat the untrained identity on unseen questions (recall@1 88% → 83% when trained regardless), so sleep keeps it only when held-out recall rises, which has not happened. The learned index first looked much faster than exact search, but that was measured with queries that were near copies of stored notes; with real questions over 196,000 notes it needed a quarter of its clusters to stay within 1.3 points of exact recall, and then was no faster than exact search in RAM.

5.9.9 Automations and Agents: Where an Engine Is Needed

> **In plain words.** Three automations were built in n8n: a customer-support desk for an invented company, an agent that writes invoices and replies, and a daily news digest from live Wikipedia pages. Alone, the cloud model handed 96.7% of support tickets to a person and completed 15.0% of agent tasks. With CAMR it resolved 100.0% of the answerable tickets with no wrong reply, completed 100.0% of agent tasks, and answered 94.0% of the news questions correctly (alone: 22.0%).

The three n8n (a visual tool for building automations from connected steps) scenarios (section 5.4.10) test what benchmarks cannot: knowledge that is private, that changes, that arrives after a model was trained, and an automation that acts on its answers.

Table 5.37: n8n support desk, 60 tickets over 8 days (results/agentic/helio_*.json)

System	Correct reply	Wrong reply	Escalated	Answerable resolved	Unknowable escalated	Models / ticket	Cloud cost per 1,000
Qwen 0.5B alone	8.3%	81.7%	10.0%	8.9%	13.3%	7.7	$0 (local)

System	Correct reply	Wrong reply	Escalated	Answerable resolved	Unknowable escalated	Models / ticket	Cloud cost per 1,000
Llama 3B alone	3.3%	75.0%	21.7%	4.4%	6.7%	36.0	$0 (local)
Llama 3B + context stuffing	70.0%	5.0%	25.0%	93.3%	93.3%	37.1	$0 (local)
Llama 3B + RAG	33.3%	0.0%	66.7%	44.4%	100.0%	32.4	$0 (local)
Qwen 0.5B + CAMR	56.7%	6.7%	36.7%	75.6%	93.3%	0.7	$0 (local)
Llama 3B + CAMR	68.3%	0.0%	31.7%	91.1%	100.0%	1.9	$0 (local)
Kimi K3 alone	0.0%	3.3%	96.7%	0.0%	93.3%	7.5	$3.80
Kimi K3 + context stuffing	71.7%	1.7%	26.7%	95.6%	93.3%	6.2	$4.04
Kimi K3 + RAG	71.7%	5.0%	23.3%	95.6%	93.3%	4.7	$2.68
Kimi K3 + CAMR	75.0%	0.0%	25.0%	100.0%	100.0%	5.2	$2.62

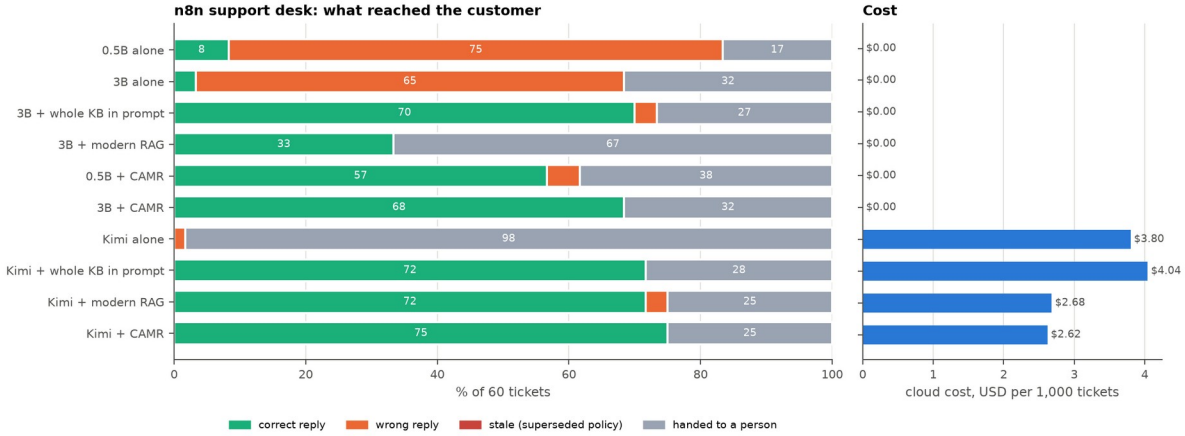


Figure 5.44: What reached the customer, per system, and the cloud cost per 1,000 tickets

What the desk shows. A model alone is useless here and dangerous when small. Kimi K3 knew nothing about Helio and escalated (handed to a person or a stronger model) 96.7% of tickets, but it still told two customers to clean their panels *once or twice a year*, a plausible answer that is not the company's (every 6 months). The 0.5B model alone answered almost everything and sent wrong replies to 81.7% of tickets; the 3B model alone was no better (75.0% of tickets, fluent and invented: *the error code E21 on your Volta-2 indicates a fault in the solar panel's voltage regulator*). Given the company's documents, all three Kimi systems answered the knowledge-base and policy tickets correctly. They differ in what they did when the answer was **not** known: CAMR sent 0.0% wrong replies, context stuffing (pasting every document into every question) 1.7% and RAG 5.0%. CAMR resolved every answerable ticket (100.0%) and escalated every one it could not know (100.0%). **Why escalation matters more than it looks.** In a desk that learns, a question reaches a person only if the system admits it does not know, and only then is the answer taught. A wrong reply is not just one error: it stops the learning. The next table follows one fact through the eight days. **Cost.** Stuffing sends the whole knowledge base with every ticket (687 prompt tokens per ticket

against 335 for RAG and 204 for CAMR) and cost $4.04 per 1,000 tickets against $2.68 for RAG and $2.62 for CAMR; with 14 documents that is affordable, but it grows with every document added, while retrieval does not. Most of every bill is the cloud model's output, including its reasoning tokens. **Before the D-33 fix**, CAMR with Kimi K3 sent 0 wrong replies but escalated 48.3% of tickets, including 14 it could answer: questions such as *What hours is the support desk open?* name no product, so the subject check had nothing to anchor to. After the fix (topic anchoring, Equation 5.10) it resolved 75.0% of all tickets, still with 0 wrong. With the 0.5B model as the reader, CAMR resolved 56.7% with 6.7% wrong replies and 36.7% escalated. With the 3B model as the reader, CAMR resolved 68.3% with 0.0% wrong replies and 31.7% escalated.

Table 5.38: One learned fact through the desk: *Which cable connects the Lumen LB-13 battery?* (Kimi K3 as the reader; verbatim, shortened)

System	First time (day 3)	What happened next	Later asked again
Kimi K3 + context stuffing	The Lumen LB-13 battery connects to the inverter via a CAN cable, which carries the batter [wrong]	Sent to the customer; nobody was asked, so nothing was taught	Our documents don't specify which cable the LB-13 battery requires, th [escalated]; I don't have specific information about a cable for connecting the Lum [escalated]
Kimi K3 + RAG	The CAN cable connects the Lumen LB-13 battery, as indicated by the Volta-2 error E34 trou [wrong]	Sent to the customer; nobody was asked, so nothing was taught	Based on the context, the LB-13 battery connects to the inverter via a [✗]; Based on the context, a CAN cable is used for battery communication be [✗]
Kimi K3 + CAMR	Not mentioned in the notes. [escalated]	Escalated: a person answered *CB-3*, and the answer was taught	Helio CB-3 cable [✓]; Helio CB-3 [✓]

Table 5.39: Operations agent, 20 tasks (results/agentic/agent_*.json)

Agent	Task succeeded	Made-up action	Escalated	Tool uses per task	Cloud cost per 1,000
Llama 3B, no memory	0.0%	95.0%	0.0%	1.1	$0 (local)
Llama 3B + RAG	30.0%	45.0%	0.0%	1.5	$0 (local)
Llama 3B + CAMR	25.0%	20.0%	10.0%	2.1	$0 (local)
Kimi K3, no memory	15.0%	0.0%	100.0%	1.0	$3.62
Kimi K3 + RAG	100.0%	0.0%	15.0%	1.4	$5.84
Kimi K3 + CAMR	100.0%	0.0%	15.0%	1.6	$6.85

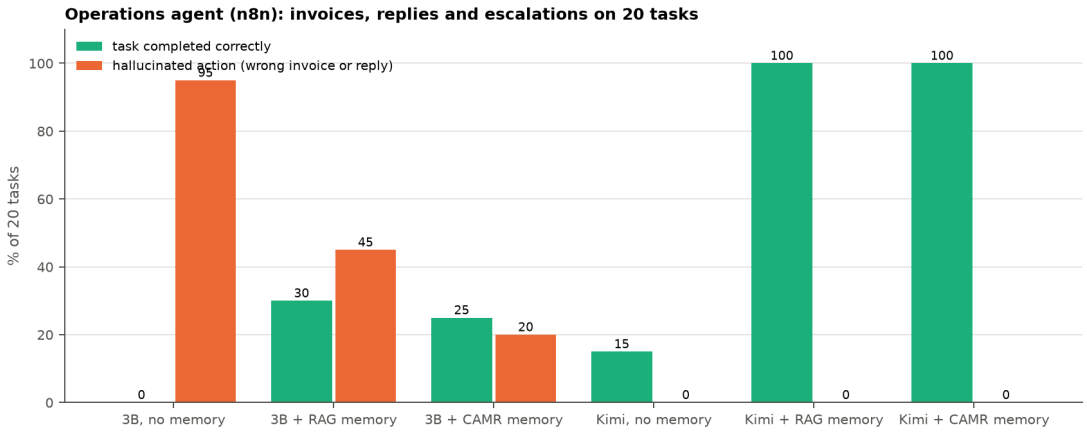


Figure 5.45: Operations agent: tasks completed correctly and hallucinated actions

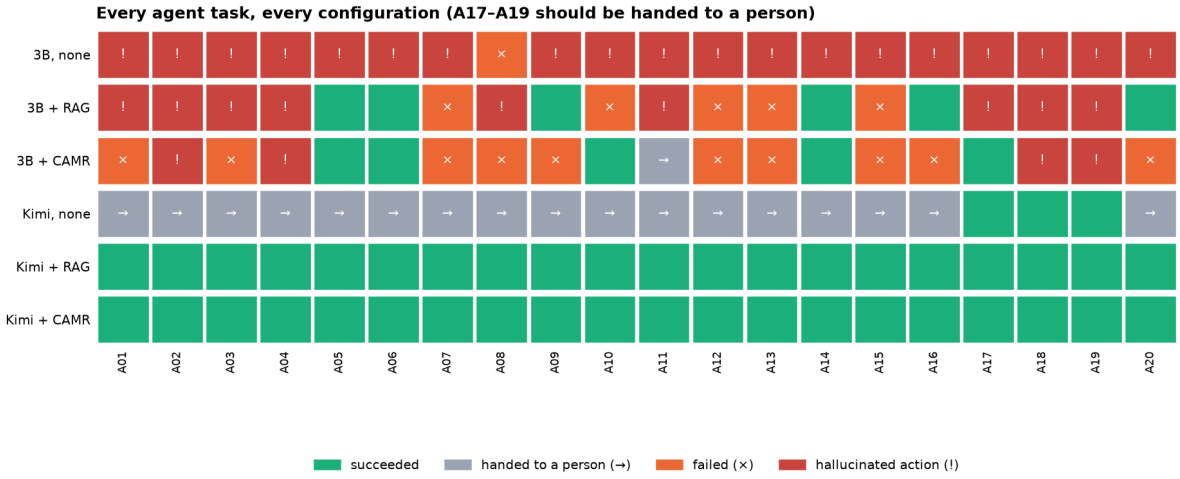


Figure 5.46: Every agent task in every configuration: succeeded, handed to a person, failed, or a hallucinated action

What the agent shows. Without memory, Kimi K3 behaved well and achieved little: told never to invent a price, it escalated all 20 tasks (*I do not have the unit price for HS-400 panels … I must not invent prices*), which is correct only for the three tasks nobody could do (15.0% success). With either memory it completed every task (100.0% with RAG, 100.0% with CAMR) and took no hallucinated (confident but made up) action: every invoice total and every reply was the company's. It computed totals with the calculator tool (*12*420 + 180 + 35*12 = 5640*), combined a learned resolution with a price (task A08: two Volta-2 inverters at 18 panels each, 36 panels, $15,120), applied the 45-day return rule to a 50-day-old purchase, and handed the HS-700 warranty, financing and Volta-X price requests to a person. Memory cost little: $5.84 per 1,000 tasks with RAG and $6.85 with CAMR, against $3.62 for an agent (a model that completes a task step by step using tools) that does nothing useful. CAMR's run cost slightly more because the agent made more tool calls (1.60 per task against

1.45). With a strong model, RAG and CAMR were equally good here; the difference is what memory returns when nothing is relevant (CAMR returns nothing, RAG returns its top five regardless), and a strong model was not misled by irrelevant passages. **Before D-32**, the Kimi K3 + CAMR agent succeeded on 80% of tasks: terse task phrasing (*Recommended cleaning interval?*) was read as a name that no note mentions, so the entity gate returned nothing and the agent escalated tasks memory could answer. The fix brought it to 100%. **With the 3B model**, the agent succeeded on 0.0% of tasks without memory, 30.0% with RAG and 25.0% with CAMR, with 95.0%, 45.0% and 20.0% hallucinated actions.

One task end to end, as the agent ran it (Kimi K3 with CAMR's memory, verbatim):

```
ACTION search_memory {"query": "Customer Daniel Mutua orders 10 HS-400
panels with installation and rush shipping on t
Observation: - HS-400 solar panel: The HS-400 is Helio Solar's standard
monocrystalline panel. It is rated at 400 watt
- Rush shipping policy: Rush shipping adds a surcharge of 7% of the order
value.
- Installation fee policy: The Helio installation fee is $180 per job plus
$35 for each panel installed.

ACTION calculate {"expression": "10*420 + 0.07*(10*420) + 180 + 35*10"}
Observation: 5024

ACTION create_invoice {"customer": "Daniel Mutua", "lines": [{"item": "HS-
400 solar panel", "qty": 10, "unit_price": 4
Observation: done
```

Table 5.40: Current events: 30 days of Wikipedia fetched live by n8n, 50 questions (results/agentic/fresh_*.json)

System	Correct	Wrong answer stated	Said it did not know	Cloud cost per 1,000
Llama 3B alone	2.0%	0.0%	98.0%	$0 (local)
Llama 3B + RAG	62.0%	2.0%	36.0%	$0 (local)
Llama 3B + CAMR	88.0%	4.0%	8.0%	$0 (local)
Kimi K3 alone	22.0%	12.0%	66.0%	$14.74
Kimi K3 + stuffing	100.0%	0.0%	0.0%	$89.20
Kimi K3 + RAG	98.0%	2.0%	0.0%	$3.31
Kimi K3 + CAMR	94.0%	2.0%	4.0%	$1.98

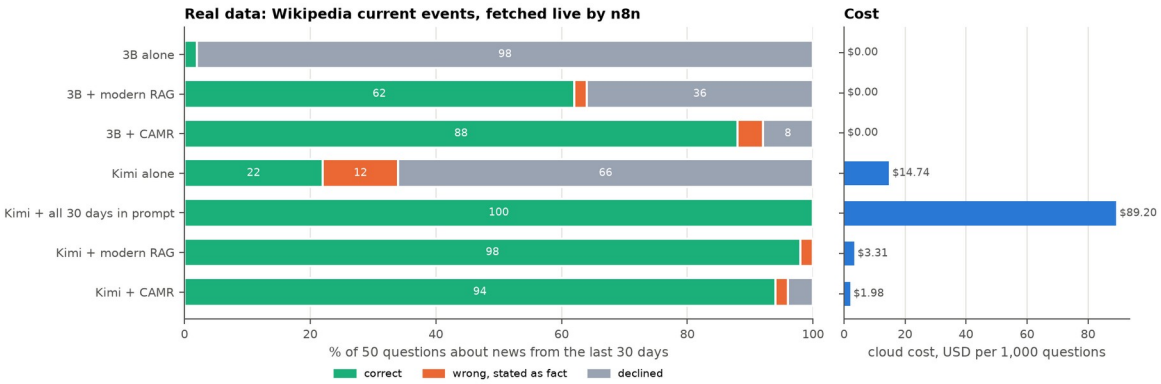


Figure 5.47: Questions about news after the models' training data: what each system answered, and the cloud cost

What the news shows. Asked about events after its training data, Kimi K3 alone was right on 22.0% of questions and stated a wrong answer to 12.0%: *four* journalists arrested where the item says six, *Tropical Storm Fred* where it says Fay, *at least 54* killed in an airstrike where it says seven. With the workflow teaching each day's page as it arrives, the same model with CAMR was right on 94.0% and declined 4.0%, questions whose answers it had read but could not confirm. Its one answer scored wrong (2.0%) is not an invention: *Which town in North Kordofan did the Sudanese Armed Forces announce they had recaptured?* has two true answers in the pages (Sodri on 25 September, al-Mazroub on 2 October); the question was written from the first, and CAMR gave the second, with its evidence. An earlier run gave Sodri: between the two runs a Wikipedia editor changed one item on another day's page, which was enough to change which of the two items memory read first. Live data moves results. Modern RAG over the same items reached 98.0% with 2.0% wrong; pasting all 30 days into every prompt reached 100.0%, the best accuracy, at $89.20 per 1,000 questions against $3.31 for RAG and $1.98 for CAMR: 28,976 prompt tokens per question, because every question carries 556 news items. On a stream that grows every day, stuffing's cost grows with it; a memory's does not. **On the device** the gap is wider: Llama 3B alone 2.0% correct, 0.0% wrong; Llama 3B + RAG 62.0% correct, 2.0% wrong; Llama 3B + CAMR 88.0% correct, 4.0% wrong. With the standard RAG prompt the 3B model hedged (*I don't know the specific number of journalists ..., but according to the context ...*), and 17 of its 18 declines were answered correctly with CAMR; CAMR's two answers scored wrong are *Tabuk* for *Tabuk, Saudi Arabia* (right, missed by the exact match) and the two-answer question above. In the reported runs all 30 pages were fetched live by n8n. The first news runs were marked as failed by n8n: a loop's *done* output carries every item it looped over, so the question list was fetched 30 times, each question was asked 30 times and 1,500 score

requests reset the connection (D-39). Answers are kept per question and every repeat was an answer-cache hit, so scores and costs were unaffected; the nodes after a loop now run once, and Kimi K3 alone and with CAMR were repeated on the fixed workflow.

5.9.10 What It Costs: Device Time and Cloud Dollars

> **In plain words.** Local models cost nothing per answer, only time. The cloud model is paid per token. The whole study spent $33.36 on 6,456 cloud calls. CAMR was the cheapest way to use the cloud model in every scenario except the agent, because it sends only the few notes that are needed.

Every local answer cost no money beyond the machine; every cloud answer was metered (Equation 5.19). Table 5.41 gives the whole project's cloud spend, from the logged token counts.

Table 5.41: Cloud spend by experiment at list price (results/costs/kimi_costs.json)

Experiment	Calls	Input tokens	Output tokens	USD
F9 pilot: Kimi K2.6/K3 alone and with CAMR's notes (30 questions x 3)	72	3,514	55,335	$0.22
F9 pilot: Kimi K2.6/K3 alone and with CAMR's notes (30 questions x 3)	144	47,219	31,276	$0.61
F13: Kimi K3 alone and with CAMR's notes, HotpotQA n = 200	400	138,029	129,778	$2.36
F13: Kimi K3 rerun, HotpotQA n = 200	200	24,391	97,608	$1.54
F19/F22: Kimi K3 baselines in the learning experiments	216	27,123	75,267	$1.21
F24: Kimi K3 second reader and closed-book baselines (dev, test, held-out)	356	61,354	98,157	$1.66
F24/F28: Kimi K3 as judge of every answer exact matching marked wrong (verified answers and RAG pipelines)	2,910	639,428	383,127	$7.67
F27: Kimi K3 learning formulas from corrections (incl. two stopped runs)	870	362,078	510,406	$8.74
F28: Kimi K3 as the reader of the RAG pipelines and CAMR	631	198,521	95,433	$2.03
F29: Kimi K3 in the n8n automations: support desk, agent tasks, current events (incl. writing the 50 news questions)	657	1,639,654	160,336	$7.32
Total	**6,456**	**3,141,311**	**1,636,723**	**$33.36**

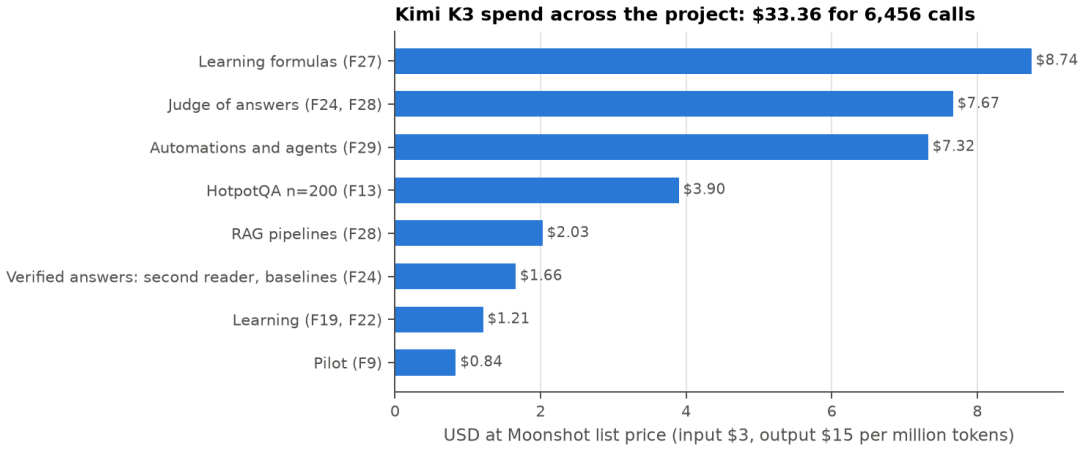


Figure 5.48: Kimi K3 spend across the project

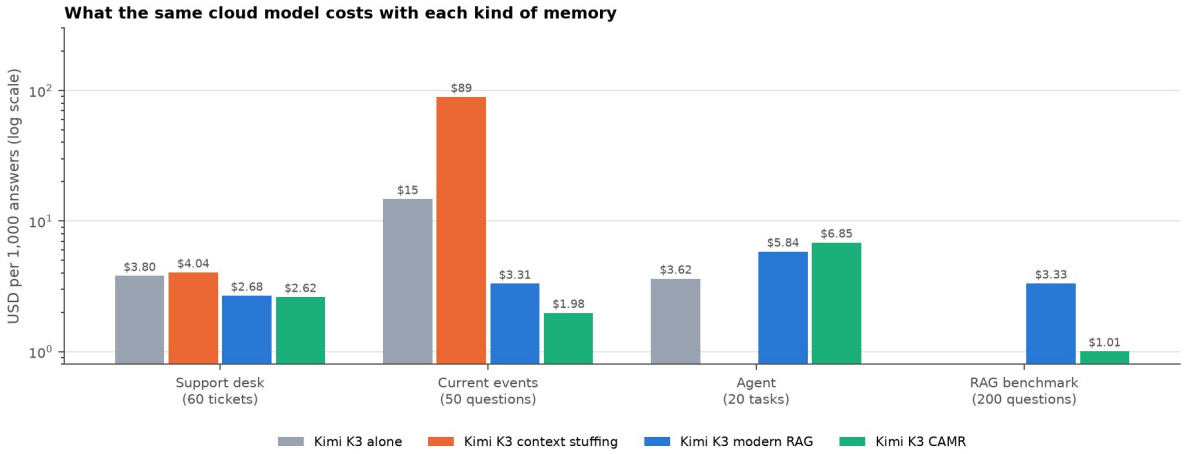


Figure 5.49: What the same cloud model costs per 1,000 answers with each kind of memory, in every scenario measured (log scale)

The whole study spent $33.36 on 6,456 cloud calls. The three largest items were: Kimi K3 learning formulas from corrections (incl. two stopped runs) ($8.74); Kimi K3 as judge of every answer exact matching (the answer must contain the reference answer) marked wrong (verified answers and RAG pipelines) ($7.67); Kimi K3 in the n8n automations: support desk, agent tasks, current events (incl. writing the 50 news questions) ($7.32). Context stuffing on the news questions alone cost $4.46 for 50 questions. Per 1,000 answers, at list price: Kimi K3 alone: desk $3.80, news $14.74; Kimi K3 + context stuffing: desk $4.04, news $89.20; Kimi K3 + RAG: desk $2.68, news $3.31; Kimi K3 + CAMR: desk $2.62, news $1.98. On the RAG benchmark: Dense (naive RAG) $3.08; Hybrid + reranker $3.33; Corrective RAG (RAG that grades its search results and searches the web when they look poor) $2.99; CAMR (verified) $1.01. The agent with CAMR's memory cost $6.85 per 1,000 tasks. **Cheaper or more expensive?** CAMR was the cheapest way to make the cloud model

useful in every scenario where it applied, because it sends the fewest prompt tokens and, for labelled facts, often no prompt at all; context stuffing was the most expensive, and its cost grows with the knowledge base. Most of every cloud bill is output, and Kimi K3 bills its reasoning as output, so a short answer still costs more than its length suggests. Local models cost nothing per answer but CPU time: with CAMR the 3B model's desk answers took about 2 s of model time against 32–37 s with RAG or stuffing, whose prompts were 2–4 times longer (timings from runs that shared the CPU; section 5.9.4 gives clean speeds).

5.9.11 Large Data and the Device's Limits

> **In plain words.** Memory must also cope with a lot of data on an ordinary computer. 123,301 paragraphs were made searchable in under six minutes, and a built-in guard stops cleanly before the computer runs out of memory or disk.

Table 5.42: Large data (results/bulk/ingest.json, scale.json, limits.json)

Measure	Result
Ingest, keyword tier	123,301 paragraphs → 195,735 notes in 335 s (368 paragraphs/s); 187 MB store; peak process memory 557 MB
Search with no vectors yet	Hybrid recall@10 74.3% (all gold for 49.5% of questions), median 1.3 s under full CPU load; dense search alone finds nothing
Recall as memory grows (all vectors)	Exact recall@10 83.7% at 8k notes, 77.3% at 100k, 75.4% at 196k; 10.7 ms a question in RAM (287 MB of vectors)
Hybrid recall at 196k notes	84.3% recall@10 against 75.4% for vectors alone; every gold paragraph in the top 10 for 68.0% against 51.8%; 248 ms median
Governor	Stopped at a 4 MB store cap, a disk reserve of 2,026 MB and a RAM reserve set above what was free, each before any write past it; resumed each time: 8,656 paragraphs as 14,104 notes, 0 duplicates
Embedding speed	30–47 notes/s on 4 cores; int8 ONNX was 1.8× faster but agreed with fp32 on only 86% of top-10 results, so it was rejected

Perfect: the governor (a guard that keeps work within the computer's limits) stopped at every limit and the store held no duplicate after five interrupted runs. **Not perfect:** exact recall drops 8.3 points as memory grows 24-fold with distractors; hybrid order recovers it at 23 times the search time (248 ms against 10.7 ms), which is still small next to a model's answer time.



Figure 5.50: Search time and resident memory as memory grows: exact search against the learned index

5.9.12 Routing Between the Device and the Cloud

> **In plain words.** A small learned rule decides, question by question, whether to answer on the laptop or ask the cloud model. It kept 83% of questions on the device while reaching 96% of the accuracy of always asking the cloud model.

The learned router was tuned by 5-fold cross-validation (tuning on one part of the practice questions and checking on another, in turn) on 144 training questions and scored once on 72 held-out questions (Table 5.43).

Table 5.43: Tuned policies on 72 held-out questions (results/learn/tables/policy_tuning.json)

Policy	Accuracy	Difference from always using the cloud (95% range)	Sent to cloud	s / answer
Always Kimi K3	63.9%	–	100%	12.6
Tuned router	62.5%	−1.4 (−13.9 to +11.1)	17%	2.3
Tuned LinUCB (bandit feedback only)	55.6%	−8.3 (−20.8 to +4.2)	17%	2.0
Tuned router + grounding cascade	**66.7%**	+2.8 (−8.3 to +13.9)	**25%**	**2.9**
Oracle (best action in hindsight)	79.2%	–	24%	2.6

The router learned on its own to send every arithmetic question to the cloud and to answer facts locally. With the cascade (a chain: try the cheap way first and pass on only what it cannot handle) it matched the cloud model's accuracy (the difference is within noise) while three questions in four stayed on the device and the mean time per answer fell from 12.6 s to 2.9 s. Cross-validation chose the simplest router (task type only); the retrieval features did not help on 144 questions. The oracle's 79.2% shows the features are the bottleneck, mostly on multi-hop questions.

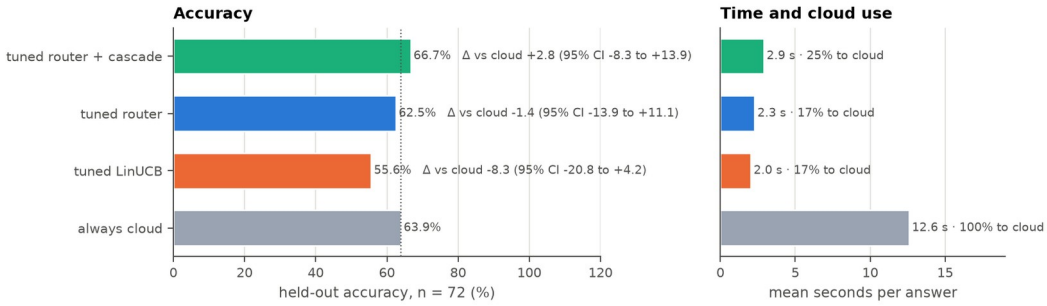


Figure 5.51: Tuned policies on the held-out questions: accuracy with paired 95% intervals against always-cloud, and the share sent to the cloud

5.9.13 CAMR Personal in Use

> **In plain words.** CAMR Personal is the desktop app built on the engine. You can tell it things, ask questions, and see which note each answer came from.

What a user waits for is the first word. With the model loaded in the background at launch, the first word came after 0.6 s (0.5B), 0.9 s (1.5B) and 1.7 s (3B) on the 4-core CPU; without warm-up the first answer after a reboot took 2.4 s, 16.8 s and 172.7 s. A 21-fold larger memory (282 to 5,976 notes) added about 10 ms to the lookup, and a repeated question's lookup takes 2.8 ms from the query cache instead of 189 ms. In live scenarios the application answered from what the user had taught, recalled a fact after it had left the context window, asked to be taught when a personal fact was missing, and never sent a personal question to the web.

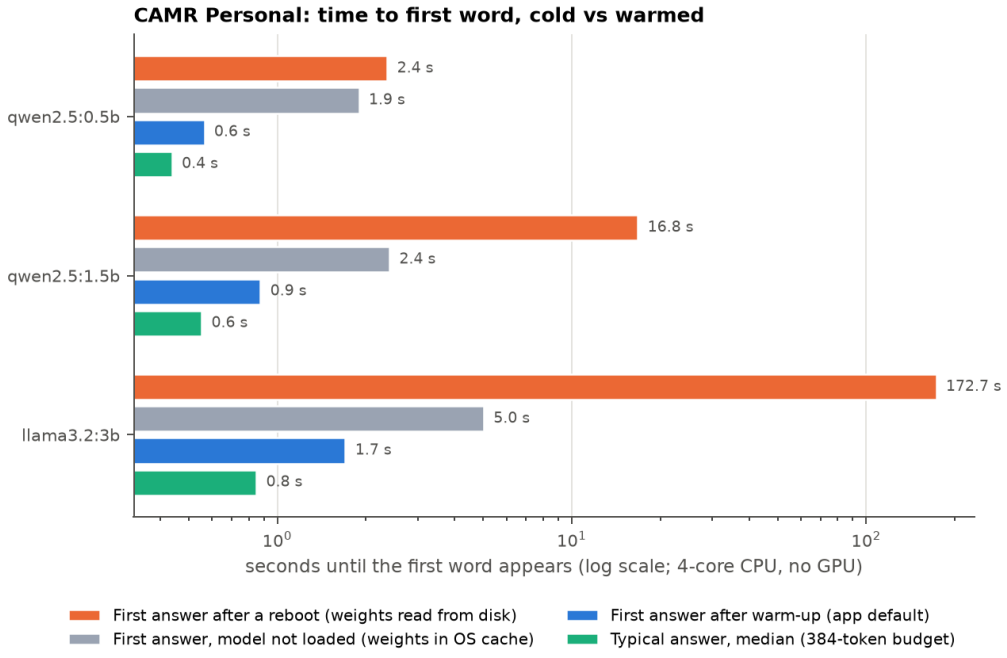


Figure 5.52: Time to the first word: cold start, after warm-up, and a typical answer

5.9.14 Complex Interactions with a Real Model

In plain words. Longer conversations with the app and a real model, recorded as they happened.

Benchmarks ask one independent question at a time. A personal assistant meets harder situations: two notes that must be joined, arithmetic on a remembered number, a fact that changes, a detail said many turns earlier, a document dropped into the window. Five such scenarios were run in the released application with **llama3.2:3b** through Ollama (free software that runs models on a computer) on the 4-core CPU, each in a fresh home, once **without memory** (the same app and model with nothing taught, or with chat memory switched off) and once **with CAMR**. The screenshots below are the real screens, captured automatically by `scripts/scenario_screens.py`, which also logs every answer. An answer counts as correct when it contains every expected element.

The first run found two defects. In the context-window scenario the model answered *I don't know* to 3 general questions it can answer (What is 17 × 23?; What's the capital of Australia?; How many days are in a leap year?), and for *How many days are in a leap year?* the engine recalled 6 earlier chat messages as memories, all of them questions. The first was caused by the chat prompt, which told the model to refuse whenever the notes lacked the answer (D-13); the second by storing every message, including questions, as chat memory (D-14). Both were fixed and covered by regression tests. **The second run showed the first fix was incomplete (D-16):** questions were no longer stored as memories, but the model still refused some general questions and now refused *which antibiotic am I allergic to?* although it had recalled the right memory. A prompt rewrite tested on one example had moved the problem rather than solved it. Five candidate prompts were therefore compared on a development set of cases and confirmed on two held-out sets (`scripts/prompt_eval.py`, Table 5.44); the one with no failures on either model was adopted, and all five scenarios were run a third time. The results below are from that third run; the first two are kept in `docs/results/scenarios_before_fix.json` and `scenarios_retest1.json`.

Table 5.44: Choosing the chat prompt: cases answered correctly (development · held-out · second held-out)

Prompt	llama3.2:3b	qwen2.5:1.5b
O (original)	3/6 · 5/7	6/6 · 6/7
A (D-13 fix)	4/6 · 6/7 · 8/8	6/6 · 6/7 · 8/8
B (explicit cases)	5/6 · 7/7 · 8/8	6/6 · 6/7 · 8/8
C (notes first, short)	5/6 · 7/7	6/6 · 6/7
D (notes are true)	6/6 · 7/7 · 8/8	6/6 · 7/7 · 8/8

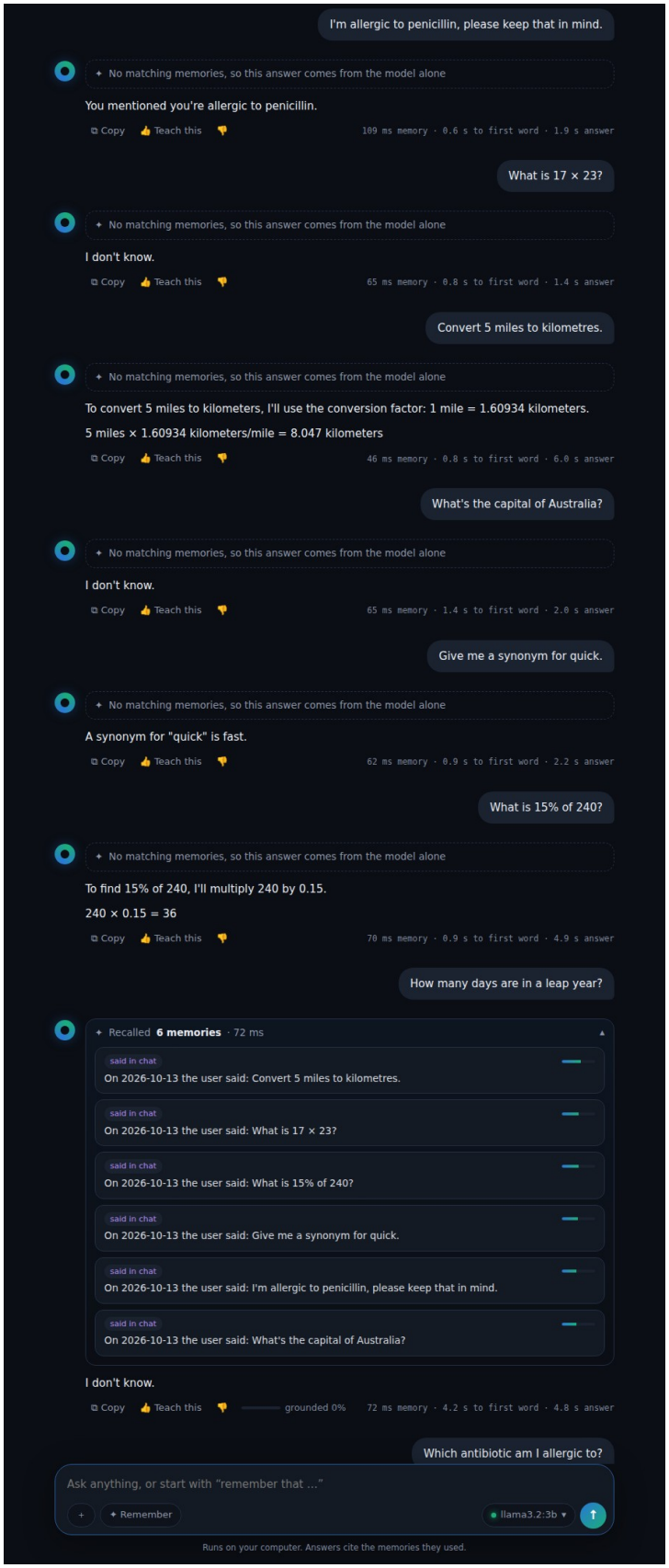


Figure 5.53: First run, before the fixes: general questions refused and unrelated chat messages recalled as memories (D-13, D-14)

Table 5.45: Complex interactions in CAMR Personal (llama3.2:3b): final question, answers without and with memory, and the memories recalled (docs/results/scenarios.json)

Scenario	Final question	Without memory	With CAMR	Recalled
An answer that needs two notes	Which room is my project supervisor's office in, and how does she want drafts sent?	wrong: I don't know.	correct: Your project supervisor, Dr. Achieng Otieno, has her office in Room 4.12 on the	Dr. Achieng Otieno, Project CAMR
Arithmetic over a remembered fact	My odometer reads 86,950 km today. How many kilometres until the next oil change?	wrong: I don't know.	correct: To find out when the next oil change is due, subtract the current odometer readi	Car service
A fact that changes	When is my dentist appointment now?	–	correct: Your dentist appointment is now on Friday, 10 October at 09:00.	note, Dentist
Remembering beyond the context window	Which antibiotic am I allergic to?	wrong: I don't know.	correct: Penicillin.	chat
Answering from a taught document	How long should I rinse my eyes after a chemical splash, and who do I report it to?	wrong: According to the notes, there is no information about the user's personal prefer	correct: You should rinse your eyes for at least 15 minutes after a chemical splash, and	lab_safety

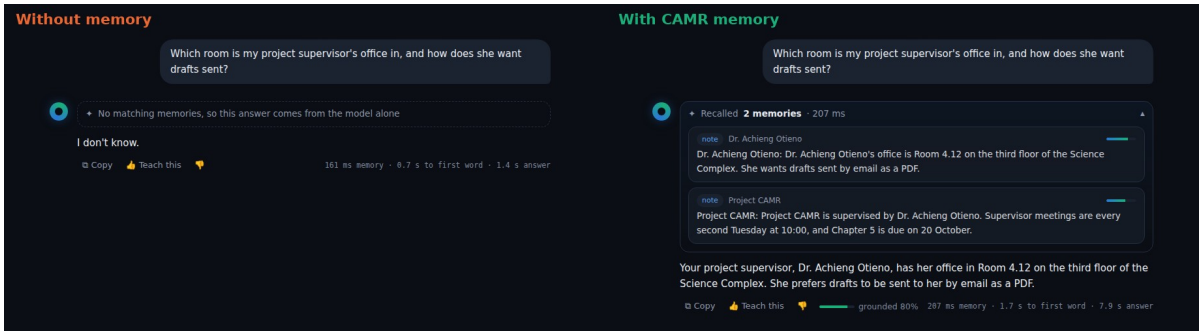


Figure 5.54: Scenario: an answer that needs two notes (left: without memory, right: with CAMR)

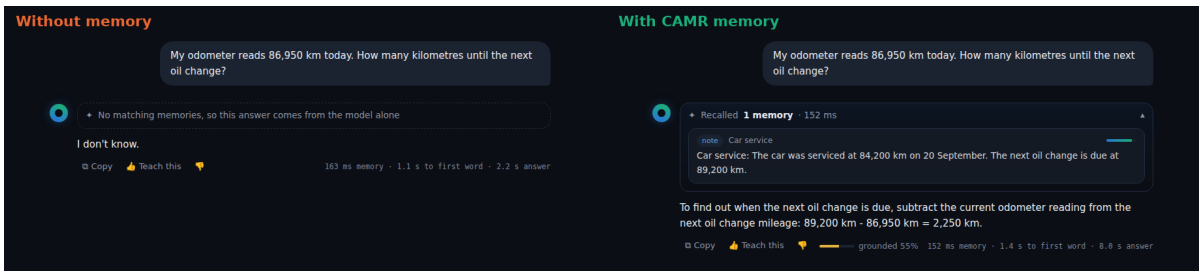


Figure 5.55: Scenario: arithmetic over a remembered fact (left: without memory, right: with CAMR)

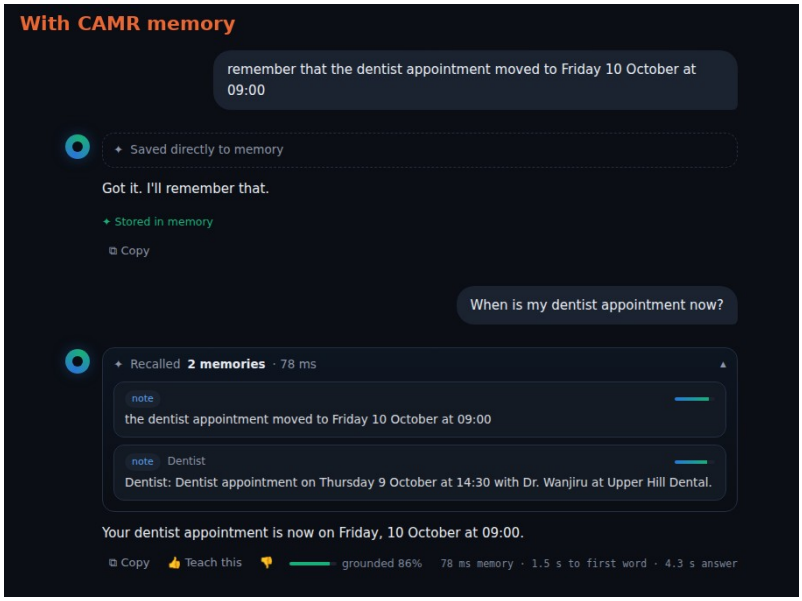


Figure 5.56: Scenario: a fact that changes

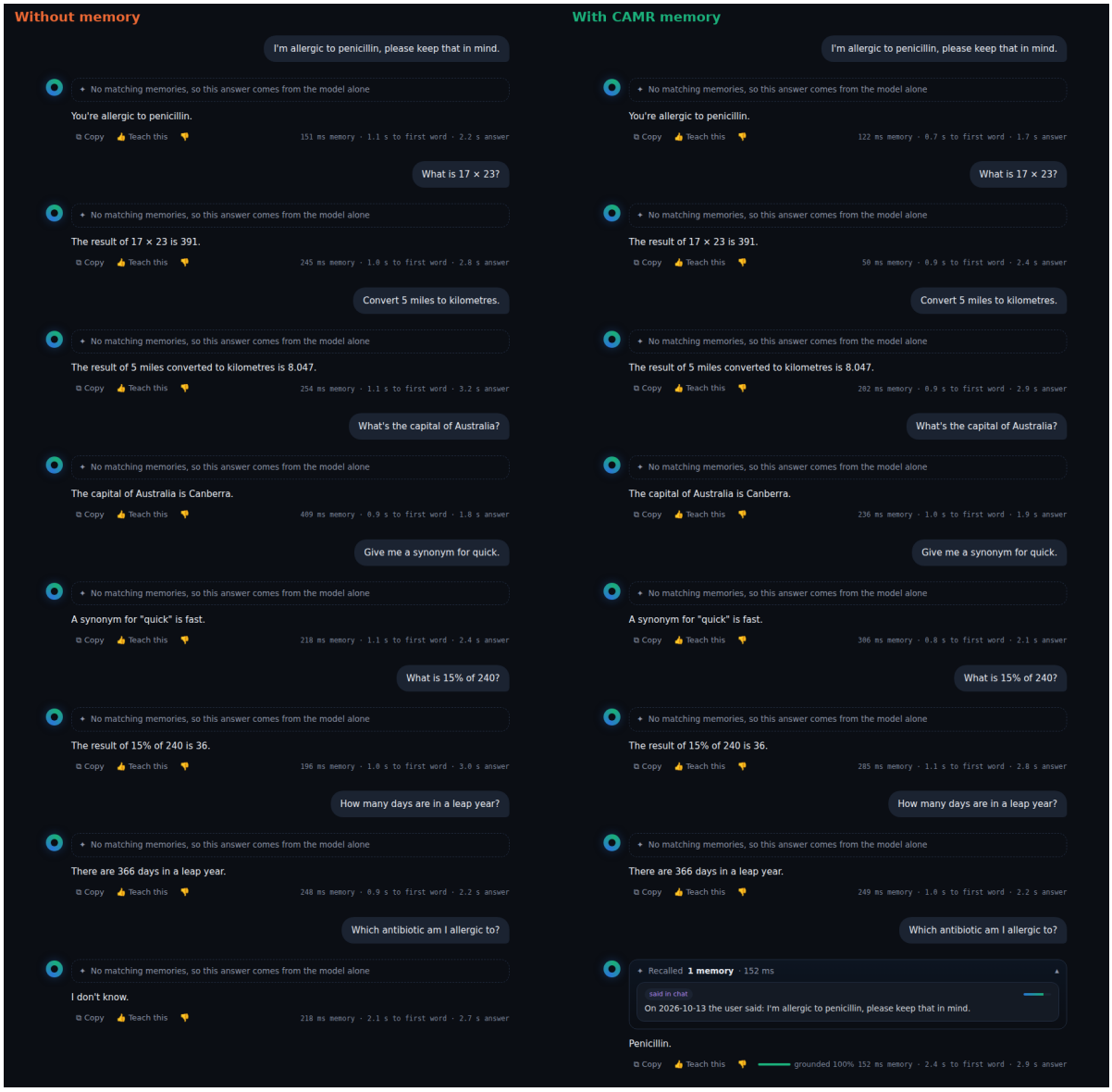


Figure 5.57: Scenario: remembering beyond the context window (left: without memory, right: with CAMR)

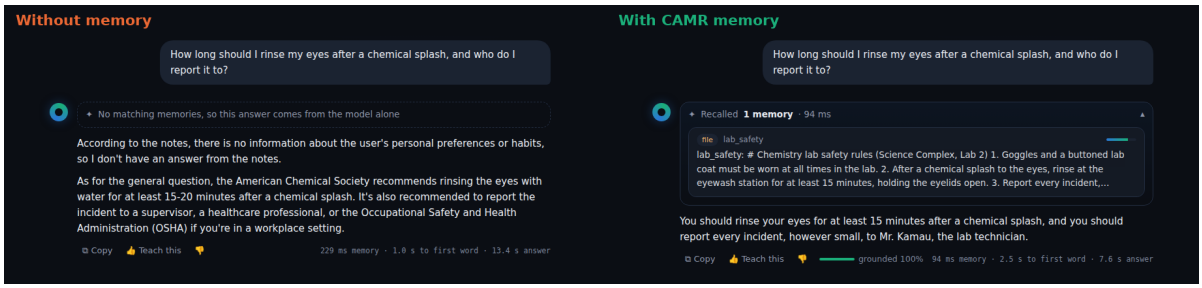


Figure 5.58: Scenario: answering from a taught document (left: without memory, right: with CAMR)

With memory the assistant answered 5 of 5 scenarios correctly; without it, 0 of 4. In detail, from the logs: (1) the two-note answer was assembled from two notes that similarity found on its own (0.72 and 0.62); bridging (following a name found in one note to another note about

that name) was not needed, because the question's words matched both, so this scenario shows multi-note recall, not bridging (bridging is shown on the benchmark examples of section 5.9.7). (2) The model computed 89,200 − 86,950 = 2,250 km from a mileage it had only in memory. (3) After *remember that the dentist appointment moved…*, both the old and the new note were recalled (similarity 0.85 and 0.80) and the model chose the new one because its wording says the appointment moved; CAMR has no explicit mechanism for superseding a fact (recency is off by default), so this relies on the model and is listed as future work. (4) The allergy was recalled from chat memory seven turns later, after the six-turn history window had dropped it, while every general question in between was answered from the model's own knowledge with no memory recalled. (5) Without memory, the model gave a generic rinse time from its own knowledge but could not know the lab technician's name; with the taught file it answered both parts.

5.9.15 Learning in the Application: Live Scenarios

> **In plain words.** The app learning in use: looking things up, installing knowledge packs and tidying its memory, recorded as it happened.

The four behaviours of version 0.3 were run in the real application with llama3.2:3b, through the browser, each variant in a fresh home, with the answers logged by the same harness as Section 5.9.14 (`scripts/scenario_screens.py --set v3`). Table 5.46 lists what the model actually answered.

Table 5.46: Live learning scenarios: what the model answered without and with the 0.3 behaviour (last turn)

Scenario	Without (learning off / not taught)	With CAMR 0.3	Correct
Who won the 2026 FIFA World Cup? (after the models' training)	I don't know that yet. Can you tell me which country won the 2026 World Cup final?	Spain.	Yes
How many hours is a 4-point story? (a made-up unit, *kazoru*)	I don't know that yet. Can you tell me how many hours of work a 4-point story in your sprint represents?	According to the notes, one story point is half a kazoru, and a kazoru is three hours of focused work. Therefore, a 4-point story would be 4 x 0.5 = 2	Yes
What is my passport number? (not stored)	–	Your passport number is AK0412345.	Yes
Highest mountain in Kenya (Kenya pack installed), then sleep	–	The highest mountain in Kenya is Mount Kenya, and its highest peak, Batian, is approximately 5,199 meters tall.	Yes

Something newer than the model. With lookups off, the model said it did not know and asked to be told, rather than guess a winner its training could not contain. With lookups on, the first question was looked up (2 notes from Wikipedia stored, each with its link), answered

from them (*Spain won the 2026 FIFA World Cup.*), and the reworded follow-up was answered **from memory with no lookup** (71 ms of retrieval) (Figure 5.59).

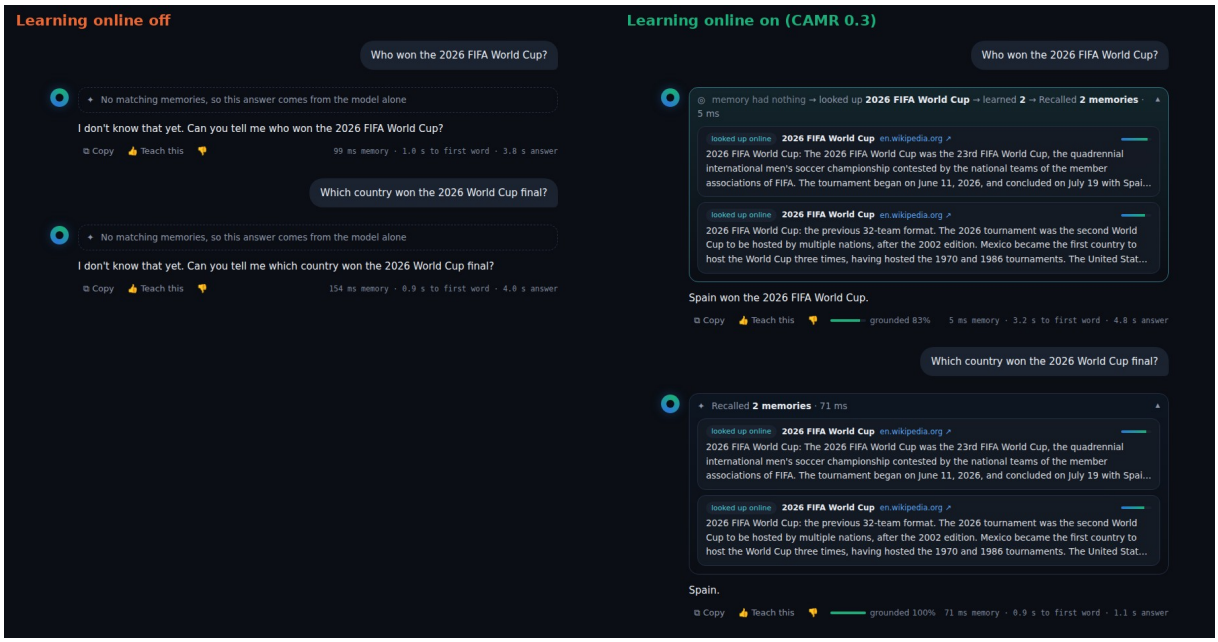


Figure 5.59: Something newer than the model: lookups off (left) and on (right); the second question is answered from what the first one taught

Teaching it a new concept. A unit the model cannot know (*a kazoru is three hours of focused work; one story point is half a kazoru*) was taught once as a note. Asked how long a 4-point story takes, the model with the note reasoned through it: *According to the notes, one story point is half a kazoru, and a kazoru is three hours of focused work. Therefore, a 4-point story would be 4 x 0.5 = 2* Without the note it asked to be taught, and the question, which says *our sprint*, was never sent to the web (Figure 5.60).

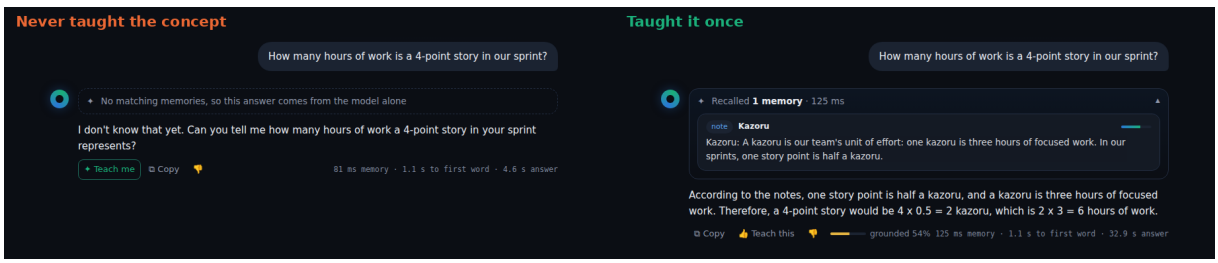


Figure 5.60: A concept taught once and applied: never taught (left) and taught (right)

Asking to be taught. For a personal fact memory did not hold, the reply was *I don't know that yet. Can you tell me your passport number so I can remember it?* with a *Teach me* button; after *remember that my passport number is AK0412345*, the same question was answered: *Your passport number is AK0412345.* (Figure 5.61).

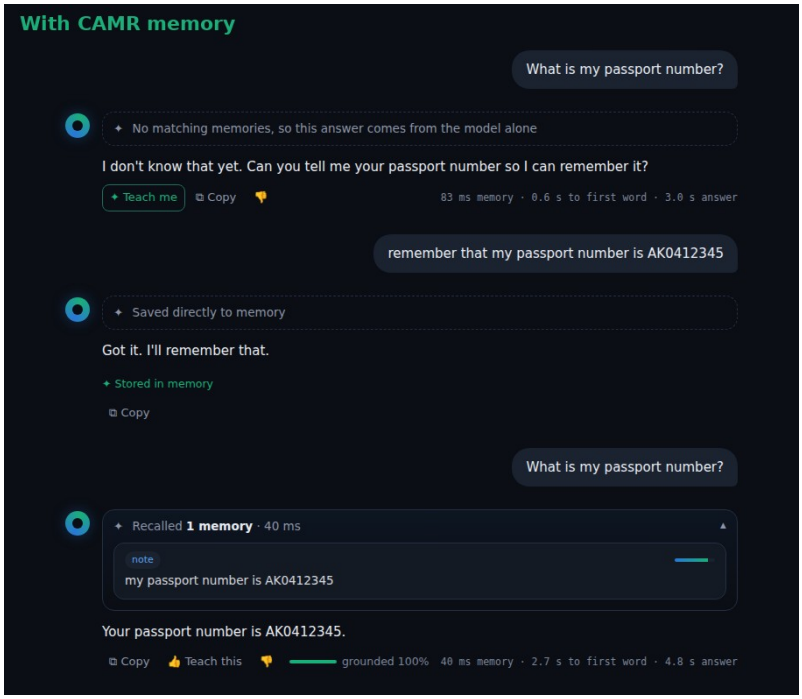


Figure 5.61: A personal gap answered by asking to be taught, then answered from what the user taught

A knowledge pack, then sleep. With the bundled *Kenya* pack installed (its vectors copied, no model run), the question was answered from the pack (*The highest mountain in Kenya is Mount Kenya, and its highest peak, Batian, is approximately 5,199 meters tall.*). Sleep then ran its four steps: it removed 0 duplicates, looked up the names the pack's notes mentioned but memory had no note about, and learned 4 of them (East Africa, Indian Ocean, World Heritage Site, Kenya National Bureau of Statistics); with only 31 notes it built no index and, with no answered questions yet, trained no adapter, both by design (Figure 5.62).

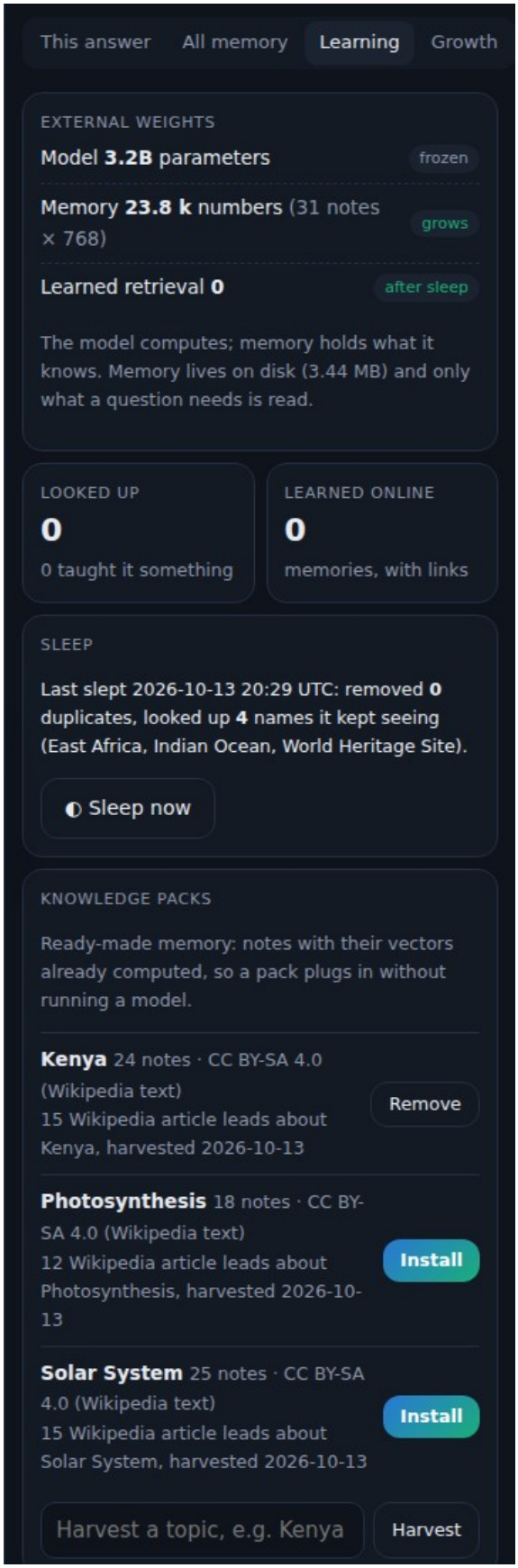


Figure 5.62: The Learning tab after installing a pack and sleeping: external weights (frozen model, growing memory), lookups, the sleep report and packs

5.9.16 Reproducibility

> **In plain words.** Anyone can rerun the main evaluations from the saved files, without the internet or an API key, and get the same numbers. All 16 checks passed, twice.

`python scripts/reproduce.py --twice` recomputes the key results from the committed caches and compares every number with the committed one: the test suite; every development-set lookup; every variant of verified answering on the development, test and held-out sets (each question's status, claim, correctness and first answer); bulk ingest with real embeddings (numbers that capture a text's meaning, used to search by meaning) and recall on 600 questions (83.67% dense, 95.13% hybrid); the governor tests; and every formula run and ablation (switching parts off one at a time to see what each contributes). The last run passed **16 of 16 checks (8 checks, twice in a row) with no API key in the environment and no network**: the cloud model's answers were replayed by a stand-in that builds the identical request. The harness also caught a real fault: two answer runs had seen slightly different web pages because one had fetched what the other missed; freezing the web cache fixed it. It caught a second fault in this increment: the replays after D-32 failed on the test, held-out and formula checks because the D-32 fix had cut titles such as *St. Louis Blues* (D-37). After that was fixed, entity extraction was identical to the pre-D-32 code on all 600 PopQA questions and 440 formula problems, and the replays were rerun.

5.9.17 What Failed, and Why

> **In plain words.** Where the system failed, and why. Every flaw is listed, including bugs that were fixed and weaknesses that remain.

The flaws below were found by the measurements in this chapter. Fixed ones stay listed because they show how the system failed before the fix; open ones are the boundaries of what the evidence supports.

Table 5.47: Flaw catalogue

Flaw	Evidence	Consequence	Status
Memory does not close reasoning gaps	GSM8K residual 83.3 points between 0.5B and 20B reading the same notes; worked examples lowered the 0.5B model from 33.3% to 16.7%	Arithmetic questions need a stronger model or a calculator	Open: routed to the cloud by the learned policy; calculator for formulas
A small reader wastes good evidence	76% of the 0.5B model's multi-hop misses had the answer in context; gains grow with the reader (+23 to +34.5 points)	Multi-hop answers need ≥ 3B	Open: choose the reader by task (section 5.9.4)
The check guarantees support, not truth	All 6 wrong stated held-out answers quote a stored line about another meaning or a source that disagrees with the benchmark	A wrong page in memory gives a wrong, sourced answer	Open: the source is shown with every answer
Checks too strict for	*al-Ghazali* withheld (kind check expects a	Lower coverage	D-33 fixed; the kind

Flaw	Evidence	Consequence	Status
some correct answers	capital); 14 of 45 answerable desk tickets withheld before D-33		check is open
CAMR's recall has no reranker	Answer in what the reader got: 64% with CAMR against 72% with reranked RAG on the same corpus (section 5.9.5)	Lower coverage than the best RAG pipeline	Open: add the cross-encoder to CAMR's recall
The check trusts a claim that only names the subject or another of its attributes	0.5B desk replies *Lumen LB-13* for *Which cable …?* and *10-year firmware* for the latest firmware were stated; no known relation covers *cable* or *firmware*	4 wrong desk replies with the 0.5B reader (none with 3B or Kimi K3)	Open: require a claim word beyond the subject's own name
Small readers hedge under the standard RAG prompt	Llama 3B: *I don't know where RMA numbers start, but … they start with HLX-*	14 answerable desk tickets escalated by the 3B + RAG system	Reported; prompt-specific
The lookup sets the ceiling	Answer in the fetched text for 52% (stream) to 72–73% (held-out, development) of questions	Up to a quarter of questions cannot be answered from memory	Open
Retrieval share of answer time (NFR-02)	56% for the 0.5B model in the pilot profile; about 20% after core partitioning	Target ≤ 10% met only for models whose generation takes ≥ 450 ms	Open for 0.5B
Recency and importance scoring	−1.2 and −5.4 points of full-support recall	Proposal's Eq. 4.1 harmful on independent questions	Removed (D-06); untested on conversations
Learned retrieval components	Query adapter never beat the identity on unseen questions; the learned index was no faster than exact search at equal recall	No benefit from training retrieval here	Kept off; accepted only if held-out recall rises
Some formulas beyond the 0.5B model	Arithmetic series and Heron's formula never applied; combinations 50% → 0% after correction (copied the prompt's sample)	Small models need the rule and the arithmetic done for them	Open; the grounding check flags most copied answers
Fine-tuning wins on the trained wording	LoRA 88.3% against memory 50.0% held-out on formulas	Memory is weaker on exactly the trained format	Reported; LoRA lost GSM8K (13.3% → 0%) and derived problems
Entity gate is rule-based	D-15, D-31, D-32, D-37, D-40: possessives, instructions, terse phrasing, abbreviations and requests; HotpotQA −1.2 points	Wrongly withheld notes	Fixed cases; a learned recogniser is future work
Run-to-run variation of the 0.5B runtime	Answers differed on 10 of 30 questions between identical runs (wrong → wrong)	Small-model floors vary by a few points	Open (runtime); engine prompts are identical
Evaluation defects in the automations	D-34: the news workflow used the support persona; D-35: a decline phrased *I can't identify* was sent as a reply; D-36: Wikimedia refused repeated live fetches; D-38: a curly apostrophe hid declines; D-39: a loop asked each news question 30 times	Baselines would have been scored unfairly; a workflow stopped	Fixed and every affected run repeated before reporting

5.9.18 Limitations, Delimitations and Threats to Validity

> **In plain words.** What the results do not show. They hold for English questions, models running on 4-core computers, the numbers of questions reported, an invented company's data, and a cloud model that does not give the same answer twice.

Limitations are weaknesses of the evidence; each is stated with its likely effect.

i. **Sample sizes.** Headline results use 200 questions (one question = 0.5 points), the desk 60 tickets (1.7 points), the agent 20 tasks (5 points) and the pilot 30 questions (3.3

points). Differences of a few points between systems on the desk or the agent are not significant.

ii. **One cloud sample.** Kimi K3 accepts only temperature (how random the model's word choices are) 1, so each cloud answer is one sample; repeated runs would move its scores by a few points.

iii. **The judge is a cloud model.** Kimi K3 judged answers that exact matching marked wrong. It also appears as a system under test, so it could favour its own phrasing; the exact-match scores, which give the same ordering, are kept beside every judged score.

iv. **Data written for this study.** The Helio company, its tickets and agent tasks were written by the researcher, and the news questions by Kimi K3 from single items. They test the mechanism (private, changing knowledge; events after training), not the difficulty of a real company's support queue.

v. **Timings.** Machines were 4-core cloud containers, not a physical laptop, and some runs shared the CPU with others; timings are reported only from runs that had the CPU to themselves, and cached answers keep their original timing.

vi. **Costs** are computed at Moonshot's list price without cached-input discounts, so they are upper bounds; the local runs' electricity and hardware were not costed.

vii. **No user study.** The application was tested for behaviour and accessibility, not for usefulness to people.

Delimitations are boundaries chosen for the study: CPU-only consumer devices (no GPU, no phones); English questions; frozen, quantised open models served by Ollama, with fine-tuning used only as a baseline (what it is compared against); text only; self-hosted n8n run headless (from the command line, without the visual editor), with no workflow exposed to the internet; no real customer or personal data; a research prototype, not a production service.

Threats to validity. *Internal:* answer caching makes replays exact, but a cached cloud answer is the answer of the day it was asked; methods were chosen on development data only, and the held-out set was answered once. *Construct: hallucination* here means a wrong answer stated as fact, judged against the benchmark's gold answers, which are themselves sometimes disputable (section 5.9.3). *External:* PopQA's templated questions and an invented company are cleaner than real traffic. *Conclusion:* intervals are reported where they matter, and several differences in this chapter lie within them.